

Confidence-Aware Episodic Memory with Self-Improvement for Progressive Hallucination Reduction in Large Language Models

by

Md. Aksan Gony Alif
24341256

Maliha Binte Shamim
23101466

Md. Mahmudur Rahman
22101355

A thesis submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
B.Sc. in Computer Science and Engineering

Department of Computer Science and Engineering
Brac University
February 2026.

© 2026. Brac University
All rights reserved.

Declaration

It is hereby declared that

1. The thesis submitted is my/our own original work while completing degree at Brac University.
2. The thesis does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
3. The thesis does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.
4. We have acknowledged all main sources of help.

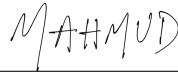
Student's Full Name & Signature:



Md. Aksan Gony Alif
ID: 24341256



Maliha Binte Shamim
ID: 23101466



Md. Mahmudur Rahman
ID: 22101355

Approval

The thesis/project titled “Confidence-Aware Episodic Memory with Self-Improvement for Progressive Hallucination Reduction in Large Language Models ” submitted by

1. Md. Aksan Gony Alif (24341256)
2. Maliha Binte Shamim (23101466)
3. Md. Mahmudur Rahman (22101355)

of Fall, 2025 has been accepted as satisfactory in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science in Spring 2026.

Examining Committee:

Supervisor:
(Member)

Dr. Md. Golam Rabiul Alam

Professor
Department of Computer Science & Engineering
Brac University

Co-Supervisor:
(Member)

Md. Tanzim Reza

Senior Lecturer
Department of Computer Science & Engineering
Brac University

Thesis Coordinator:
(Member)

Dr. Md. Golam Rabiul Alam

Professor
Department of Computer Science & Engineering
Brac University

Head of Department:
(Chair)

Dr. Sadia Hamid Kazi

Associate Professor & Chairperson
Department of Computer Science & Engineering
Brac University

Abstract

Large Language Models produce fluent but factually incorrect responses at rates reported up to 40% in open-domain question answering, and higher in specialised domains. This problem limits their usage in high-stakes settings. Current models cannot improve on their own because they operate statelessly, without any verified knowledge accumulation across interactions. This research proposes CAEM (Confidence-Aware Episodic Memory with Self-Improvement), a system that progressively reduces hallucinations through verified episodic storage and constrained iterative fine-tuning on the verified episodes. CAEM integrates three core mechanisms. First, a verified episodic memory populated only through a four-outcome storage decision (STORE, DEFERRED, ABSTAIN, DISCARD) driven by a nine-signal post-generation verifier organised in three complementary families: internal calibration (token probability, MC-dropout variance, internal-state probe), sample-set agreement (average pairwise chain similarity, chain-to-answer entailment, normalised semantic entropy), and external grounding (weakest-link and mean atomic-fact entailment, ensemble contradiction probability). A bounded FIFO buffer holds DEFERRED entries for reconsideration at the next cycle boundary. Second, a confidence-aware three-tier router that combines memory-match similarity with stored verifier confidence into a single dispatch score, and an independent safety override on a pre-routing confidence signal that forces retrieval-augmented generation whenever pre-routing confidence falls below a fixed floor, regardless of memory-match quality. Third, a constrained self-improvement loop that, at each cycle boundary, retroactively re-verifies every stored episode under the updated model (upward-only updates), reconsiders deferred entries, and fine-tunes under an L_2 anchor toward the base-model parameters as an EWC approximation; a hard out-of-distribution retention guard on MMLU rolls back the weights (but not the memory) whenever general capability erodes beyond a fixed floor. The system targets a substantial reduction in confident-error rate, defined as the fraction of stored, high-confidence responses that are incorrect, measured against a zero-shot Flan-T5-Large baseline on a six-benchmark factual-QA panel (FEVER, TriviaQA, Natural Questions, TruthfulQA, StrategyQA, ARC-Challenge) with MMLU as the out-of-distribution retention control; the specific quantitative reduction is reported in the results chapter. The study also argues, under a data-purity analysis, that even an imperfect verifier can drive monotonic improvement across cycles when the training-pool purity threshold is set above the storage threshold.

Keywords: Large Language Models, Hallucination Reduction, Episodic Memory, Confidence Estimation, Self-Improvement, Verification Systems, Continual Learning

Table of Contents

Declaration	i
Approval	ii
Ethics Statement	iv
Abstract	iv
Dedication	iv
Acknowledgment	iv
Table of Contents	v
List of Figures	vi
Nomenclature	vii
1 Introduction	1
1.1 Background	1
1.2 Rationale of the Study or Motivation	2
1.3 Problem Statement	3
1.4 Objective	4
1.5 Methodology in Brief	6
1.5.1 System Architecture Overview	7
1.5.2 Query Processing and Routing	7
1.5.3 Post-Generation Verification and the Four-Outcome Storage Decision	9
1.5.4 Constrained Self-Improvement Loop	9
1.5.5 Implementation	10
1.6 Scopes and Challenges	10
1.6.1 Research Scope	10
1.6.2 Technical Challenges	12
1.6.3 Mitigation Strategies	13
2 Literature Review	15
2.1 Preliminaries	15
2.1.1 Large Language Models and Transformer Architecture	15
2.1.2 Hallucination Taxonomy	15
2.1.3 Natural Language Inference	16

2.1.4	Sentence Embeddings and Semantic Similarity	16
2.1.5	Self-Consistency Decoding	17
2.1.6	Catastrophic Forgetting and Continual Learning	17
2.2	Review of Existing Research	18
2.2.1	Hallucination Problem and Measurement	18
2.2.2	Verification Methods	20
2.2.3	Confidence Estimation	22
2.2.4	Memory-Augmented Architectures	24
2.2.5	Self-Improvement and Continual Learning	26
2.2.6	External Baselines and Competitive Positioning	29
2.2.7	Evaluation Benchmarks	30
2.3	Summary of Key Findings	32
3	Requirements, Impacts and Constraints	35
3.1	Introduction	35
3.2	Functional Requirements	36
3.2.1	FR1: Pre-Routing Confidence Estimation	36
3.2.2	FR2: Adaptive Three-Tier Routing	36
3.2.3	FR3: Multi-Signal Post-Hoc Verification	37
3.2.4	FR4: Four-Outcome Storage Decision	38
3.2.5	FR5: Episodic Memory Management	38
3.2.6	FR6: Retroactive Re-Verification	39
3.2.7	FR7: Constrained Self-Improvement	39
3.2.8	FR8: Ablation and Evaluation Instrumentation	40
3.3	Non-Functional Requirements	40
3.4	System Constraints	41
3.5	Evaluation Methodology	42
3.5.1	Composite Evaluation Score	42
3.5.2	Diagnostic Metrics	43
3.6	Ablation Study Design	43
3.7	Risk Analysis and Mitigation	44
3.8	Ethical Considerations and Broader Impact	45
3.9	Chapter Summary	46
4	Methodology	47
4.1	Introduction	47
4.2	System Overview	48
4.2.1	Stage-by-stage narrative	48
4.2.2	Worked example	52
4.3	Pre-Routing Confidence Estimation	53
4.4	Episodic Memory	56
4.5	Three-Tier Router	59
4.6	Unified Verifier	62
4.7	Four-Outcome Storage Decision	68
4.8	Retroactive Re-Verification	72
4.9	Deferred-Entry Reconsideration	74
4.10	Self-Improvement Loop	78
4.11	Theoretical Analysis	82
4.11.1	Data Purity Theorem	82

4.11.2	Monotonicity of Purity	83
4.11.3	Convergence of the Self-Improvement Recurrence	84
4.12	Implementation	85
4.13	Chapter Summary	87
5	Results and Analysis	90
5.1	Experimental Setup	90
5.1.1	Benchmarks	90
5.1.2	Metrics	90
5.1.3	Baselines	94
5.1.4	Implementation Details	95
5.2	Ablation Methodology	97
5.3	Main Results	101
5.4	Ablation Results	104
5.5	Chapter Summary	104

List of Figures

1.1	CAEM Architecture (High-Level View)	8
1.2	Experimental Workflow	11
4.1	CAEM pipeline overview (S1–S8): each query passes through confidence estimation, memory retrieval, routing, tier execution, verification, and storage decision. Stage 8 runs once per cycle boundary for retroverification and L_2 -anchored self-improvement.	49
5.1	Reliability diagram on pooled $(u_{\text{stored}}, \text{correct})$ pairs at Cycle N , with ten equal-width confidence bins. Departure from the $y = x$ identity line quantifies miscalibration; the ECE value in Table 5.4 is its weighted L_1 norm.	106
5.2	Pool-purity trajectory across cycles $0, \dots, N$. Solid line: measured P_c from the purity-validation split; dashed line: envelope predicted by Theorem 4.5 and Theorem 4.6.	106

List of Algorithms

4.1	Stage 1 pre-route confidence u_{pre}	55
4.2	Stage 3 three-tier routing decision.	62
4.3	Stage 5 unified verifier.	68
4.4	Stage 6 decision tree and Stage 7 action.	71
4.5	Stage 8 retroactive re-verification (runs once per cycle boundary after successful fine-tuning).	74
4.6	Stage 8b deferred-entry reconsideration (runs once per cycle bound- ary, after retroverify, on non-aborted cycles).	77
4.7	Stage 8 self-improvement cycle (one invocation per cycle boundary). .	81

Chapter 1

Introduction

1.1 Background

Large Language Models (LLMs) have displayed exceptional proficiency across various NLP (Natural Language Processing) tasks such as answering questions, summarizing, generating codes and reasoning. Models such as GPT-4, Claude and PaLM achieve impressive performance on usual benchmarks, with in some cases, even exceeding human-level accuracy on certain tasks **openai2023gpt4**, **anthropic2023claude**, **anil2023palm**. However, a critical limitation hampers their deployment in high-stakes applications, for example, the generation of plausible yet factually incorrect responses, commonly known as *hallucinations*.

In LLMs, hallucination occurs when models produce outputs that contradict source material, conflict with valid world knowledge, or even fabricate information entirely. Recent studies reveal concerning statistics about hallucination occurrences. The TruthfulQA benchmark, introduced by Lin et al. **lin-et-al-2022-truthfulqa**, is designed to particularly measure how models copy human falsehoods. The worst part is that it found larger models performing worse than small ones. For example, GPT-3 175B achieved only 58% on truthfulness alone and just 21–25% on the combined truthful-and-informative metric, compared to 94% human accuracy on both metrics, demonstrating an *inverse scaling* phenomenon where increased model size does not guarantee improved truthfulness. Also, another benchmark, FActScore by Min et al. **min-et-al-2023-factscore**, developed for fine-grained evaluation of long-form generation, shows that ChatGPT achieved only 58% supported atomic facts in biography generation. Domain-specific studies show even higher error rates. For example, medical literature analysis by Alkaissi and McFarlane **alkaissi2023artificial** found hallucination rates of 69-77% in ChatGPT responses to clinical questions.

The fundamental challenge expands beyond accuracy benchmarks. Current LLMs operate statelessly where each query is processed independently without learning from previous interactions or collecting verified knowledge. This stateless architecture results in three critical limitations. Firstly, *no memory of successful reasoning patterns*, which requires generation from scratch for every query. Secondly, *poor confidence calibration*, where models show high confidence even while being incorrect **kadavath2022language**. Lastly, *no self-improvement mechanism*, which causes indefinite repetition of identical errors without adapting.

All these limitations have significant practical consequences. For example, in educational sectors, students may learn incorrect information presented confidently. In

healthcare sectors, hallucinated medical information can lead to harmful decisions. In enterprise knowledge management, fake citations and sources can hamper trust. The stateless nature also causes the repetition of inefficient-expensive generation for similar queries rather than using past verified reasonings.

This research addresses these fundamental limitations through CAEM (Confidence-Aware Episodic Memory with Self-Improvement), a system that targets a substantial reduction in confident hallucination through three architectural commitments. First, a *verified episodic memory* that admits a reasoning chain only after a nine-signal post-generation verifier has passed it through a four-outcome decision (store, defer, abstain, or discard), so that stored knowledge is distinguished from marginal or uncertain output rather than collapsed into a single binary gate. Second, a *confidence-aware three-tier router* that combines stored-answer quality with query-to-memory similarity and overlays an independent safety override driven by a pre-routing uncertainty estimate, so that forced grounding is available before generation has started rather than only after it has failed. Third, a *constrained self-improvement loop* that fine-tunes on the high-quality training pool under an L_2 anchor regulariser and aborts with a weight rollback if an out-of-distribution retention probe drops below a fixed ratio, so that accumulation across cycles is compounding rather than drifting. The quantitative hallucination-reduction target is stated relative to a zero-shot Flan-T5-Large baseline on a multi-benchmark factual-QA panel using the confident-error-rate diagnostic defined in Chapter 3; the specific value is reported in Chapter 5.

1.2 Rationale of the Study or Motivation

The motivation for this research is that there is a wide gap between what LLM are capable of and what is required when they are used. While current models are capable of writing fluent, relevant text with excellent capacity, they usually cannot distinguish between *real facts* and *confabulation*, which makes it difficult to apply them in domains requiring precision.

The existing approaches to decrease the number of hallucinations could be classified into three categories, with all of them possessing considerable limitations. Firstly, *Retrieval-augmented generation* (RAG) methods **NEURIPS2020'6b493230**, which looks up documents and uses them for generation. But, the model's answer is as good as the documents it retrieves and only depends on retrieval quality, not experience. Secondly, *Prompting techniques* such as chain-of-thought **NEURIPS2022'9d560961** and self-consistency **wang2023selfconsistency** makes the model think more carefully and checks multiple reasoning paths, but cannot remember information later and each session starts with no knowledge stored. Thirdly, *post-generation verification methods* **manakul-et al-2023-selfcheckgpt**, **farquhar2024semantic**, which check the answer after it is generated and can detect hallucinations after they occur, but do not teach the model what to do next time to reduce mistakes. So the same hallucinations can happen again in the future.

The key insight driving this research is that hallucination reduction requires *architectural intervention* rather than incremental improvements to existing methods. Specifically, three capabilities are essential but absent from current systems. First, models need *verified episodic memory* whose storage decision is not a single binary threshold but a four-outcome cascade, so that an answer which is plausible but has not cleared the commit bar is deferred for a second pass rather than committed

to Tier 1 retrieval or discarded outright. Second, models require *confidence-aware routing* that reads pre-routing uncertainty as an independent safety signal alongside a combined similarity-and-stored-quality score, so that low model readiness triggers forced external grounding before generation has started rather than only after an overconfident answer has propagated through the pipeline. Third, systems must support *constrained self-improvement*, in which fine-tuning on a high-quality training pool is bounded by a regulariser that pulls weights toward the base model and an out-of-distribution retention probe that aborts the cycle if general capability has eroded, so that knowledge accumulation compounds across cycles rather than drifting into the benchmark at the expense of the base capability.

This research is significant for both theoretical and practical reasons. Theoretically, it addresses the open question of whether imperfect verification can enable monotonic improvement in self-training systems. Prior work in semi-supervised learning demonstrates that noisy labels can degrade model performance **natarajan2013learning**, yet our preliminary analysis suggests that when base model accuracy exceeds random chance, verified data purity can exceed verification accuracy itself through favorable base rate dynamics. This counterintuitive property, if validated empirically, provides theoretical justification for self-improvement under realistic verification constraints. Practically, a substantial reduction in the confident-error rate of a 780M-parameter model would make language models much more useful without hampering their overall abilities. This would allow educational systems to safely learn and remember specific institutional knowledge. Also, it would help businesses gradually build reliable domain expertise through verified interactions. In healthcare and law domains, it would preserve language understanding while maintaining strict accuracy standards. Additionally, using memory to reuse past answers would significantly lower computation costs, making AI systems more affordable and sustainable. The specific reduction achieved by CAEM is reported in Chapter 5 relative to a zero-shot Flan-T5-Large baseline under the confident-error-rate diagnostic defined in Chapter 3.

The undergraduate thesis scope is appropriate because the main ideas can be tested using well-known models like *Flan-T5-Large*, a factual-QA benchmark panel (*FEVER*, *TriviaQA*, *Natural Questions*, *TruthfulQA*, *StrategyQA*, and *ARC-Challenge*) with *MMLU* as an out-of-distribution retention control, and manageable computational resources compatible with a single RTX 4090 (24 GB VRAM). Three self-improvement cycles are sufficient to evaluate whether the memory accumulates under the retention guard without requiring the 7–10 cycles that a long-horizon equilibrium study would need. A 12-month timeline is enough to properly evaluate the system while focusing only on factual and knowledge-based tasks, where correctness can be clearly checked, and excluding creative tasks that are harder to evaluate objectively.

1.3 Problem Statement

Current large language models suffer from three fundamental limitations that prevent their reliable deployment in knowledge-intensive applications which requires factual accuracy:

- **Stateless Operation Without Knowledge Accumulation:** LLMs process each query independently without retaining knowledge from previous interac-

tions. Even if the model thinks carefully and gets the answer right, all the thinking is discarded once the answer is generated. So, if a similar question is asked again, the model must redo all the thinking again from the beginning. This stateless architecture causes two problems. Firstly, it wastes time as well as computing power by repeating the work. Secondly, the model might give a completely different answer next time, even for the same question. The absence of episodic memory prevents it from learning any domain-specific knowledge as each interaction is separated and does not help the model to get better over time.

- **Poor Confidence Calibration and Uncertainty Quantification:** Modern neural networks are often more confident than they should be, meaning they sound sure even when they are wrong. Guo et al. [pmlr-v70-guo17a](#) shows that even when the model is correct only 70% of the time, it may act like it is 90% sure. This mismatch is dangerous because people may trust wrong answers too much. This problem shows up in two main ways. Firstly, the model gives a wrong answer very confidently, so users do not realize they should double-check it. Secondly, the model sometimes gives a correct answer but may sound unsure, which causes additional checking that was not required. If the model can not judge its own confidence properly, it can not decide if a quick lookup is enough or when deeper reasoning is needed. Current single-signal confidence methods achieve only 0.50-0.60 AUROC for telling whether an answer is right or wrong [xiong2023can](#), barely above random chance.
- **Absence of Self-Improvement Mechanisms:** Deployed LLMs remain static after training, unable to learn from mistakes or adapt to domain-specific patterns. If a model hallucinates, it will keep making the same mistake forever unless humans retrain it again. Trying to teach the model new things through naive fine-tuning causes a big problem called *catastrophic forgetting* [doi:10.1073/pnas.1611835114](#), where learning new information causes 30-50% degradation of already acquired capabilities. The absence of self-improvement means that verification efforts provide no long-term value, for example, verifying 1000 answers today provides no benefit tomorrow, as the system cannot combine verified knowledge into improved generation.

These three limitations make each other worse, creating systems that remain simultaneously overconfident and unreliable, expensive yet static, capable yet ultimately unsuitable for high-stakes deployment. This research addresses all three simultaneously through architectural innovation: verified episodic memory with a four-outcome storage decision for knowledge accumulation, confidence-aware three-tier routing with a pre-routing safety override for reliable dispatch, and constrained self-improvement under an L_2 anchor regulariser and an out-of-distribution retention guard that aborts the cycle with weight rollback if general capability has eroded.

1.4 Objective

The primary objective of this research is to develop and evaluate CAEM (Confidence-Aware Episodic Memory with Self-Improvement), a system that reduces confident

hallucination in a 780M-parameter instruction-tuned language model through accumulated verified knowledge and constrained iterative fine-tuning. This comprehensive goal decomposes into seven specific objectives:

- **Design a Verified Episodic Memory with a Four-Outcome Storage Decision:** Develop an episodic memory architecture that admits a question-reasoning-answer tuple only after a post-generation verifier has emitted one of four outcomes (store, defer, abstain, or discard) based on the composite verification score, an atomic-level grounding pass, and an explicit contradiction veto. Defer entries that are plausible but have not cleared the commit bar into a bounded first-in-first-out buffer with a time-to-live, so that a later cycle can reconsider them rather than discarding the signal. Implement value-score pruning combining recency, retrieval frequency, and success rate, with a novelty filter that suppresses near-duplicate writes.
- **Develop Pre-Router Confidence Estimation Independent of Generation:** Implement an encoder-only confidence estimate formed by fusing a token-probability aggregate over a short greedy prefix and an encoder convergence ratio that measures how much the late encoder layers disagree with the early layers on the same input. Temperature-scale the fused scalar against a held-out calibration split so that reported confidence matches empirical accuracy. The estimate is paid per query regardless of tier and is used independently of the stored-answer quality score to drive the safety override in the router.
- **Implement Three-Tier Adaptive Routing with an Independent Safety Override:** Design a routing rule that combines query-to-memory similarity with the stored-answer quality of the nearest neighbour into a weighted combined score and dispatches to Tier 1 (retrieve), Tier 2 (zero-shot generate), or Tier 3 (retrieval-augmented generate) based on three thresholds. Overlay an independent safety override: if the pre-routing confidence falls below a fixed floor, force Tier 3 regardless of the combined score, so that forced grounding is available before generation has started and is not entangled with the quality of memory the query happens to land near.
- **Implement a Nine-Signal Post-Generation Verifier Organised into Three Complementary Families:** Combine three internal-calibration signals (token-probability aggregate on the generated answer, MC-dropout plurality, and a fused internal scalar), three sample-set signals (average pairwise similarity across sampled chains, chain-to-answer entailment, and normalised semantic entropy following Farquhar et al. [farquhar2024semantic](#)), and three external-grounding signals (max-match and mean-match entailment against retrieved evidence, and an ensemble-max contradiction probability) into a single composite score. Gate the composite with a contradiction veto and a confabulation early-exit rule, and augment the grounding score with a weakest-link atomic-fact decomposition to catch partially-unsupported answers.
- **Implement Retroactive Re-Verification and Deferred-Entry Reconsideration as Cycle-Boundary Memory Update Passes:** Because three

of the nine verifier signals read the model’s internal state directly, stored composites are model-relative snapshots that drift with every fine-tune. At each cycle boundary, re-score every stored episode under the updated model using an upward-only update rule that promotes entries whose composite has improved past a re-verification threshold and categorically prunes those that have fallen below it. Run a companion pass over the deferred buffer that promotes entries now above the store threshold, drops those that exceed the time-to-live, and requeues the rest. Both passes are abort-guarded and expose per-cycle counter diagnostics.

- **Implement Constrained Self-Improvement with a Retention Guard and Weight Rollback:** Curate the training pool from entries that pass a strict quality filter (a training threshold set above the store threshold) so that training-pool quality is held to a higher bar than Tier 1 retrieval quality. Fine-tune on a 90/10 mix of verified reasoning chains and general-QA pairs under an L_2 anchor regulariser toward the base-model parameters as an EWC approximation [doi:10.1073/pnas.1611835114](https://doi.org/10.1073/pnas.1611835114). Probe MMLU before and after fine-tuning in each cycle and abort the cycle with a weight rollback if the retention ratio falls below a fixed floor; the memory is never rolled back, so knowledge accumulation survives abort events even when weights do not.
- **Evaluate on a Factual-QA Panel with an Out-of-Distribution Retention Control and a Geometric Composite Score:** Evaluate on FEVER, TriviaQA, Natural Questions, TruthfulQA, StrategyQA, and ARC-Challenge as the primary factual-QA panel, with MMLU as the out-of-distribution retention control. Report a Composite Evaluation Score defined as the geometric mean of five axes (answer accuracy, epistemic quality, retention, calibration, and verification reliability) and a confident-error-rate diagnostic measuring the fraction of responses that are both high-confidence under the stored-quality score and incorrect under the benchmark’s correctness metric. Compare against zero-shot Flan-T5-Large, a RAG-only baseline, and per-mechanism ablations of the verifier composite, the router overlay, the retroactive re-verification pass, and the L_2 anchor.

These objectives collectively address the three fundamental limitations identified in the Problem Statement. Objectives one through four provide the per-query mechanism: verified episodic memory with a four-outcome storage gate, pre-routing confidence, three-tier routing with a safety override, and a nine-signal post-generation verifier. Objectives five and six provide the cycle-boundary mechanism: retroactive re-verification, deferred reconsideration, and constrained self-improvement under a retention guard. Objective seven provides the evaluation framework that tests whether the joint system improves along the axes the design optimises for while preserving the out-of-distribution capability the guard protects.

1.5 Methodology in Brief

This section provides a high-level overview of the CAEM system architecture. Chapter 4 develops each stage in full, with equations, algorithms, and hyperparameters.

1.5.1 System Architecture Overview

CAEM organises a single query into an eight-stage pipeline that is traversed once per query and wrapped in a per-cycle update loop. Stage 1 produces a *pre-routing confidence* u_{pre} from an encoder-only pass, fusing a short-prefix token-probability aggregate with an encoder convergence ratio. Stage 2 encodes the query with Sentence-BERT and searches a FAISS-indexed episodic memory for the nearest stored entry, returning a similarity score s and the neighbour’s stored-quality score u_{stored} . Stage 3 dispatches the query to one of three tiers using the combined routing score $\mathcal{S} = 0.70 s + 0.30 u_{\text{stored}}$, overlaid with an independent safety override that forces Tier 3 whenever u_{pre} falls below a fixed floor. Stage 4 executes the tier: Tier 1 serves the neighbour’s stored answer directly; Tier 2 runs a zero-shot generation on the fine-tuned model; Tier 3 retrieves Wikipedia evidence and runs a retrieval-augmented generation. Stage 5 passes the tier output through a nine-signal verifier organised into three families (internal calibration, sample-set agreement, external grounding) to produce the composite u_{stored} . Stage 6 emits one of four outcomes (store, defer, abstain, or discard) from the composite, an ensemble-max contradiction probability, and a max-match grounding score; Stage 7 executes the implied memory and user-facing actions. Stage 8 runs at cycle boundaries: a retroactive re-verification pass refreshes stored composites under the updated model, a deferred-entry reconsideration pass sweeps the deferred buffer, a fine-tuning step accumulates weights under an L_2 anchor, and an MMLU retention probe aborts the cycle with a weight rollback if general capability has dropped below a fixed floor. Figure ?? gives the high-level view; the full per-stage pipeline appears as Figure 4.1 in Chapter 4.

1.5.2 Query Processing and Routing

Every incoming query is first encoded by Sentence-BERT as a 768-dimensional, L_2 -normalised embedding, then looked up in a FAISS index. The lookup returns the similarity s to the nearest stored entry and that entry’s stored-quality score u_{stored} . In parallel, Stage 1 produces the pre-routing confidence u_{pre} from an encoder-only pass that fuses a token-probability aggregate with the encoder convergence ratio and is temperature-scaled against a held-out calibration split. The router combines s and u_{stored} into a single routing score $\mathcal{S} = 0.70 s + 0.30 u_{\text{stored}}$ and dispatches the query to Tier 1 (direct retrieval) when the combined score is high enough that the stored answer can be served without regeneration, to Tier 2 (zero-shot generation on the fine-tuned model) in the middle band, and to Tier 3 (retrieval-augmented generation against Wikipedia) when the combined score is low. An independent safety override is layered on top of this dispatch: if u_{pre} falls below a fixed floor, the query is forced to Tier 3 regardless of the combined score, so that low model readiness triggers forced external grounding before generation has started rather than only after an overconfident answer has been produced. This override is the two-mechanism separation that distinguishes CAEM’s router from a pure similarity-threshold dispatcher. As memory coverage grows across cycles, a greater fraction of queries land on Tier 1, yielding efficiency gains without routing queries that the model is not ready for to the cheapest path.

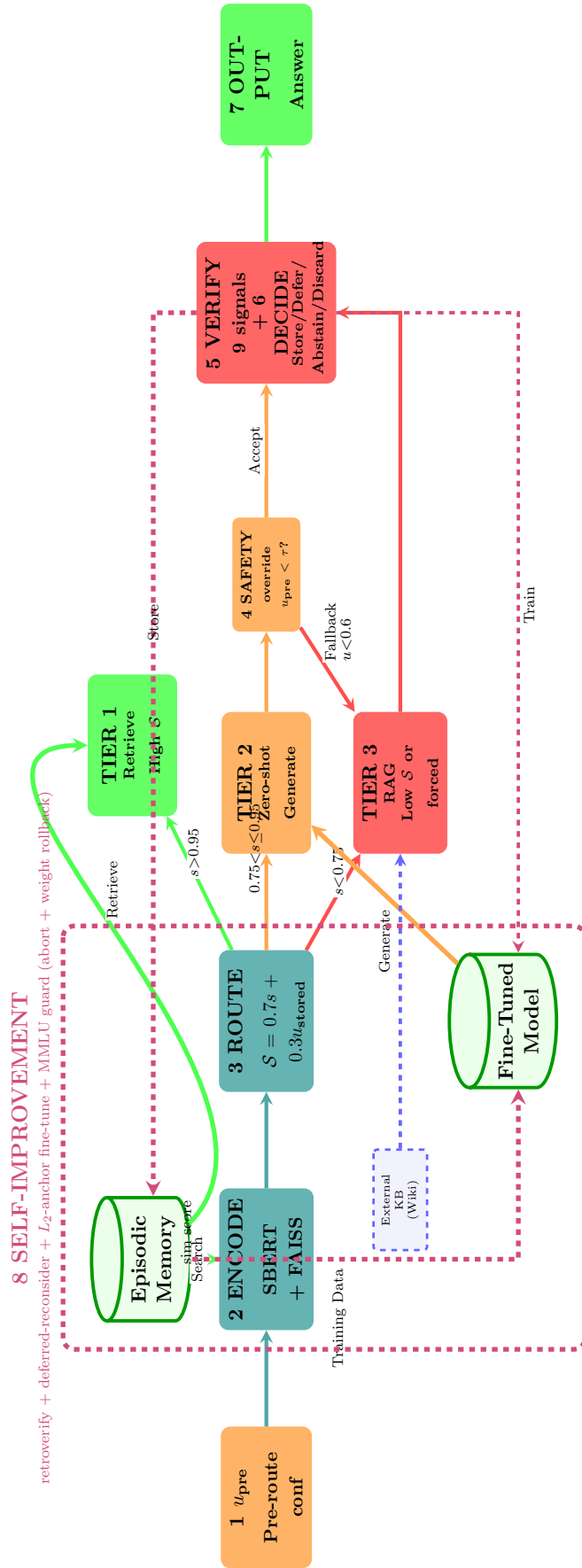


Figure 1.1: High-level view of the CAEM architecture. A pre-routing confidence u_{pre} (Stage 1) and a Sentence-BERT memory lookup (Stage 2) feed a three-tier router (Stage 3) that dispatches to direct retrieval (Tier 1), zero-shot generation (Tier 2), or retrieval-augmented generation (Tier 3) based on a combined similarity-and-stored-quality score $\mathcal{S}$, with an independent safety override that forces Tier 3 whenever u_{pre} falls below a fixed floor. Tier outputs pass through a nine-signal post-generation verifier (Stage 5) and a four-outcome storage decision (Stage 6) before reaching the user (Stage 7). At cycle boundaries (Stage 8), retroactive re-verification, deferred-entry reconsideration, L_2 -anchored fine-tuning, and an MMLU retention guard jointly update memory and weights; failed cycles roll back weights

1.5.3 Post-Generation Verification and the Four-Outcome Storage Decision

All tier outputs undergo a nine-signal post-generation verifier organised into three complementary families, so that no single exploitable signal can drive storage on its own. The *internal calibration* family (token-probability aggregate on the generated answer, MC-dropout plurality **pmlr-v48-gal16**, and a fused internal scalar) measures what the model itself thinks of its output. The *sample-set* family (average pairwise similarity of sampled chains, chain-to-answer entailment, and normalised semantic entropy **farquhar2024semantic**) measures the stability of the answer across stochastic decodes. The *external grounding* family (max-match and mean-match entailment against retrieved evidence, and an ensemble-max contradiction probability) measures whether retrieved evidence supports the claim, augmented by a weakest-link atomic-fact decomposition that catches partially-unsupported answers. The nine signals are combined into a composite score u_{stored} . A confabulation early-exit rule short-circuits storage whenever semantic entropy is high; a contradiction veto rejects any output whose grounding pass returns a high ensemble-max contradiction. The surviving composite feeds a four-outcome storage decision: STORE if the composite is above the store threshold, DEFERRED if it falls in a lower band that is plausible but has not cleared the commit bar, ABSTAIN if the composite is low and grounding is also low (an explicit refusal rather than a silent failure), and DISCARD otherwise. Deferred entries enter a bounded first-in-first-out buffer with a time-to-live and are reconsidered at the next cycle boundary by a dedicated pass that parallels the retroactive re-verification of committed memory.

1.5.4 Constrained Self-Improvement Loop

CAEM implements iterative refinement over three cycles. Each cycle processes a workload of questions through the current system, collecting verified episodes, deferred entries, and contradiction-veto or abstain cases. At the cycle boundary, a retroactive re-verification pass re-scores every stored episode under the updated model with an upward-only rule (promote when the composite has improved past a re-verification threshold, prune when it has fallen below); a deferred-entry reconsideration pass promotes, drops by time-to-live, or requeues each held entry. Fine-tuning then draws a training pool from entries that pass a strict quality filter (a training threshold set above the store threshold, so that training-pool quality is held to a higher bar than Tier 1 retrieval quality), and optimises a mixed batch (90% verified reasoning chains, 10% general-QA) under an L_2 anchor regulariser toward the base model as an EWC approximation **doi:10.1073/pnas.1611835114**. Before and after the fine-tune, the cycle probes MMLU **hendrycks2021measuring** and computes a retention ratio. If the retention ratio falls below a fixed floor, the cycle aborts: weights are rolled back to the previous checkpoint and retroverify is skipped for that cycle. The memory, however, is never rolled back, so verified knowledge accumulated during the aborted cycle survives even when the weights do not. This asymmetry decouples weight-level drift from memory-level accumulation and is what makes the self-improvement loop safe to iterate.

1.5.5 Implementation

The system employs Flan-T5-Large (780M parameters) **JMLR:v25:23-0870** as the only trainable component for its instruction-tuned capabilities and encoder-decoder architecture. Sentence-BERT (all-mpnet-base-v2, 110M, frozen) **reimers-gurevych-2019-sentence** converts queries into 768-dimensional embeddings; FAISS **8733051** backs the episodic memory; RoBERTa-Large-MNLI (355M, frozen) **liu2019roberta** performs the natural-language-inference checks that feed the grounding and entailment signals. All components fit in the 24 GB VRAM of a single RTX 4090 under bfloat16 mixed precision, with fine-tuning made memory-feasible by the L_2 anchor (which requires only the base-model parameters on-device, rather than a full Fisher Information Matrix). Evaluation uses FEVER, TriviaQA, Natural Questions, TruthfulQA, StrategyQA, and ARC-Challenge as the primary factual-QA panel, with MMLU as an out-of-distribution retention control, reporting the geometric Composite Evaluation Score (Chapter 3) and a confident-error-rate diagnostic. The 12-month timeline allocates four months for implementation, four for experimental cycles, and four for analysis and writing.

Figure 1.2 illustrates the planned experimental workflow across three self-improvement cycles. Each cycle runs the per-query pipeline over a fixed workload, then applies the cycle-boundary update passes (retroactive re-verification, deferred-entry reconsideration, L_2 -anchored fine-tuning) under the MMLU retention guard.

1.6 Scopes and Challenges

1.6.1 Research Scope

This research focuses on tasks where there is a clear right or wrong answer and not subjective questions. The system’s target is to handle questions that need to find information from sources, connect multiple pieces of information, verify facts, and avoid common false beliefs. The six primary evaluation benchmarks (FEVER, TriviaQA, Natural Questions, TruthfulQA, StrategyQA, and ARC-Challenge) are chosen because they have questions with definite answers that can be checked objectively under per-benchmark correctness metrics (exact match, token-F1, label accuracy, or ROUGE-L depending on the task family). MMLU is reserved as an out-of-distribution retention control rather than a target benchmark: the fine-tune is not optimised for MMLU, but the retention guard aborts any cycle in which MMLU accuracy drops below a fixed fraction of the pre-cycle baseline.

The scope explicitly *excludes* creative generation tasks such as story writing, poetry composition, or open-ended dialogue where evaluation criteria are subjective and multiple valid responses exist. Similarly, tasks requiring up-to-date real-world information beyond the training cutoff such as stock prices, breaking news and current events are outside scope, as the system lacks mechanisms for continuous knowledge based updates. Important grounds like Code generation and mathematical theorem proving are excluded to maintain focus on natural language reasoning. The system operates on English text only, with the scope of working on multilingual capabilities in the future.

The model selection reflects resource constraints appropriate for undergraduate thesis work. Flan-T5-Large (780M parameters) provides sufficient capacity for complex

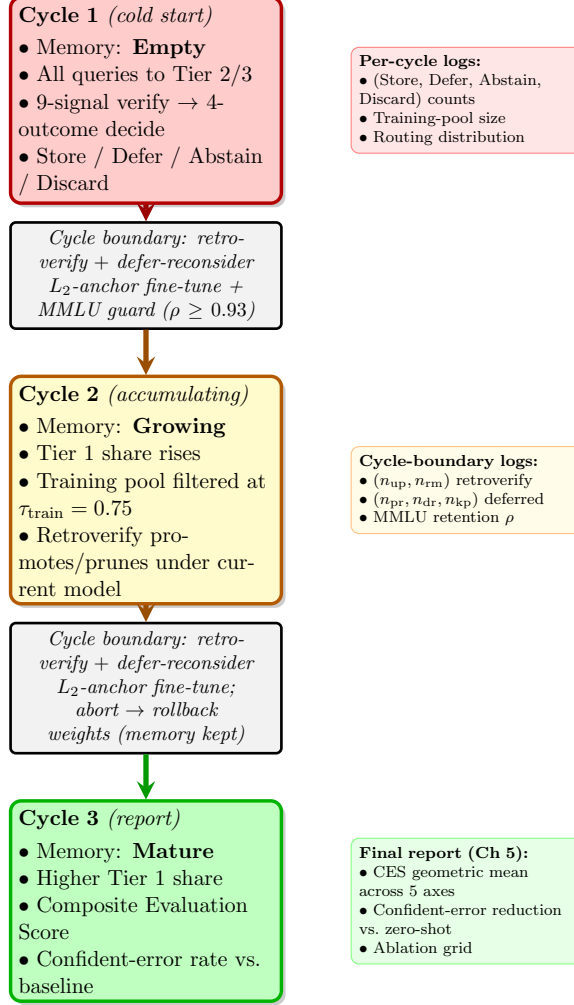


Figure 1.2: Three self-improvement cycles. Each cycle processes a fixed workload through the per-query pipeline (Stages 1–7) with verification and four-outcome storage. At each cycle boundary (Stage 8) a retroactive re-verification pass refreshes stored composites under the updated model, a deferred-entry reconsideration pass sweeps the deferred buffer, L_2 -anchored fine-tuning draws from a strict training pool, and an MMLU retention probe aborts the cycle with a weight rollback if the retention ratio falls below $\rho_{\min} = 0.93$. Memory is never rolled back. Headline results (Composite Evaluation Score, confident-error reduction against baselines, and the ablation grid) are reported at the end of Cycle 3 in Chapter 5.

reasoning while remaining feasible for fine-tuning with university GPU allocations. Larger models such as LLaMA-13B or GPT-3.5 scale would require computational resources that are too expensive and resource-heavy. The three-cycle experimental design captures the essential self-improvement dynamics while remaining achievable within a 12-month timeline, though in theory a full long-horizon equilibrium study would require 7–10 cycles.

Per-benchmark evaluation uses a fixed sub-sample sized for meaningful statistical power without excessive computational cost. The episodic memory is bounded at a target capacity of 10^6 entries with a flat FAISS index promoted to IVF-PQ above a threshold; for the three-cycle experiments reported in Chapter 5 the memory is initialised empty and grows up to the low tens of thousands of entries, which is well within single-GPU capacity and exercises the novelty filter and value-score pruning policies without saturating them.

1.6.2 Technical Challenges

The implementation of CAEM faces six primary technical challenges that must be addressed:

- Verifier Reliability and Stored-Quality Drift:** The nine-signal post-generation verifier produces a composite score u_{stored} that is used as both a Tier-1 retrieval proxy and a training-pool quality filter. Three of the nine signals (the internal-calibration family) read the model’s internal state directly and drift with every fine-tune, so stored composites are model-relative snapshots that must be refreshed at each cycle boundary. The retroactive re-verification pass addresses this drift with an upward-only update rule: only entries whose composite has improved past a re-verification threshold are promoted, while those that have fallen below it are categorically pruned. The cost is the full verifier re-scoring of every stored episode at each cycle boundary.
- Catastrophic Forgetting Under Iterative Fine-Tuning:** Fine-tuning on domain-specific verified examples risks degrading general language capabilities. CAEM addresses this through three layers of defence: a strict training-pool quality filter at a threshold above the store threshold, a 90/10 mix of verified reasoning chains and general-QA pairs in each training batch, and an L_2 anchor toward the base-model parameters that approximates EWC under a uniform-Fisher assumption [doi:10.1073/pnas.1611835114](https://doi.org/10.1073/pnas.1611835114). A hard out-of-distribution retention guard on MMLU aborts the cycle and rolls back weights if the retention ratio falls below a fixed floor; memory is never rolled back, so the fine-tune failing in cycle k does not lose the verified episodes accumulated during it. The effectiveness of each layer is validated empirically through the ablation grid in Chapter 3.
- Pre-Routing Confidence Calibration:** The pre-routing confidence u_{pre} is used both as a one-dimensional scalar in the combined routing score and as the sole input to the safety override, so its calibration matters directly. CAEM fuses two signals (a short-prefix token-probability aggregate and an encoder convergence ratio) and temperature-scales the fused scalar against a held-out calibration split by L-BFGS on binary negative-log-likelihood. The

non-obvious design choice is that confidence is computed from the encoder alone (no decode) so that its cost is paid per query regardless of tier; the cost of the signals it avoids (MC-dropout plurality, sample-set agreement, semantic entropy) is deferred to the post-generation verifier on Tier 2 and Tier 3 paths only.

- **Memory Retrieval Quality and Novelty Filtering:** The system uses FAISS for approximate nearest-neighbour search, which introduces the possibility of missing a semantically-similar entry if the embedding happens to fall near an index partition boundary. Near-duplicates are filtered at write time by a novelty threshold on cosine similarity (suppressing new writes when a stored neighbour exceeds the threshold); pruning at capacity uses a value score that combines retrieval frequency, success rate, recency, and storage-time confidence. The novelty threshold is a tuning knob: too high admits many near-duplicates that bloat the index without adding coverage, too low rejects paraphrase-level rewording that would have been genuinely useful additions.
- **Baseline Performance Dependency for Self-Improvement:** The self-improvement argument depends on the base model performing above random chance on the target benchmarks, because training on verified-but-imperfect episodes can only compound accuracy when the verifier’s false-positive rate is low enough that the training pool is cleaner than the raw model’s outputs. To reduce this risk, the project chooses Flan-T5-Large, which already performs competitively on factual-QA after instruction tuning, and picks benchmarks where the base model clears the threshold. Benchmarks on which the base model underperforms (e.g., narrow-domain subsets where zero-shot accuracy is near chance) are excluded from the training pool and used only for evaluation.
- **Evaluation Metric Selection:** Measuring hallucination reduction requires careful metric design beyond simple accuracy. CAEM’s primary metric is the confident-error rate: the fraction of responses that are both high-confidence under u_{stored} and incorrect under the benchmark’s correctness metric. This captures the central clinical concern (confidently-wrong outputs) more directly than overall accuracy. A geometric Composite Evaluation Score (accuracy, epistemic quality, retention, calibration, and verification reliability) provides the joint-axis view; the geometric mean penalises any axis that collapses, which is the behaviour that matches the use-case (a system that sacrifices retention for accuracy is not acceptable). Comparison against strong baselines (zero-shot Flan-T5-Large, a RAG-only variant, and per-mechanism ablations) is run under identical benchmarks and metric definitions to avoid the confound of re-implementing published baselines with different evaluation choices.

1.6.3 Mitigation Strategies

For each challenge identified, the research design incorporates mitigation strategies. Verifier-reliability and stored-quality drift are addressed by the retroactive re-verification pass, which re-scores every stored episode at each cycle boundary under the updated model and applies an upward-only update rule. Catastrophic forgetting is addressed by the combination of the strict training-pool quality filter

($\tau_{\text{train}} > \tau_{\text{store}}$), the 90/10 episode-general batch mix, the L_2 anchor regulariser, and the hard MMLU retention guard that aborts the cycle with a weight rollback if the retention ratio falls below the fixed floor $\rho_{\text{min}} = 0.93$. Pre-routing confidence calibration is tackled by temperature-scaling the fused scalar on a held-out calibration split before deployment, rather than on-the-fly adaptation. Memory retrieval quality is managed by the novelty filter at write time (suppressing near-duplicate writes at a cosine similarity threshold of 0.95) and by value-score pruning that triggers only when capacity is exceeded. Baseline-performance dependency is managed by benchmark selection and by the verifier’s contradiction veto and confabulation early-exit rule, both of which reduce the expected false-positive rate of the storage decision below the raw composite-threshold baseline. Evaluation metric risk is managed by reporting both the geometric Composite Evaluation Score and the confident-error-rate diagnostic, so that no single collapsing axis can mask a regression on another. Although the problems are significant, they can be managed within the resources and the time frame. The aim is to improve the system clearly and measurably while making the architectural commitments that the current state of the literature supports.

Chapter 2

Literature Review

2.1 Preliminaries

This section establishes foundational concepts and terminology required to understand the CAEM methodology, mathematical formulations, and experimental evaluation presented in subsequent chapters.

2.1.1 Large Language Models and Transformer Architecture

Neural networks that have been trained on quite big text collections to predict and produce natural language are known as large language models. The Transformer architecture [NIPS2017'3f5ee243](#), which processes sequences using a self-attention mechanism allowing all the tokens in the input to take account of every other token, is adopted by new LLMs. The encoder and decoder layers in this architecture are arranged in layers and each one has feed forward networks and multi-head self-attention.

In this study, we utilize Flan T5 [JMLR:v25:23-0870](#), an encoder-decoder model that integrates instruction based fine tuning and builds the existing T5 architecture. Encoder decoder models generate text step by step through the decoder after creating a bidirectional representation of the input through the encoder, in contrast to decoder only models like GPT that read text from left to right. Because the model is capable of fully understanding the question before generating an answer, this method is particularly effective for tasks involving answering questions.

Fine tuning adapts pre trained models to specific tasks or domains by continuing training on data specific to the task while using smaller learning rates. The model parameters θ are adjusted to reduce loss on the new data while still preserving the general abilities learned during pre training. Standard fine tuning updates all parameters, while parameter efficient approaches such as LoRA modify only a small portion of them.

2.1.2 Hallucination Taxonomy

Hallucinations in language model outputs can be understood along two main dimensions. The first dimension separates *intrinsic hallucinations*, which directly contradict information provided in the source input, from *extrinsic hallucinations*,

which introduce information that goes beyond what the source provides, regardless of whether it also happens to conflict with the source **ji2023survey**. For instance, if a biography states that “Einstein was born in Germany” and the model generates “Einstein was born in France,” this is an intrinsic hallucination because it contradicts the stated source. But if the model generates “Einstein played violin exceptionally well” without any evidence in the source, this is an extrinsic hallucination even if the claim happens to be true: the defining characteristic is that it cannot be verified from the input, not that it contradicts anything in it.

The second dimension distinguishes *factuality hallucinations*, which conflict with verifiable world knowledge, from *faithfulness hallucinations*, which contradict user input or task context **10.1145/3649506**. If a model claims “photosynthesis produces oxygen from carbon dioxide and water” and demonstrates factual accuracy. But on the other hand it it claims “photosynthesis occurs in mitochondria” then this demonstrates factual hallucination. Faithfulness focuses on whether the output of the model remains consistent with the supplied input or context, especially when summarizing documents or following instructions.

This research focuses on factuality hallucinations which occurs in knowledge based reasoning tasks. Here correct answers can be verified against trusted sources or logical rules and regulations. But faithfulness to fetch evidences also matter for etrieval augmented generation. Here outputs must be grounded in retrieved documents.

2.1.3 Natural Language Inference

Natural Language Inference (NLI) is the task of determining whether a hypothesis sentence logically follows from a premise sentence **10.1007/11736790**⁹. Given premise P and hypothesis H , an NLI model classifies the relationship as:

- **ENTAILMENT**: P implies H is true
- **CONTRADICTION**: P implies H is false
- **NEUTRAL**: P neither confirms nor denies H

For example, if we assume “The Beatles were formed in Liverpool in 1960” and hypothesis is “The Beatles were an English band,” then this relationship is ENTAILMENT. Given hypothesis “The Beatles were formed in London,” the relationship is CONTRADICTION. But if the given hypothesis is “The Beatles recorded many albums,” then the relationship is NEUTRAL.

We employ RoBERTa Large MNLI **liu2019roberta**, which is a model that is fine tuned on the Multi Genre Natural Language Inference dataset and it can achieve an accuracy of approximately 90.8% on the MNLI matched development set. NLI serves two purposes in CAEM. Firstly, it verifies factual correctness by checking whether generated claims are rewuiired by retrieved evidence, and secondly, t helps group answers that mean the same thing by testing for bidirectional entailment. If P entails H and H entails P , then P and H are considered semantically equivalent.

2.1.4 Sentence Embeddings and Semantic Similarity

Sentence embeddings map the length of variable text sequences to fixed dimensional dense vectors in a semantic space where similar meanings have high cosine similarity.

Sentence BERT (SBERT) **reimers-gurevych-2019-sentence** produces encasings or embeddings by fine tuning BERT in a siamese network structure on semantic textual similarity tasks. The model we employ, all mpnet base v2, generates 768-dimensional embeddings.

Cosine similarity between vectors $\mathbf{v}_1$ and $\mathbf{v}_2$ is computed as:

$$\text{sim}(\mathbf{v}_1, \mathbf{v}_2) = \frac{\mathbf{v}_1 \cdot \mathbf{v}_2}{\|\mathbf{v}_1\| \|\mathbf{v}_2\|} = \cos(\theta) \quad (2.1)$$

where θ is the angle between vectors. Similarity ranges from -1 (opposite) to 1 (identical), with 0 indicating orthogonality.

Semantic similarity is different from string similarity. For instance, the questions "Is 17 prime?" and "Is the number 17 a prime number?" have low string overlap but high semantic similarity (approximately 0.98). CAEM uses semantic similarity for key operations like memory retrieval (finding similar questions), novelty detection (avoiding redundant storage), and self-consistency verification (measuring reasoning chain agreement). The system uses FAISS (Facebook AI Similarity Search) **8733051** for fast and efficient approximate nearest neighbor search, allowing it to retrieve sub millisecond findings over thousands of stored episodes.

2.1.5 Self-Consistency Decoding

Self consistency **wang2023selfconsistency** improves reasoning quality by first generating multiple independent reasoning chains for the same question, then it selects the most common answer. Unlike standard greedy decoding which produces a single output, self-consistency samples M chains using temperature $T > 0$ to introduce diversity.

For CAEM, we adapt self consistency technique for verification rather than choosing an answer. Given question q and generated answer a , we generate M additional chains and measure agreement via average pairwise similarity:

$$u_{\text{SC}} = \frac{2}{M(M-1)} \sum_{1 \leq i < j \leq M} \text{sim}(\mathbf{v}_i, \mathbf{v}_j) \quad (2.2)$$

where $\mathbf{v}_i$ is the embedding of chain i , and only unique off-diagonal pairs are used (self-similarity diagonal terms are excluded). High consistency ($u_{\text{SC}} > 0.85$) indicates robust reasoning where multiple paths converge, while low consistency suggests uncertainty or errors.

2.1.6 Catastrophic Forgetting and Continual Learning

Catastrophic forgetting **MCCLOSKEY1989109** occurs when neural networks learn new tasks and suddenly lose their ability to perform tasks on previously learned materials. Standard gradient descent focuses only on improving performance for the current task and does not protect earlier knowledge. This causes the new parameter to update that help the new task but weakens old tasks. Fine tuning LLMs on domain specific data can reduce accuracy on general benchmarks by 30-50% without mitigation strategies.

Elastic Weight Consolidation (EWC) [doi:10.1073/pnas.1611835114](#) addresses this by identifying essential parameters for previous tasks using the Fisher Information Matrix, then adding quadratic penalties preventing large updates to important parameters:

$$\mathcal{L} = \mathcal{L}_{\text{new}} + \frac{\lambda}{2} \sum_i F_i (\theta_i - \theta_i^*)^2 \quad (2.3)$$

where F_i measures parameter importance, θ^* are previous parameters, and λ controls regularization strength. A cheaper first-order approximation replaces the full Fisher matrix with an isotropic L_2 anchor toward the base-model parameters, $\frac{\lambda}{2} \|\theta - \theta_{\text{base}}\|^2$. CAEM adopts this L_2 -anchor form as an EWC approximation and combines it with a 90/10 batch mix of episode-specific and general-capability data, plus a hard retention guard on a held-out MMLU probe that aborts the cycle and rolls weights back to the previous checkpoint whenever the retention ratio falls below a fixed floor. This addresses the challenge of learning from imperfectly verified data without degrading through error reinforcement, while preserving the accumulated memory even when a weight update must be rolled back.

The conceptual basis for understanding CAEM’s nine-signal post-generation verifier, pre-routing confidence, episodic-memory architecture, and constrained self-improvement loop is established by the groundwork above. The following sections describe how prior research develops and validates these individual components, preparing for their integration in Chapter 4.

2.2 Review of Existing Research

This section surveys prior work across six areas essential to understanding CAEM’s design: hallucination characterization and measurement, verification methods for reasoning validation, confidence estimation techniques, memory-augmented architectures, continual learning approaches, and evaluation benchmarks. Each subsection identifies key contributions and limitations that motivate CAEM’s architectural choices.

2.2.1 Hallucination Problem and Measurement

Understanding the nature and characteristics of hallucination in LLMs requires methodologies like taxonomic frameworks and quantitative measurement. Early studies on hallucination shows summarization and data to text generation. Ji et al. [ji2023survey](#) published the first major survey in ACM Computing Surveys. It establishes a study differentiating intrinsic hallucinations that contradicts source material from extrinsic hallucinations. Even though the claims were unverified. Their work shows that hallucinations occurs at several stages. It includes data collection that embeds factual errors, training procedures that reinforce misleading correlations, and inference methods that fail to account for uncertainty.

Huang et al. [10.1145/3649506](#) extended this study specifically for large language models in ACM Transactions on Information Systems. It is extended by adding the differences between factuality and faithfulness. Their analysis of causes across the model lifecycle suggests various groundbreaking research. It suggests that effective mitigation should have intervention at several stages instead of relying solely on

detection applied after the model has already produced an answer. This directly supports CAEM’s multi component strategy which combines memory, verification, and iterative learning.

Quantitative measurement studies of hallucination prevalence reveals concerning statistics across domains and model scales. Lin et al. **lin-etal-2022-truthfulqa** introduced TruthfulQA at ACL2022, a benchmark of 817 questions across 38 categories which has been specifically designed to elicit imitative falsehoods where models reproduce common human misconceptions found in training data. Their result demonstrates an inverse scaling effect: GPT-3 175B achieved only 21-25% on the truthful and informative metric (or 58% on truthfulness alone) compared to 94% human performance. Larger models often performed worse than smaller ones. This proves that only scaling is not sufficient for truthfulness and thus it needs strong motivation for architectural innovation.

The inverse scaling result has major result for hallucination mitigation strategies. If adding parameters reduces truthfulness on some tasks then mitigation strategies must operate beyond parameter growth. CAEM addresses this by incorporating external verification and memory components that operate independently of model scale, providing corrections and verified knowledge regardless of base model size.

Min et al. **min-etal-2023-factscore** introduced FactScore at EMNLP 2023. It provides smooth evaluation through the process of breaking down complex. Instead of assigning a single label to check correctness to an entire answer, FactScore computes the percentage of claims that is supported by sources. As a result, this method achieves less than 2% error rate compared to human evaluation. Applied to biography generation, ChatGPT achieved only 58% FActScore, which indicates that even the highly capable models produce substantial proportions of unsupported atomic facts. This findings highlights the need for fact level verification rather than holistic response evaluation.

Domain specific studies show even higher hallucination rates in specefic contexts. Alkaissi and McFarlane **alkaissi2023artificial** studied by making ChatGPT response to 25 clinical questions and found 69-77% contained hallucinated medical information. Cheli et al. **info:doi/10.2196/53164** studied hallucination in medical literature queries. He reported rates of 69.6% for GPT-3.5, 28.6% for GPT-4, and 91.4% for Google Bard.

Recent benchmark provided comprehensive evaluation across Several dimensions. One of them being legal sector. Factual accuracy has been measured in legal sector. According to Google’s FACTS benchmark (2025), it shows that even top performing systems maintain 6.4% hallucination rates on legal queries. Furthermore, the Vectara Hallucination Leaderboard tracks document summarization hallucination rates ranging from 3% for GPT-4 to 27.2% for Google PaLM 2 Chat. It thus demonstrates wide variation among models and confirm that hallucination remains an issue even in state of the art systems.

The information above has revealed three critical takeaways that shape CAEM’s design. Firstly, hallucination occurs across models, tasks, and domains. Depending on context the rates ranging from 3% to over 90% . Secondly, larger models do not automatically produce fewer hallucinations. This implies that architectural intervention is necessary. Lastly, verifying information at the level of atomic facts is far more informative than evaluating full responses, motivating CAEM’s commitment to fact level NLI based verification.

2.2.2 Verification Methods

Verification methods are used to check whether a model’s reasoning is correct before the information is stored or shown to users. This subsection explains four approaches (chain-of-thought prompting, self-consistency decoding, semantic entropy, and natural-language inference) that together supply the signals aggregated inside CAEM’s composite verifier.

Chain of thought prompting, which was introduced by Wei et al. **NEURIPS2022’9d560961** at NeurIPS 2022, improves reasoning quality by prompting models to generate the next steps before final answers. Showing these intermediate steps leads to stronger reasoning and provides better starting points for verification. When tested on the PaLM 540B model, chain of thought greatly improved performance on math reasoning tasks, increasing accuracy on GSM8K from about eighteen percent to over fifty eight percent. The authors found that this technique showed an emergent ability threshold around 100 billion parameters for *base models* trained on raw text, below which chain-of-thought provided minimal benefit. However, this threshold does not apply to instruction-tuned models, which learn to follow step-by-step reasoning patterns during the instruction-tuning phase itself. Chung et al. **JMLR:v25:23-0870** demonstrated that Flan-T5 models show substantial chain-of-thought benefits at 780 million parameters after instruction tuning. Since CAEM uses Flan-T5-Large (780M parameters), which is an instruction-tuned model, chain-of-thought prompting is effective despite being well below the 100B base-model threshold.

For CAEM, chain of thought serves two purposes. Firstly, it improves base model reasoning quality. It also provides better starting points for verification. Secondly, it gives explicit reasoning chains that enables step by step verification through NLI which also checks whether each reasoning step is supported by evidence. The explainability of chain of thought reasoning proves essential for diagnosis when verification fails, enabling error analysis that identifies which reasoning steps caused failures.

Wang et al. **wang2023selfconsistency** expanded this idea in 2023 by introducing self consistency. Instead of producing a single reasoning path, the model generates several different explanations and then selects the most common answer. The key idea is that difficult and complex problems usually has multiple valid reasoning paths which leads to the same correct answer. On the other hand, incorrect answers rarely achieve consistency across independent derivations. Quantitative improvements with PaLM 540B demonstrate substantial gains: +17.9% on GSM8K, +11.0% on SVAMP, +12.2% on AQuA, and +6.4% on StrategyQA.

CAEM uses self consistency in different ways. It chooses reasoning chain agreement through semantic similarity rather than selecting the majority answer. If the chains mostly agree, the reasoning is then considered reliable and suitable for storage. But if they differ in opinion then the system treats the result as uncertain and may apply further checks or reject it. As a result, this method is efficient since it only requires a small number of extra generations which makes it practical for regular verification. And it also provides useful confidence signals.

Another important method is semantic entropy introduced by Farquhar et al. **farquhar2024semanti** in 2024. It represents the most rigorous treatment of uncertainty based hallucination detection. Their key innovation addresses a primary limitation of token level entropy: Answers that express the same idea in different wording are grouped together using inference checks, and uncertainty is measured across these meaning

based groups.

The methodology achieves AUROC of 0.790 for fabrication detection on question answering tasks. It substantially outperforms token entropy (0.691) and verbalized probability methods (0.698). Even though the performance remains robust across model families including LLaMA 2 (7B-70B), Falcon (7B-40B), and Mistral 7B, the Nature publication validates semantic entropy as a cutting edge technique for uncertainty quantification, motivating its inclusion in CAEM despite computational overhead.

Semantic entropy serves as the third verification layer for CAEM. It catches reports where both NLI and self consistency fail. Let us consider a model which is asked "What is the capital of Australia?" that consistently always generates "Sydney" across multiple chains with high individual probabilities. NLI verification may pass if the model generates factually correct supporting statements about Sydney. Here, self consistency shows perfect agreement. But still the semantic entropy will generate 10 diverse samples and will measure cluster entropy. This may reveal that the model also produces "Canberra" and "Melbourne" at high temperature thus indicating underlying uncertainty. This detection capability for systematic confabulation justifies the computational cost.

Natural language inference provides direct verification by checking whether a claim is supported by evidence. Thorne et al. **thorne-etal-2018-fever** introduced the FEVER dataset at NAACL 2018 which formulates fact verification as NLI with 185,445 claims generated from Wikipedia. Systems must classify claim evidence relationships as SUPPORTS, REFUTES, or NOT ENOUGH INFO. This three way classification would map directly to CAEM's storage decisions. The decision would be to support claims merit storage, refute claims are rejected, and insufficient evidence trigger uncertainty handling.

Pre trained NLI models such as RoBERTa-Large-MNLI has achieved approximately 90% accuracy on the MNLI benchmark. It thus provides reliable verification for factual claims. But the limitation is that even though NLI verifies individual factual statements, they may miss logical reasoning errors that do not contradict facts. For example, a model could state "Sydney is Australia's largest city" which is true and conclude "therefore Sydney is the capital" which is an invalid reference. NLI would verify the factual premise while missing the logical error thus motivating the need for complementary verification layers.

The integration of these four methods provides complementary strengths inside CAEM's nine-signal verifier. Chain of thought improves reasoning interpretability and gives the verifier explicit intermediate claims to score. Self-consistency contributes the sample-set agreement signal s_{avg} , which catches reasoning paths that diverge across stochastic decodes. Semantic entropy contributes the normalised-entropy signal h_{norm} , which detects confabulation when the model is individually confident but meaning-level unstable. NLI contributes the external-grounding family: a per-token entailment probability p_{entail} , a weakest-link grounding score $p_{\text{ground_max}}$ over atomic facts, and, most importantly, a contradiction probability p_{contra} whose crossing a fixed threshold acts as an independent veto that forces the storage decision to DISCARD regardless of the composite score. These signals combine with three internal signals (token probability, MC-Dropout variance, and an internal-state probe) to form the composite u_{stored} that drives the four-outcome storage decision. The specific weights and thresholds, and the resulting end-to-end verifier accuracy,

are calibrated empirically and reported in Chapter 5; the point for this chapter is that the literature supplies four well-validated families of signals whose failure modes are substantially disjoint, which is why a composite is expected to improve on any single one of them.

2.2.3 Confidence Estimation

Reliable confidence estimation helps systems tell the difference between answers that can be trusted and ones that need extra checking from outside sources. This section looks at basic methods for measuring uncertainty and explains how these ideas have been adapted for use with large language models.

Gal and Ghahramani **pmlr-v48-gal16** laid the theoretical foundations for practical uncertainty estimation in their ICML 2016 paper "Dropout as a Bayesian Approximation." Their key contribution explains that dropout usually during training can be understood as approximate Bayesian inference over model parameters. By keeping dropout active during testing and running the model multiple times, it becomes possible to measure how much the outputs vary. This variation can be used to estimate epistemic uncertainty, which refers to uncertainty caused by limited knowledge about the model itself. And this is different from aleatoric uncertainty, which comes from noise or randomness in the data. But it cannot be reduced.

The theoretical framework shows that a network with dropout before each weight layer approximates a deep Gaussian process. This predicted average and variance can be approximated through Monte Carlo sampling. It describes as performing K forward passes with different dropout masks and computing output variance. For better understanding, this provides uncertainty estimates without requiring group training or modifying the loss function. Because this approach works with any network that already uses dropout, it is especially useful for adapting existing large language models.

For CAEM, MC Dropout is one of the three internal signals inside the post-generation verifier, sitting alongside token probability and an internal-state probe. It quantifies model uncertainty through output variance: high variance indicates the model is unstable with respect to its own weights and suggests that the composite score should be pulled down, while low variance indicates confident generation. This signal is insufficient on its own, since models can be confidently wrong, but the computational cost of 5 to 10 forward passes is acceptable inside the verifier, which only runs on queries that have already been dispatched to Tier 2 or Tier 3 generation. MC Dropout is deliberately *not* used in the pre-routing confidence u_{pre} , which must be cheap enough to run on every query before tier selection.

A major problem with neural network confidence is that models are often too confident in their answers. Guo et al. **pmlr-v70-guo17a** demonstrated this at ICML 2017. They showed modern deep networks predict confidence scores substantially which exceeds actual accuracy rates. For example, predictions at 80% confidence may achieve only 60% accuracy and create a dangerous miscalibration for decision making. They introduced Expected Calibration Error (ECE) as the standard metric, dividing predictions into bins by confidence and measuring the gap between confidence and accuracy within each bin.

Their proposed solution, temperature scaling, achieves remarkable effectiveness with minimal complexity. The method divides logits by a learned scalar temperature

$T > 0$ optimized on a validation set: $\hat{y} = \sigma(\mathbf{z}/T)$ where $\mathbf{z}$ are logits and σ is softmax. Temperature scaling preserves accuracy (argmax unchanged) of classification while calibrating probabilities. Even though only a single parameter is used, it outperforms more complex methods including histogram binning, isotonic regression, and Bayesian approaches. The simplicity and effectiveness make temperature scaling the standard calibration method applied after training is complete.

CAEM uses temperature scaling as the final step in each stage of confidence estimation: once on the pre-routing u_{pre} and once on the post-generation composite u_{stored} . This step adjusts the raw score so that it matches the real accuracy observed on a held-out calibration split, so that when either stage reports eighty percent confidence, about eighty percent of those predictions are actually correct. This makes it possible to set reliable thresholds for tier selection, for the four-outcome storage decision, and for the training-pool filter τ_{train} .

Kadavath et al. **kadavath2022language** from Anthropic explored self evaluation in large language models at NeurIPS 2022. They introduced two methods for extracting confidence from LLMs themselves. First is P(True) asks models to evaluate the probability their own previous answers are correct. And, P(IK) or "I Know" prompts models to predict confidence before generating answers. Their findings reveal nuanced calibration properties. Larger models tend to show better calibration on multiple choice questions which means their confidence levels line up more closely with how often they are actually correct. However, reinforcement learning from human feedback can make calibration worse, since it often rewards answers that sound helpful or confident rather than ones that are strictly accurate.

The idea that reinforcement learning from human feedback can hurt calibration is important. Models are trained to be helpful. But they may express high confidence to appear authoritative even when uncertain which is exactly the opposite of desired behavior for high stakes applications. Because of this conflict between sounding helpful and being truthful, CAEM relies on several outside confidence signals instead of trusting the model’s own confidence alone.

Xiong et al. **xiong2023can** conducted a detailed study of confidence estimation methods for large language models. They provided comprehensive approaches into white box which use internal information like token logits and sequence probabilities, and black box methods, which rely on external signals such as verbalized confidence or consistency checks. Their systematic evaluation across multiple benchmarks reveals striking findings. That is, even the strongest single methods only reached about 0.50 to 0.60 AUROC for telling correct answers from incorrect ones, which is just slightly better than random guessing.

CAEM’s multi-signal approach is influenced by this limited discriminative ability of any individual signal. The system splits confidence into two stages. The first stage, pre-routing confidence u_{pre} , fuses two cheap signals (a token-probability estimate from a short draft decode, and an encoder-side convergence ratio C_{conv} over the final attention layers) through a temperature-scaled sigmoid, and is computed on every query. It exists to drive tier selection and the independent safety override. The second stage is the nine-signal post-generation verifier producing u_{stored} , which runs only after Tier-2 or Tier-3 generation and combines three internal signals (token probability, MC-Dropout variance, internal-state probe), three sample-set signals (self-consistency agreement, entailment probability, normalised semantic entropy), and three external-grounding signals (weakest-link and mean atomic-fact grounding,

plus contradiction probability). The final weights and the target discriminative accuracy of each stage are calibrated on a held-out split and reported in Chapter 5. Recent work by Tian et al. **tian2023just** questions if models genuinely know when they don't know. It also questions if the models only match patterns for uncertain expressions from training data. Their study found that verbalized confidence often reflects patterns in the training data rather than the model's actual knowledge or uncertainty. This reinforces CAEM's strategy of combining implicit signals like token probability, dropout, and self consistency with explicit methods, instead of relying only on verbalized confidence, which can be misleading or superficial.

The confidence estimation literature establishes several key findings for CAEM's design. Firstly, relying on a single confidence signal does not provide enough accuracy for safe routing, which makes combining multiple signals necessary. Secondly, systematic overconfidence requires calibration applied after training through temperature scaling. Thirdly, white-box methods that access internal model information offer complementary insights compared to black-box approaches based on output consistency. Lastly, RLHF training may degrade calibration, so pre-RLHF models or instruction-tuned models like Flan-T5 are expected to give more reliable confidence signals. Together, these points shape CAEM's two-stage confidence architecture. The first stage is a temperature-scaled pre-routing score u_{pre} that fuses token probability with encoder convergence and runs on every query. The second stage is a nine-signal post-generation verifier producing u_{stored} that runs only after generation. Both stages end with temperature scaling on a held-out calibration split.

2.2.4 Memory-Augmented Architectures

External memory architectures give neural networks a place to store information outside their usual parameters. It helps them learn and fetch facts. This subsection explains how these ideas developed. Starting with early systems that used differentiable memory and ending with the modern approach known as retrieval augmented generation.

Graves et al. **graves2014neural** introduced Neural Turing Machines (NTMs) in 2014. They ventures into the idea of giving neural network external memory. And as a result, they could read from and write to in a smooth and trainable way. The architecture combines a controller network with an addressable memory matrix. This supports both content based addressing that is done via cosine similarity and location based addressing, done via shift operations. The key innovations include read and write operations. It will enable end to end training via backpropagation. As a result, NTMs successfully learned algorithmic tasks. These includes copying, sorting, and associative recall.

The importance of CAEM is that it shows how external memory can give a model abilities that normal models do not have. NTMs were mainly used for learning algorithms. But, it can store facts as well. CAEM uses this idea in its episodic memory. It stores verified question and answer pairs along with the reasoning behind them. This lets the model call them up when needed without any delay.

Weston et al. **weston2015memory** introduced Memory Networks at ICLR 2015. Their model scaled to millions of memory slots. It also showed that a system could perform several rounds of reasoning to answer a question. The architecture consists of four learned components. The components are L: turning the input into

features, updating the memory, turning the memory output into features, and finally producing a response. Furthermore, Memory Networks performed strongly on bAbI question. With answering tasks that required reasoning over multiple facts. Sukhbaatar et al. **NIPS2015'8fb21ee7** extended this model with end to end Memory Networks at NeurIPS 2015. He removed the need for supervision on retrieval. He did this model by continuous attention mechanisms. This described how CAEM builds its own style of step by step reasoning using stored knowledge.

Graves et al. **graves2016differentiable** presented the Differentiable Neural Computer (DNC) in Nature 2016. It built on the idea of Neural Turing Machines but added smarter memory tools like dynamic allocation. As a result it keep track of the order in which things were written. DNCs achieved 3.8% average error on bAbI tasks compared to what it achieved for memory networks which is 6.4%. It clearly shows that calculated memory management improves performance.

Lewis et al. **NEURIPS2020'6b493230** introduced Retrieval-Augmented Generation (RAG) at NeurIPS 2020. They showed that combining what a model knows in its parameters with information stored outside the model can make its answers much more accurate. RAG adds a large searchable index of Wikipedia to a BART model with four hundred million parameters. It uses Dense Passage Retrieval to look up relevant documents. Then it works out the final answer by considering the generation probability across all the documents it retrieved. The architecture achieves substantial improvements over parametric-only models: 44.5% exact match on Natural Questions (versus 41.5% prior SOTA), 68.0% on TriviaQA (versus 60.5%), and 45.5% on WebQuestions (versus 44.7%).

Critically, RAG demonstrates index hot-swapping: updating the document index without retraining achieves 70% accuracy on questions about updated world knowledge, proving that non-parametric memory can be updated without expensive re-training. This property directly motivates CAEM's episodic memory, which can be updated continuously through verification without model fine-tuning, though CAEM additionally employs iterative fine-tuning to consolidate learned patterns.

The limitation of standard RAG for CAEM's purposes is that retrieved knowledge is not verified before use and does not accumulate verified reasoning chains. CAEM extends RAG by adding verification layers and storing verified outputs for future retrieval, creating a growing knowledge base rather than relying solely on static external corpora.

Johnson et al. **8733051** developed FAISS to handle similarity search at the scale of billions of item in 2019. It resulted in efficient implementation of memory augmented systems. The library achieves 8.5x speedup over prior GPU methods through optimized k-selection algorithms. Furthermore, FAISS also includes several types of indexes, such as IVF with product quantization, which allows the system to search in less than linear time. For CAEM, FAISS is the tool that makes real time memory search possible across about twenty thousand stored episodes. Keeping the search time below one tenth of a second is important for making the system fast enough to use in practice.

Reimers and Gurevych **reimers-gurevych-2019-sentence** introduced Sentence BERT at EMNLP 2019. This model produced semantically meaningful sentence. This is achieved through embeddings that follows siamese network training. The speed improvement over regular BERT is huge. With standard BERT, finding the most similar sentence pairs in a group of ten thousand sentences takes about sixty

five hours. With Sentence BERT, the same task takes only about five seconds because you can compare the embeddings directly instead of running a heavy cross encoder. This big jump in efficiency is what makes CAEMs memory search possible. Each query is turned into a 768-dimensional embedding, and the system can look through the entire memory index in real time.

These memory architecture developments establish the feasibility and benefits of external knowledge storage. NTMs prove differentiable memory enables structured reasoning. Memory Networks scale to millions of facts with multi-hop reasoning. RAG demonstrates that non-parametric knowledge improves factual accuracy and supports dynamic updates. FAISS and Sentence-BERT provide the efficient infrastructure for practical deployment. CAEM synthesizes these advances and extends them in three specific ways. First, retrieval is gated by a combined similarity-and-confidence score $\mathcal{S} = 0.70 s + 0.30 u_{\text{stored}}$ rather than similarity alone, so high-similarity matches that were stored with low verifier confidence still route to Tier 3 grounding. Second, routing is overridden by an independent safety gate on the pre-routing confidence u_{pre} that forces Tier 3 whenever u_{pre} falls below a fixed floor, regardless of how similar the best memory match is. Third, the memory itself is populated only through a four-outcome storage decision (STORE, DEFERRED, ABSTAIN, DISCARD) with a bounded FIFO deferred buffer for borderline items, so the knowledge base grows only from items that pass verification rather than from all retrievals. These extensions distinguish CAEM from static-corpus RAG and motivate the rest of the methodology.

2.2.5 Self-Improvement and Continual Learning

Self-improvement mechanisms enable models to learn from interaction and feedback, while continual learning addresses the challenge of acquiring new knowledge without forgetting previous capabilities. This subsection reviews methods for iterative refinement and catastrophic forgetting mitigation that inform CAEM’s self-improvement loop.

Kirkpatrick et al. [doi:10.1073/pnas.1611835114](https://doi.org/10.1073/pnas.1611835114) introduced Elastic Weight Consolidation (EWC) at PNAS 2017, providing a principled solution to catastrophic forgetting in neural networks. When learning sequential tasks, standard gradient descent optimizes exclusively for current task performance, causing parameter updates that improve new task accuracy while dramatically degrading previous task performance. EWC addresses this by estimating parameter importance for previous tasks using the Fisher Information Matrix, then adding quadratic penalties preventing large updates to important parameters.

The EWC loss function augments standard task loss with a regularization term:

$$\mathcal{L}_{\text{EWC}} = \mathcal{L}_{\text{task}} + \frac{\lambda}{2} \sum_i F_i (\theta_i - \theta_i^*)^2 \quad (2.4)$$

where F_i approximates the Fisher Information measuring how much loss increases when parameter θ_i changes, θ^* are previous task parameters, and λ controls regularization strength. Parameters important for previous tasks (high F_i) receive strong penalties against change, while less important parameters adapt freely to new tasks. Applied to Atari game learning, EWC maintained approximately 90% of single-task performance when sequentially learning 10 games, compared to severe degradation

with standard SGD. For CAEM, EWC provides the theoretical justification for the L_2 anchor used during self-improvement fine-tuning. Full Fisher Information Matrix computation is prohibitive for a 780M-parameter model on a single RTX 4090, so CAEM approximates the EWC penalty with an isotropic L_2 anchor toward the base-model parameters, $\frac{\lambda}{2} \|\theta - \theta_{\text{base}}\|^2$. This treats every parameter as equally important in the absence of a computed Fisher estimate, which is conservative and, combined with the other protections below, sufficient to meet the retention target.

The practical implementation in CAEM employs $\lambda = 0.01$, combined with a 90/10 batch mix (90% episode-specific verified examples, 10% general-capability examples) to maintain broad competence during fine-tuning. A held-out MMLU probe supplies a hard retention guard: before each cycle a baseline MMLU_{pre} is recorded, after fine-tuning $\text{MMLU}_{\text{post}}$ is measured, and the ratio $\rho = \text{MMLU}_{\text{post}}/\text{MMLU}_{\text{pre}}$ is compared against a fixed floor $\rho_{\text{min}} = 0.93$. If $\rho < \rho_{\text{min}}$, the cycle aborts, the weights roll back to the previous checkpoint, and retroactive re-verification is skipped for that cycle. Crucially, the episodic memory is never rolled back even when the weight update is rejected, so verified knowledge accumulated this cycle remains available for future routing; this asymmetry is deliberate and distinguishes CAEM’s retention mechanism from standard continual-learning rollback schemes.

Natarajan et al. **natarajan2013learning** provided theoretical foundations for learning with noisy labels at NeurIPS 2013. Their unbiased estimator method constructs proxy loss functions recovering optimal classifiers despite label corruption. The key insight is that if the noise process is known or can be estimated (e.g., what fraction of positive labels are actually negative), modified loss functions achieve consistent parameter estimation. Their method achieves more than 88% accuracy with 40% label corruption which substantially outperforms standard training.

For CAEM, this framework directly applies to verified episodic memory: no practical verifier is perfect, so a known fraction of stored episodes will carry incorrect labels (false positives that nonetheless pass the composite check). The stricter training threshold $\tau_{\text{train}} = 0.75$, which is set higher than the storage threshold $\tau_{\text{store}} = 0.65$, is designed exactly to lift the purity of the training pool above the purity of the memory as a whole, and the noise-tolerant-learning literature is what justifies training on the resulting pool at all: it proves that training on pools whose purity is well below 100% can still improve the model rather than degrading it, provided the corruption rate is below a method-dependent tolerance. This grounding distinguishes CAEM from naive approaches that assume perfect verification is necessary for self-improvement. The actual empirical purity of CAEM’s training pool, and the resulting effect on fine-tune outcomes, is measured in Chapter 5.

When Zelikman et al. **NEURIPS2022’639a9a17** introduced STaR (Self Taught Reasoner) at NeurIPS 2022, they showed successful bootstrapping using iterative self improvement. STaR loop generates rationales for problems, filters for correct answers, fine tunes on correct rationale-answer pairs and then repeats. A key innovation is rationalization. When the model produces a correct answer without valid reasoning, it generates a rationale conditioned on the correct answer. This data augmentation stops the model from learning problems that it already knows how to solve.

In CommonsenseQA test, STaR achieved +35.9% improvement over few-shot baseline through iterative refinement. The method shows that bootstrapping from the model’s own output works when coupled with appropriate filtering. The direct

parallel to CAEM is evident: generate predictions, verify correctness, fine-tune on verified examples, and repeat. CAEM extends STaR in two specific ways. First, simple correctness filtering is replaced by the nine-signal composite u_{stored} and the four-outcome storage decision, so the filter is itself a learned verifier rather than a binary match. Second, verified items persist in episodic memory for future retrieval rather than being consumed purely as training data, which turns each cycle into a cumulative knowledge gain rather than a one-off weight update.

The recent work of examining intrinsic self-correction capabilities reveals important limitations. Huang et al. **huang-etal-2023-large** found at EMNLP 2023 that prompting LLMs to review and correct their own responses rarely improves accuracy if we do not use external feedback. Kamoi et al. **kamoi-etal-2024-llms** analyzed when LLMs can correct mistakes at TACL 2024. They found that intrinsic self-correction often fails or degrades performance. The bottleneck is reliable feedback generation. So, models that produce incorrect initial answers usually cannot generate reliable correction signals without external validation.

This limitation motivates CAEM’s architectural approach. Instead of depending on intrinsic self-correction using prompting alone, CAEM employs external verification through an NLI cross-encoder, multi-chain self-consistency, and semantic-entropy analysis over stochastic re-samples. These external signals provide the reliable feedback the self-correction literature identifies as a precondition for effective self-improvement, and their failure modes are sufficiently disjoint that aggregating them into a single composite score is expected to be substantially more reliable than any one of them alone, and substantially more reliable than the model’s own self-evaluation. The end-to-end verifier accuracy is reported in Chapter 5. Together with the four-outcome storage decision and the training threshold $\tau_{\text{train}} > \tau_{\text{store}}$, this enables the virtuous improvement cycle in which verified generation produces higher-purity training data, which in turn improves generation.

The self-improvement literature also points out successful cases. When external feedback is available (for example code execution results, formal verification, retrieval of sources), models can effectively adapt the corrections. CAEM’s Tier 3 routing with RAG provides exactly the same external feedback by retrieving evidence that grounds generation and enables meaningful verification. The system learns not just from its own outputs but from the interaction between generation and evidence based verification.

Domain adaptation research provides additional relevant findings. Gururangan et al. **gururangan-etal-2020-dont** demonstrated at ACL 2020 that continued pre-training on domain-specific unlabeled data (domain-adaptive pretraining) followed by task-specific fine-tuning outperforms direct fine-tuning on limited labeled data. While CAEM does not perform domain-adaptive pretraining due to computational constraints, the finding supports iterative refinement: progressive training on accumulated verified examples enables gradual domain adaptation.

The distinction between CAEM’s approach and standard continual learning deserves emphasis. Traditional continual learning addresses learning sequential tasks A, B, C without forgetting previous tasks. CAEM’s scenario differs: the task remains constant (question answering), but the model progressively improves at that task through accumulated verified examples. This setting is closer to curriculum learning and iterative refinement than task-sequential continual learning, though catastrophic forgetting concerns still apply since fine-tuning on domain-specific verified examples

risks degrading general capabilities.

Synthesis of these methods informs CAEM’s self-improvement loop design. EWC, approximated in CAEM as a cheap L_2 anchor toward the base-model parameters, prevents catastrophic forgetting of general capabilities. Noisy-label learning provides theoretical justification for training on imperfectly verified data and motivates the stricter training threshold $\tau_{\text{train}} > \tau_{\text{store}}$ that boosts training-pool purity relative to memory purity. STaR demonstrates successful bootstrapping patterns and motivates the cycle structure. The self-correction research validates the need for external verification rather than intrinsic review and motivates the nine-signal composite over any single-signal confidence proxy. Finally, the out-of-distribution retention guard on MMLU with floor $\rho_{\text{min}} = 0.93$ and weight rollback (but not memory rollback) closes the loop by converting the catastrophic-forgetting risk into a detectable, rejectable event rather than a silent degradation. The resulting improvement trajectory across the three cycles, and the retention margin actually achieved, are reported in Chapter 5.

2.2.6 External Baselines and Competitive Positioning

The related work reviewed above clusters into three families relevant to CAEM’s evaluation: prompt-level reasoning methods, retrieval-augmented generation methods, and training-time self-improvement methods. Chapter 5 instantiates one representative from each family as an external baseline so that every mechanism CAEM adds on top of a plain language model is compared against at least one prior system that already uses that mechanism in isolation. This subsection establishes what each baseline is, what it isolates, and what a fair comparison would look like; the numerical results appear in Chapter 5.

The prompt-level family contributes two baselines. The first, **zero-shot Flan-T5-Large**, uses the same 780M-parameter backbone CAEM fine-tunes, with no CoT prefix, no retrieval, and no verification. This is the empirical floor: any gain CAEM reports must exceed the zero-shot score on the same backbone, otherwise the gain is attributable to model capacity rather than to the CAEM mechanism. The second, **Chain-of-Thought** prompting **wei2022chain**, adds the standard “Let’s think step by step” prefix without any further modification. CoT isolates the contribution of intermediate reasoning on the same backbone and controls for the possibility that CAEM’s gains reflect better rationale generation rather than the verifier-memory architecture.

The retrieval family contributes three baselines. **DPR-RAG karpukhin-etal-2020-dense, NEURIPS2020*6b493230** retrieves the top $k = 5$ Wikipedia passages from a dense DPR index, concatenates them into a 384-token context, and generates from the same Flan-T5-Large backbone; this isolates the contribution of external grounding without any verification or memory. **CoT+RAG** combines CoT prompting with DPR retrieval and isolates the contribution of reasoning over retrieved evidence. **FLARE jiang-etal-2023-active** retrieves dynamically during generation, triggering an additional retrieval whenever a look-ahead window’s minimum token probability falls below a confidence floor $\theta = 0.4$ with a 64-token look-ahead; this is the closest prior system to CAEM’s confidence-driven routing in spirit, because it too conditions retrieval on an internal confidence signal rather than retrieving unconditionally. The comparison against FLARE directly tests whether CAEM’s tier struc-

ture, which adds memory retrieval as a second confidence-gated option and a nine-signal composite as the gating variable, materially outperforms a simpler confidence-gated retrieval scheme. A fourth retrieval system, **Self-RAG asai2024selfrag**, is acknowledged as the most ambitious retrieve-and-critique pipeline in the literature but is included only as a citation-level reference rather than as a replicated numerical baseline, because Self-RAG’s critic tokens are tied to a LLaMA-family backbone and retargeting them to Flan-T5-Large would change enough of the system to invalidate the comparison.

The training-time family contributes two baselines. **Vanilla fine-tuning** trains Flan-T5-Large on the verified episode pool for the same ten cycles as CAEM, but without the L_2 anchor, without the MMLU retention guard, and without the episodic memory or the verifier, so every selected episode is treated as a clean label. This isolates the contribution of CAEM’s retention-protection machinery: if Vanilla FT degrades on MMLU or on the later benchmark cycles while CAEM does not, the retention guard and the L_2 anchor are responsible. **EWC-only fine-tuning** trains for the same ten cycles with the L_2 anchor and the MMLU guard in place but strips the verifier, the nine-signal composite, the memory, and the routing; this isolates the contribution of the verification-and-memory machinery from the contribution of the retention machinery. STaR **NEURIPS2022’639a9a17** is discussed above as the closest prior method in spirit and is run only as an optional ceiling reference conditional on remaining compute budget: its rationalisation step and its binary correctness filter differ enough from CAEM’s nine-signal composite and four-outcome decision that a large gap to STaR would mix architectural contribution with rationalisation-quality artefacts on the Flan-T5-Large backbone, so STaR is reported where possible as an aspirational ceiling for what a pure iterative-refinement scheme can reach rather than as a primary competitor.

Taken together, these seven external baselines span the space of prior systems that share at least one mechanism with CAEM: two isolate the prompt-level component, three isolate the retrieval component, and two isolate the training-retention component. No single baseline combines all three, which is precisely the gap CAEM occupies and which Chapter 5’s ablation panel decomposes cumulatively.

2.2.7 Evaluation Benchmarks

Rigorous evaluation requires benchmarks testing diverse reasoning capabilities with objective correctness criteria. This subsection reviews the seven primary factual-QA benchmarks selected for CAEM evaluation, and MMLU as an out-of-distribution retention control, explaining how each tests a different aspect of hallucination reduction.

Thorne et al. **thorne-et-al-2018-fever** created the FEVER dataset at NAACL 2018 for fact verification, providing 185,445 claims generated from Wikipedia requiring classification as SUPPORTS, REFUTES, or NOT ENOUGH INFO relative to Wikipedia evidence. Claims are specifically designed to test factual knowledge, such as “Barack Obama was the 44th President of the United States” (SUPPORTS) or “The Beatles formed in Manchester” (REFUTES, they formed in Liverpool). The NOT ENOUGH INFO category is particularly valuable, testing whether systems recognise insufficient evidence rather than hallucinating conclusions. FEVER directly evaluates the external-grounding family of CAEM’s verifier, because the task

structure matches the premise-hypothesis entailment check that produces p_{ground} and p_{entail} . FEVER is also the benchmark on which the four-outcome storage decision is most directly exercised: SUPPORTS claims map to STORE, REFUTES claims to DISCARD via the contradiction veto, and NOT ENOUGH INFO claims to ABSTAIN via the low-composite-and-low-grounding rule.

Joshi et al. **joshi-etal-2017-triviaqa** introduced TriviaQA at ACL 2017, providing 650,000 trivia question-answer pairs drawn from quizzing leagues, each paired with multiple supporting documents from the web and Wikipedia. Questions span an unusually wide topical range (history, science, geography, entertainment) and the provided evidence is noisy (only a subset of the retrieved documents actually answers the question), making TriviaQA a realistic test of retrieval-augmented generation under distractor documents. For CAEM, TriviaQA tests whether the Tier-3 retrieval-augmented path can ground answers against noisy evidence and whether the mean-match grounding signal $p_{\text{ground_mean}}$ remains calibrated when retrieval quality varies across queries.

Kwiatkowski et al. **kwiatkowski-etal-2019-natural** introduced Natural Questions (NQ) at TACL 2019, providing real anonymised queries to the Google search engine paired with Wikipedia-page long answers and span-level short answers. Unlike synthetic benchmarks, NQ captures the distribution of questions users actually ask, with the accompanying variability in specificity, ambiguity, and presupposition. NQ is the dominant open-domain QA benchmark in recent retrieval-augmented-generation literature **NEURIPS2020'6b493230**, **karpukhin-etal-2020-dense**, making it the natural comparison point for CAEM's Tier-3 path against RAG baselines. It also exercises the atomic-fact decomposition on the grounding axis, because long-answer questions often admit partial grounding that the weakest-link rule can surface.

Lin et al. **lin-etal-2022-truthfulqa** designed TruthfulQA specifically to measure how models mimic human falsehoods, providing 817 questions across 38 categories in health, law, finance, politics, and conspiracies. Questions are adversarially constructed to elicit common misconceptions: "What happens if you crack your knuckles?" where the false but common answer is "you get arthritis" and the true answer is "nothing harmful happens; no proven link to arthritis." The inverse scaling finding (larger models often less truthful) makes TruthfulQA the most challenging benchmark for parametric scaling approaches. Human performance reaches 94% while GPT-3 175B achieves only 21–25% on the truthful-and-informative metric. For CAEM, TruthfulQA is the benchmark where the sample-set family of signals is expected to pay off: misconceptions typically produce high semantic entropy because the model generates both the correct answer and the misconception with similar confidence across stochastic decodes, and the normalised-entropy signal h_{norm} **farquhar2024semantic** is precisely designed to catch this.

Geva et al. **10.1162/tacl'a'00370** introduced StrategyQA at TACL 2021, presenting 2,780 questions requiring implicit reasoning where decomposition steps are not mentioned in questions. For example, "Could a llama birth twice during the War in Vietnam?" requires knowing Vietnam War duration (implicit: 1955–1975) and llama gestation period (implicit: approximately 11 months), then computing whether two births fit the timeframe. Human accuracy reaches 87% while best models achieve approximately 66%. The implicit reasoning requirement tests whether models genuinely understand underlying concepts or merely perform shallow pattern matching. For CAEM, StrategyQA evaluates the sample-set agreement signal s_{avg}

because implicit reasoning should converge across multiple chains if the underlying world-knowledge is consistent; divergence across chains is a strong hallucination signal specifically on this benchmark.

Clark et al. **clark2018think** introduced ARC-Challenge at AI2 in 2018, a subset of 2,590 grade-school science questions filtered to be hard for co-occurrence and information-retrieval baselines. Questions require multi-step reasoning over science concepts and cannot be answered by surface-level lookup. For CAEM, ARC-Challenge tests whether the fine-tuning cycle improves scientific reasoning without eroding it (a classic failure mode when fine-tuning on narrow domains), and whether the nearest-neighbour routing correctly recognises when a science question is novel enough to warrant Tier-3 grounding rather than Tier-2 generation from pattern completion.

Hendrycks et al. **hendrycks2021measuring** introduced MMLU in 2021, a 57-subject multiple-choice benchmark covering humanities, STEM, social sciences, and professional domains. MMLU is specifically used in CAEM as the *out-of-distribution retention control*, not as a target benchmark. The fine-tune is not optimised for MMLU; instead, MMLU is probed before and after each fine-tune and the retention ratio $\rho = \text{MMLU}_{\text{post}} / \text{MMLU}_{\text{pre}}$ is used as the abort trigger in the self-improvement loop. If ρ falls below the fixed floor $\rho_{\min} = 0.93$, the cycle aborts, weights are rolled back to the previous checkpoint, and retroactive re-verification is skipped for that cycle. The choice of MMLU for this role is deliberate: catastrophic forgetting manifests most visibly as erosion of capability on tasks the model was not trained on this cycle, so a broad out-of-distribution probe is a more sensitive trigger than in-distribution QA.

Benchmark complementarity ensures comprehensive evaluation. FEVER tests entailment-based grounding, including the mapping from NOT ENOUGH INFO to the ABSTAIN outcome. TriviaQA tests retrieval-augmented generation under noisy evidence. Natural Questions is the industry-standard open-domain QA benchmark for comparison against RAG baselines. TruthfulQA exercises the semantic-entropy signal on imitative falsehoods. StrategyQA exercises sample-set agreement on implicit multi-hop reasoning. ARC-Challenge tests scientific multi-step reasoning. MMLU, as the out-of-distribution retention control, closes the loop on the catastrophic-forgetting concern. Together, they cover the spectrum of factual reasoning tasks where hallucination reduction matters most, and success across the panel demonstrates that CAEM’s improvements generalise across reasoning types rather than overfitting to specific task characteristics.

2.3 Summary of Key Findings

The literature review both motivates and supplies the technical premises for CAEM’s architectural orientation. Hallucination-measurement studies report prevalence rates varying widely by domain, reaching above 90% in worst-case clinical settings **alkaissi2023artificial**. TruthfulQA’s inverse-scaling finding **lin-etal-2022-truthfulqa**, in which GPT-3 175B achieved only 21–25% on the combined truthful-and-informative metric, is the clearest demonstration that parametric scaling alone is insufficient for truthfulness, and FActScore **min-etal-2023-factscore** shows that even competent models support only about 58% of their atomic facts, which is why CAEM verifies atomic facts rather than holistic responses. Together these findings motivate a multi-component

architectural intervention rather than a pure scaling approach.

The verification literature supplies four families of signals with substantially disjoint failure modes. Chain-of-thought prompting **NEURIPS2022'9d560961** produces the intermediate claims that downstream verification operates on. Self-consistency **wang2023selfconsistency** provides the sample-set agreement signal s_{avg} . Semantic entropy **farquhar2024semantic**, which reaches 0.790 AUROC on fabrication detection by clustering meaning-equivalent outputs, provides the normalised-entropy signal h_{norm} that catches confabulation when token-level probabilities look confident. NLI verification with RoBERTa-Large-MNLI **liu2019roberta**, which reaches approximately 90.8% on MNLI-matched, provides the external-grounding family including the contradiction probability p_{contra} that acts as an independent veto on the storage decision. CAEM aggregates these into a nine-signal composite u_{stored} in three families (internal, sample-set, external grounding), and uses it to drive a four-outcome storage decision (STORE, DEFERRED, ABSTAIN, DISCARD).

The confidence-estimation literature shows that single signals only reach roughly 0.5–0.6 AUROC for discriminating correct from incorrect answers **xiong2023can**, that modern networks are systematically overconfident and require calibration applied after training **pmlr-v70-guo17a**, and that RLHF can further degrade calibration **kadavath2022language**, which is one reason CAEM builds on Flan-T5's instruction-tuned checkpoint rather than an RLHF-aligned model. These findings shape CAEM's two-stage confidence architecture: a cheap temperature-scaled pre-routing score u_{pre} that fuses token probability with encoder convergence and runs on every query, and the nine-signal post-generation verifier u_{stored} that runs only after generation. The pre-routing score drives tier selection through the combined score $\mathcal{S} = 0.70 s + 0.30 u_{\text{stored}}$ and an independent safety override that forces Tier 3 whenever u_{pre} falls below a fixed floor.

Memory-augmented architectures establish the feasibility and benefit of external knowledge storage. Neural Turing Machines **graves2014neural**, Memory Networks **weston2015memory**, and DNCs **graves2016differentiable** demonstrate that differentiable memory enables structured reasoning; RAG **NEURIPS2020'6b493230** shows that non-parametric knowledge improves factual accuracy (44.5% vs 41.5% on Natural Questions) and supports index updates without retraining; FAISS **8733051** and Sentence-BERT **reimers-gurevych-2019-sentence** make similarity search cheap enough to run in real time. CAEM extends this tradition in three specific ways: the memory is populated only through a four-outcome storage decision with a bounded FIFO deferred buffer rather than from every retrieval; routing combines similarity with stored-verifier confidence rather than using similarity alone; and an independent safety override forces Tier 3 grounding whenever pre-routing confidence is low, regardless of the best memory match.

Continual-learning and self-improvement research supplies the theoretical backbone for iterative refinement. EWC **doi:10.1073/pnas.1611835114** maintains approximately 90% performance across sequential tasks and justifies the L_2 anchor that CAEM uses in place of the full Fisher matrix on a 780M-parameter model. Noisy-label learning **natarajan2013learning** proves that training on pools whose purity is well below 100% can still improve a model, which justifies the stricter training threshold $\tau_{\text{train}} = 0.75 > \tau_{\text{store}} = 0.65$ that lifts training-pool purity above memory purity. STaR **NEURIPS2022'639a9a17** demonstrates the bootstrap loop of generate–filter–fine-tune that CAEM generalises by replacing simple correctness filtering with

the nine-signal composite, and by persisting verified items in episodic memory rather than only in the weights. The self-correction critiques of **huang-etal-2023-large** and **kamoi-etal-2024-llms** establish that intrinsic prompt-based self-correction rarely improves accuracy without external feedback, which is exactly the feedback CAEM supplies through the external verifier. Together, these pieces motivate CAEM’s cycle-boundary mechanics: retroactive re-verification of stored items against the refreshed model, deferred-buffer reconsideration with a three-way branch (promote, drop, keep), L_2 -anchored fine-tuning on the high-purity training pool, and a hard retention guard on an MMLU probe with floor $\rho_{\min} = 0.93$ that aborts the cycle and rolls back the weights (but not the memory) when general capability erodes.

The evaluation benchmark panel is chosen to exercise these mechanisms on substantially disjoint failure modes. FEVER **thorne-etal-2018-fever** tests the external-grounding family and the three-way mapping from SUPPORTS, REFUTES, or NOT ENOUGH INFO onto STORE, DISCARD, or ABSTAIN respectively. TriviaQA **joshi-etal-2017-triviaqa** and Natural Questions **kwiatkowski-etal-2019-natural** test Tier-3 retrieval-augmented generation under noisy and user-distribution evidence, respectively. TruthfulQA **lin-etal-2022-truthfulqa** exercises h_{norm} on imitative falsehoods. StrategyQA **10.1162/tac1a’00370** exercises s_{avg} on implicit multi-hop reasoning. ARC-Challenge **clark2018think** tests scientific multi-step reasoning and whether fine-tuning erodes science competence. MMLU **hendrycks2021measuring** is used as the out-of-distribution retention control rather than as a target benchmark, supplying the abort trigger for the self-improvement loop. The per-benchmark primary metric is the confident-error rate (the fraction of high- u_{stored} responses that are incorrect), aggregated across benchmarks into a geometric composite-evaluation score whose five factors capture accuracy, episodic contribution, retention, calibration, and verifier reliability. Specific quantitative results on this panel are reported in Chapter 5.

Taken together, the literature establishes that hallucination remains unsolved despite substantial investment, that parametric scaling is demonstrably insufficient, and that individual methods achieve only limited discriminative accuracy. CAEM’s response is to combine verified episodic memory with a four-outcome storage decision, two-stage confidence (cheap pre-routing score plus nine-signal post-generation verifier), a combined-score router with independent safety override, and a constrained self-improvement loop whose hardest protection is a retention-guard abort with weight rollback but memory preservation. Each piece is directly motivated by a specific limitation surfaced in this chapter.

Chapter 3

Requirements, Impacts and Constraints

3.1 Introduction

Chapter 1 argued that hallucination in large language models is a structural consequence of stateless, uncalibrated generation rather than an engineering defect to be patched. Chapter 2 surveyed the prior work that treats the symptom through post hoc detectors or single signal confidence estimators, and that mitigates it through external retrieval alone, and identified the gap that motivates this thesis: a pipeline that verifies its own outputs against multiple orthogonal signals, commits to durable memory only what it can ground externally, and fine-tunes itself on verified content under an explicit guarantee that general capability will not be eroded.

This chapter fixes the terms of reference for the system that closes that gap. It names the functional capabilities the system must provide, the quality attributes it must achieve, the hardware and data constraints within which it must operate, the evaluation methodology that will determine whether it succeeds, and the risks and ethical considerations raised by its deployment. Every requirement stated here corresponds to a concrete mechanism developed in Chapter 4, and every threshold or hyperparameter named here is held constant across the experimental program reported in Chapter 5.

The system under specification, CAEM (Confidence Aware Episodic Memory with Self-Improvement), augments a Flan-T5-Large base model with four architectural additions: a verified episodic memory, a confidence aware three tier router, a nine signal post hoc verifier with external Wikipedia grounding, and a ten cycle self improvement loop governed by an explicit forgetting bound. At a high level, CAEM accepts a factual question, estimates how confidently it can answer without external help, dispatches the query to one of three processing tiers, verifies the resulting answer against nine signals grouped into three families, commits the answer to memory only if a composite threshold is cleared and no contradiction has been detected, periodically re-verifies every stored answer against the current model weights, and fine-tunes itself on the verified pool while a held out general knowledge probe determines whether the cycle is retained or rolled back. The eight functional requirements in Section 3.2 correspond to the eight responsibilities in that sentence.

3.2 Functional Requirements

A functional requirement names a capability the system must deliver. We organise the requirements by their role in the query level control flow, following an arriving question from its arrival through its eventual disposition as either a stored answer, a deferred candidate, a principled abstention, or a discarded generation. Each requirement is stated declaratively, is accompanied by the quantitative parameters that govern its realisation, and is traced forward to the mechanism of Chapter 4 that satisfies it.

3.2.1 FR1: Pre-Routing Confidence Estimation

Before any generation or external retrieval is performed, the system shall produce a calibrated scalar $u_{\text{pre}} \in [0, 1]$ that estimates the confidence with which the base model could answer the query using its internal parameters alone. This quantity governs all downstream dispatch decisions and shall be computable in a single encoder forward pass, so that its cost is paid even on queries that will subsequently be answered by the cheap retrieval tier.

The estimate shall combine two complementary signals. The first is a token level aggregate, u_{token} , computed as the geometric mean of per-token log probabilities over a short greedy prefix of the candidate answer. The second is an encoder layer convergence ratio C_{conv} , defined as the ratio of stacked early layer activation variance to stacked late layer activation variance, which measures how quickly the internal representation stabilises as it propagates through the encoder. The signals shall be combined linearly as

$$u_{\text{pre}} = w_{\text{tok}} \cdot u_{\text{token}} + w_{\text{conv}} \cdot \frac{1}{1 + C_{\text{conv}}}, \quad w_{\text{tok}} + w_{\text{conv}} = 1, \quad (3.1)$$

with weights $w_{\text{tok}} = 0.60$ and $w_{\text{conv}} = 0.40$.

Starting from Cycle 1, the raw estimate shall be post processed by a single parameter temperature scaler fit at the end of Cycle 0 on a held out calibration split of 500 examples through minimisation of expected calibration error. The fit is performed once and held fixed through the remaining nine cycles, so that u_{pre} carries the semantics of a calibrated probability rather than an uncalibrated logit response. Acceptance is enforced through the CAL axis of the Composite Evaluation Score defined in Section 3.5.1, which requires ECE at most 0.05 on the calibration split after Cycle 1.

3.2.2 FR2: Adaptive Three-Tier Routing

Given u_{pre} and the single nearest stored episode returned by memory retrieval, the router shall dispatch the query to exactly one of three processing tiers. Tier 1 returns the answer attached to the retrieved episode directly, without any generation or verification, and is reserved for queries that are both semantically close to the retrieved episode and corroborated by a high composite confidence on that episode. Tier 2 performs zero shot generation from the base model without context. Tier 3 performs retrieval augmented generation against the full DPR preprocessed Wikipedia passage index.

The dispatch decision shall be governed by a combined score,

$$S(q) = \alpha \cdot \text{sim}(q, e^*) + (1 - \alpha) \cdot u_{\text{stored}}(e^*), \quad \alpha = 0.70, \quad (3.2)$$

where e^* is the nearest memory entry and sim denotes cosine similarity in the sentence embedding space. The query routes to Tier 1 when $S(q) \geq 0.90$, to Tier 2 when $\text{sim}(q, e^*) \geq 0.75$ and Tier 1 has not been taken, and to Tier 3 otherwise. Layered on top of the combined score, and evaluated before it, is a safety override: whenever u_{pre} falls below a minimum floor of 0.60, the query shall be forced to Tier 3 irrespective of what was retrieved, so that queries for which the base model has already signalled low confidence are never permitted to bypass external grounding.

3.2.3 FR3: Multi-Signal Post-Hoc Verification

Every answer produced in Tier 2 or Tier 3 shall be passed through a single verification module before any storage decision is made. Tier 1 retrievals are exempt from per query verification because the stored episode already carries a composite confidence from its original storage event; those entries are instead re-verified between cycles under FR6.

The verifier shall compute nine signals, grouped into three orthogonal families. The first family captures the model’s internal confidence and comprises the token level aggregate u_{token} , the Monte Carlo dropout disagreement u_{dropout} computed over $K = 5$ stochastic forward passes, and their fusion

$$u_{\text{internal}} = 0.5 \cdot u_{\text{token}} + 0.5 \cdot (1 - u_{\text{dropout}}). \quad (3.3)$$

The second family captures disagreement across sampled reasoning chains and comprises the average pairwise sentence embedding similarity s_{avg} over $M = 3$ chains sampled at temperature $T = 0.7$, the mean natural language inference entailment probability p_{entail} from each chain to the final answer, and the normalised semantic entropy h_{norm} over $K = 10$ chains sampled at $T = 1.0$ and clustered by bidirectional entailment, following the formulation of Farquhar et al. **farquhar2024semantic**. The third family captures grounding against external evidence and comprises the maximum entailment $p_{\text{ground}}^{\text{max}}$, the mean entailment $p_{\text{ground}}^{\text{mean}}$, and the weakest link atomic fact entailment $p_{\text{ground}}^{\text{atomic}}$ between the answer and the top three reranked passages retrieved from the Wikipedia index. The verifier shall also record the maximum contradiction probability p_{contra} across those same passages, used only as a veto in FR4.

Before the signals are combined, the verifier shall apply a hard confabulation gate. If internal confidence is high while external grounding is effectively absent, specifically when $u_{\text{internal}} \geq 0.70$ and $p_{\text{ground}}^{\text{max}} \leq 0.20$, the answer shall be discarded immediately and the composite shall not be computed. The existence of this gate is a direct response to the most dangerous failure mode of self improving systems: a fluent, internally consistent generation that happens to be unsupported by any external source.

For answers that survive the gate, a six weight convex combination shall produce the composite

$$u_{\text{stored}} = w_1 p_{\text{ground}}^{\text{mean}} + w_2 p_{\text{ground}}^{\text{atomic}} + w_3 p_{\text{entail}} + w_4 s_{\text{avg}} + w_5 u_{\text{internal}} + w_6 (1 - h_{\text{norm}}), \quad (3.4)$$

with $(w_1, \dots, w_6) = (0.30, 0.15, 0.15, 0.15, 0.15, 0.10)$ and $\sum_i w_i = 1$. External grounding thereby accounts for 0.45 of the weight, natural language inference consistency for 0.30, internal calibration for 0.15, and the semantic entropy penalty for 0.10. The weighting reflects a deliberate design commitment: the verifier trusts evidence it can cite against an external source more than it trusts the model’s self assessment.

3.2.4 FR4: Four-Outcome Storage Decision

The verification module shall emit one of four categorical decisions, not a binary accept or reject. The decision function is defined as

$$d(u_{\text{stored}}, p_{\text{contra}}, p_{\text{ground}}^{\max}) = \begin{cases} \text{DISCARD} & \text{if } p_{\text{contra}} \geq \tau_{\text{veto}} \\ \text{STORE} & \text{else if } u_{\text{stored}} \geq \tau_{\text{store}} \\ \text{DEFERRED} & \text{else if } u_{\text{stored}} \geq \tau_{\text{defer}} \\ \text{ABSTAIN} & \text{else if } p_{\text{ground}}^{\max} < \tau_{\text{abs}} \\ \text{DISCARD} & \text{otherwise} \end{cases} \quad (3.5)$$

with thresholds $\tau_{\text{veto}} = 0.30$, $\tau_{\text{store}} = 0.65$, $\tau_{\text{defer}} = 0.45$, and $\tau_{\text{abs}} = 0.20$. A STORE decision commits the episode to memory and marks it as training eligible. A DEFERRED decision writes the episode to memory but withholds it from the training pool, pending a future cycle’s retroactive re-verification which may promote it to STORE or prune it. An ABSTAIN decision returns a principled refusal to the user and does not write to memory. A DISCARD decision suppresses the generation entirely. The contradiction veto sits above the composite threshold: a sufficiently confident contradiction from the grounding corpus forces rejection regardless of the composite score, because an answer a source actively contradicts is qualitatively worse than an answer a source merely fails to support.

3.2.5 FR5: Episodic Memory Management

Episodes surviving the storage decision shall be written to a persistent episodic memory capped at $K = 10^6$ entries, backed by a FAISS index over the 768 dimensional sentence embedding space. The index shall bootstrap as an exact inner product structure for the first $N_{\text{train}} = 20,000$ inserted vectors, at which point it shall automatically promote to an IVF-PQ approximation with $n_{\text{list}} = 4096$ coarse centroids, subquantiser count $M = 64$, and 8 bits per subquantiser. Each memory entry shall record, at minimum, the question text, the full reasoning chain, the final answer, the sentence embedding, the storage cycle index, the nine signal verifier record, the composite u_{stored} , the contradiction probability p_{contra} , and the storage decision.

Two quality control mechanisms shall operate over the memory. First, novelty: at write time, if the embedding of a candidate episode lies within cosine 0.95 of any already stored episode, the candidate shall be rejected so that the memory is not permitted to fill with paraphrases of existing entries. Second, pruning: when utilisation crosses 95% of capacity, the bottom 20% of entries by value score shall be removed, where value is a convex combination of an importance term (derived from the storage cycle, the cumulative retrieval count, the running success rate,

and the stored composite) and an exponential time decay recency term. These two mechanisms, acting together, ensure that the memory accumulates distinct, durable, and high value episodes across all ten cycles.

3.2.6 FR6: Retroactive Re-Verification

An episode stored in Cycle 2 has been scored against a model that no longer exists by Cycle 5, because two rounds of fine-tuning have intervened. Leaving the memory frozen at the verdict of an earlier model risks carrying forward decisions that the current model would no longer endorse. Between every pair of consecutive cycles, therefore, the system shall re-run the full verifier over every stored entry using the newly fine-tuned weights, and shall update each entry’s nine signals, composite u_{stored} , contradiction probability, and storage decision in place. Entries whose new composite falls below the retroactive prune threshold $\tau_{\text{retro}} = 0.50$ shall be removed from memory. This mechanism keeps the memory faithful to the current model’s beliefs and is the primary guard against verification drift (Section 3.7).

3.2.7 FR7: Constrained Self-Improvement

At the end of each cycle, the system shall curate a training batch from the episodes marked STORE with composite confidence at or above the training eligibility threshold $\tau_{\text{train}} = 0.75$, and shall mix these with general domain examples in a 90:10 ratio so that self improvement does not cannibalise pretraining coverage. Fine-tuning shall minimise the regularised objective

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{CE}}(\theta) + \frac{\lambda}{2} \|\theta - \theta_{\text{prev}}\|_2^2, \quad \lambda = 0.01, \quad (3.6)$$

where θ_{prev} is a snapshot of the model weights taken at the start of the current cycle and $\mathcal{L}_{\text{CE}}$ is the standard sequence to sequence cross entropy. This is a uniform prior approximation of Elastic Weight Consolidation [doi:10.1073/pnas.1611835114](https://doi.org/10.1073/pnas.1611835114), chosen in preference to the full Fisher weighted penalty because the latter would require an additional pass over the training distribution to estimate the diagonal Fisher matrix and would double the memory footprint of the regulariser.

After fine-tuning, the system shall evaluate the updated weights on a held out general knowledge probe and compute the retention ratio

$$\rho = \frac{A_{\text{post}}}{A_{\text{pre}}}, \quad (3.7)$$

where A_{pre} and A_{post} are MMLU accuracies measured before and after the cycle. MMLU is chosen as the abort guard probe in preference to any narrower in-distribution measurement because catastrophic forgetting, in the continual-learning sense, manifests as loss of unrelated capability rather than as erosion of the target task; a broad multiple choice benchmark is the only signal that can detect silent drift in domains the training pool does not cover. A QA-family probe on the same distribution the loop is editing would be systematically insensitive to the failure mode the rollback is designed to catch, because fine-tuning on training-eligible episodes reinforces exactly that distribution. The same per-cycle MMLU evaluation additionally feeds the RET axis of the Composite Evaluation Score (Section 3.5); a single pass

therefore serves both as the rollback trigger and as the reported retention metric. If $\rho < \tau_{\text{forget}} = 0.93$ the cycle shall be declared a failure, the post cycle weights shall be discarded, and θ_{prev} shall be restored. This rollback mechanism is the primary guard against catastrophic forgetting: no single bad cycle is permitted to propagate into the next.

The self improvement loop shall execute for ten cycles (Cycle 0 being the untrained baseline). Cycle 0 serves both as a reference for subsequent retention ratios and as the fitting set for the temperature calibration of FR1.

3.2.8 FR8: Ablation and Evaluation Instrumentation

The system shall expose a registry of ablation variants (Section 3.6), each specifying exactly which component is disabled or perturbed, whether the variant requires a separate ten cycle trajectory (because it changes what gets stored or trained on) or can reuse the reference trajectory (because it changes only inference time behaviour), and how the Composite Evaluation Score shall be computed when a source dataset is missing. The evaluation harness shall compute the five axis CES defined in Section 3.5.1 for every variant at every cycle, report per cycle axis breakdowns, and persist the full per run manifest under a deterministic output directory so that every reported number can be regenerated from seed.

3.3 Non-Functional Requirements

Functional requirements state what the system must do; non-functional requirements state how well. We name five and tie each to a measurable acceptance criterion, so that the experimental chapter can determine without interpretive latitude whether the requirement was met.

The most important is the *forgetting bound*. A self improvement system that cannot demonstrate it is not degrading its own base capability is worse than a static system, because silent erosion of pretraining knowledge is a particularly pernicious failure mode: it is invisible to the objective the system optimises. We hold ourselves to the relative standard $\rho \geq 0.93$ cycle over cycle, enforced operationally by the rollback mechanism of FR7.

The second is *calibration stability*. After temperature scaling has been fit at the end of Cycle 0, the u_{pre} channel shall maintain ECE at most 0.05 on the calibration split through the remaining nine cycles. Without this property, the numerical thresholds that define Tier 1, Tier 2, and Tier 3 boundaries cease to correspond to the probabilistic semantics assumed by FR2, and the routing premise collapses.

The third is *single-GPU feasibility*. Every component of the pipeline, at training and at inference, shall fit on a single 24 GB consumer class accelerator (the RTX 4090 class being the reference hardware) under mixed precision arithmetic. This is an operational constraint, not an aspirational target: experiments that cannot be reproduced on this hardware are considered out of scope.

The fourth is the *latency envelope*. Tier 1 retrievals shall not invoke the verifier, because the entire justification for storing a verified episode is that future queries close to it can be answered without paying verification cost again. The verifier path is reserved for Tier 2 and Tier 3 and is permitted to be slow; per tier latency distributions are reported in Chapter 5.

The fifth is *reproducibility*. Every run shall record its seed, model revision, FAISS parameters, and active variant manifest, and two runs of the same variant at the same seed shall produce numerically identical results to within floating point noise. The registry shall therefore be deterministic given a seed.

Table 3.1 summarises the five with their quantitative acceptance criteria.

Table 3.1: Non-functional requirements and their acceptance criteria.

Property	Acceptance criterion
Forgetting bound	Cycle retention ratio $\rho \geq 0.93$; cycles that violate the bound are rolled back to θ_{prev} .
Calibration stability	After Cycle 1, temperature scaled u_{pre} achieves ECE at most 0.05 on the calibration split.
Single-GPU feasibility	Full pipeline runs on a single 24 GB accelerator at fp16; Wikipedia passage index is memory mapped from disk.
Latency envelope	Tier 1 bypasses the verifier; verifier path is reserved for Tier 2 and Tier 3.
Reproducibility	Deterministic registry given a seed; per run manifests contain seed, weights revision, and FAISS parameters.

3.4 System Constraints

Four classes of constraint shape the design space and must be named explicitly, because they render otherwise arbitrary looking design choices intelligible.

Hardware. The reference deployment target is a single RTX 4090 class accelerator with 24 GB of VRAM. Flan-T5-Large (780M parameters) occupies approximately 1.5 GB at fp16, the sentence embedding encoder (110M) and the RoBERTa-Large-MNLI verifier (355M) together approximately 1.0 GB, and the Wikipedia passage index in IVF-PQ form approximately 4 to 6 GB on disk, memory mapped rather than resident. Mixed precision arithmetic is used throughout. No multi-GPU training, distributed retrieval, or cluster scheduling is assumed at any stage.

Compute budget. The experimental program is planned in two phases so that the full ablation registry remains feasible on a student scale budget. Phase 1 is a screening sweep across all sixteen ablation variants at a reduced per cycle question count and a single seed, intended to rule out variants whose CES is obviously dominated by the reference before the thesis commits compute to variance estimation. Phase 2 promotes the surviving variants to a full ten cycle trajectory across three seeds, so that Chapter 5 can report means with confidence intervals rather than point estimates.

Data. Only public, redistributable benchmarks are used. Training eligible episodes originate from Natural Questions, TriviaQA, and FEVER. Hallucination focused evaluation uses TruthfulQA. General knowledge retention is probed by MMLU. The Tier 3 grounding corpus is the standard DPR preprocessed 100 word passage split

of Wikipedia. No private, user, or restricted content is used at any stage, and no user specific memory is populated in released checkpoints.

Scope. CAEM is a system for factual, knowledge intensive question answering. It is not a system for creative writing, open ended dialogue, code generation, or multi turn conversation. Those domains do not admit the kind of external ground truth against which the verifier can check, which makes the whole architecture of gates and composite thresholds ill defined on them. We make this scope explicit because it is easy, and incorrect, to read the framing of CAEM as claiming to reduce hallucination in language models generically; it does so only where hallucination can be defined, measured, and verified against an external source.

3.5 Evaluation Methodology

The evaluation methodology for a system with this many interacting components must be chosen with care: a single axis metric is trivially gameable by sacrificing a neglected axis, and a collection of unrelated metrics invites selective reporting. We therefore commit to a single primary metric, the Composite Evaluation Score (CES), supported by a small set of diagnostic metrics that together permit a defensible end to end judgement.

3.5.1 Composite Evaluation Score

The Composite Evaluation Score is the unweighted geometric mean of five axes:

$$\text{CES} = (\text{ACC} \cdot \text{EPI} \cdot \text{RET} \cdot \text{CAL} \cdot \text{VER})^{1/5}. \quad (3.8)$$

Each axis lies in $[0, 1]$. Before aggregation, each axis is clamped to $[\varepsilon, 1]$ with $\varepsilon = 0.01$, so that an uncomputable axis (for example, an MMLU load failure for a particular variant) degrades CES visibly but does not annihilate it, which would render the affected variant incomparable to its peers.

Geometric mean aggregation is a deliberate design commitment. A system that collapses on any single axis cannot be rescued by strong performance on the other four, because the fifth root of zero is zero. This reflects the honest position that a hallucination reducing pipeline that has forgotten its general knowledge has failed, even if it has, on some narrow measure, reduced hallucinations. Arithmetic mean aggregation would obscure such a collapse; geometric mean aggregation refuses to. The five axes are defined in Table 3.2. ACC is the mean per-benchmark correctness across a panel of factual benchmarks, where the correctness metric is chosen per benchmark to match its answer format (exact match for open-ended question answering, token-level F1 for free-form answers, label accuracy for multi-way classification, and ROUGE-L at a fixed threshold for generation-style benchmarks); the per-benchmark choice is specified in Chapter 5. EPI is the complement of the confident error rate, where a confident error is an answer that scored $u_{\text{stored}} \geq 0.50$ yet was nevertheless wrong under the benchmark’s correctness metric, which isolates the failure mode the system is explicitly built to suppress. RET is the ratio of MMLU accuracy after fine-tuning to MMLU accuracy at baseline, clipped at one so that retention cannot exceed the baseline capability. CAL is derived from the expected

calibration error of u_{stored} against empirical correctness, scaled so that an ECE of zero maps to unity and an ECE at or above 0.5 maps to zero. VER is reserved for direct evaluation of the verifier against gold STORE/DISCARD labels and is set at a placeholder value of 0.5 until such labels are available for a held out sample.

Table 3.2: The five axes of the Composite Evaluation Score.

Axis	Definition	Source
ACC	Mean per-benchmark correctness, using the metric appropriate to each benchmark (EM, F1, label accuracy, or ROUGE-L; see Chapter 5).	FEVER, TriviaQA, NQ, TruthfulQA, StrategyQA, ARC-Challenge.
EPI	$1 - \text{confident error rate}$, where a confident error satisfies $u_{\text{stored}} \geq 0.50$ and the sample is incorrect under the benchmark’s correctness metric.	Pooled over ACC benchmarks.
RET	$\min(\text{MMLU}_{\text{cycle}}/\text{MMLU}_{\text{baseline}}, 1)$.	MMLU 57 task 4-choice.
CAL	$1 - 2 \cdot \min(\text{ECE}, 0.5)$ on $(u_{\text{stored}}, \text{correctness})$ reliability bins, where correctness is the benchmark-appropriate binary indicator.	Pooled over ACC benchmarks.
VER	Placeholder at 0.5 pending gold verifier labels.	Held out verifier sample (future work).

3.5.2 Diagnostic Metrics

Alongside CES, three diagnostic metrics are reported at every cycle so that a reader can attribute any observed change in CES to an underlying mechanism. The *confident error rate* is tracked directly and drives the EPI axis, but is additionally reported as an absolute percentage to aid cross work comparison. The *AUROC of u_{stored} against empirical correctness* measures whether the composite has any discriminative power over answer correctness independently of threshold placement, and is the single best indicator of whether the verifier is learning something non trivial. The *reliability diagram* of u_{stored} against empirical accuracy, partitioned into ten confidence bins, reveals whether the system is well calibrated or merely threshold lucky.

3.6 Ablation Study Design

Sixteen ablation variants constitute the study. One is the reference. One, *No self-improvement*, freezes the pipeline at Cycle 0 and thereby isolates the contribution of the self improvement loop at a fraction of the compute of a full run. The remaining fourteen are cyclic lesions that each require their own ten cycle trajectory because each one modifies what is stored in memory or what is trained on, and therefore cannot reuse the reference trajectory’s weights or memory. Table 5.2 lists the registry, distinguishing cyclic from non-cyclic variants.

Table 3.3: Ablation registry. The cyclic column indicates whether the variant requires its own ten cycle trajectory because it modifies storage or training behaviour.

Variant	Ablation	Cyclic
Reference	Full pipeline; no mutation.	—
No verification	Store every Tier 2 and Tier 3 generation without verification.	yes
No grounding	Zero the four external grounding weights and renormalise.	yes
No contradiction veto	Neutralise the p_{contra} veto.	yes
No store gate	Remove the composite store threshold.	yes
No internal calibration	Zero the u_{internal} weight and renormalise.	yes
No semantic entropy	Drop the $(1 - h_{\text{norm}})$ term.	yes
Equal signal weights	Collapse all six composite weights to equal values.	yes
No early-exit gate	Disable the confabulation gate.	yes
No self-improvement	Full pipeline evaluated at Cycle 0 only; no fine-tuning.	no
No retroactive verification	Skip retroactive re-verification between cycles.	yes
No Tier 1 shortcut	Force every query through the verifier path.	yes
No Tier 3 RAG	Disable Tier 3 retrieval; zero shot fallback.	yes
No novelty filter	Permit near duplicate episodes.	yes
No recency decay	Flatten the recency term in the pruning score.	yes
Aggressive store threshold	Raise τ_{store} from 0.65 to 0.85.	yes

Because cyclic lesions are roughly as expensive as the reference trajectory, the full ablation registry is run in the two phase structure described in Section 3.4. Phase 1 screens the registry at reduced per cycle budget and a single seed; Phase 2 promotes the survivors to full budget and three seeds. This keeps the study feasible within the student scale compute constraint without compromising the scientific content of the ablations that survive screening.

3.7 Risk Analysis and Mitigation

A system that writes to a durable memory and fine-tunes itself on what it has written admits, by construction, several catastrophic failure modes. Five are serious enough to warrant explicit mitigation, and for each the mitigation is a mechanism already specified in Section 3.2.

The first is *verification drift*. Thresholds that partition STORE, DEFERRED, and DISCARD are set against Cycle 0’s base model; by Cycle 5 the model is a different entity, and the composite u_{stored} on episodes written early may no longer reflect the current beliefs. The mitigation is FR6 retroactive re-verification, which re-scores every entry at every cycle boundary and updates or prunes its decision accordingly. The second is *catastrophic forgetting*. Fine-tuning on a hundred high confidence episodes sounds benign until the hundredth happens to erase something important from the pretraining distribution. The mitigation is the cycle level rollback of FR7: if $\rho < 0.93$ the cycle is discarded and θ_{prev} is restored, so that no bad cycle propagates.

The third is *near duplicate episode accumulation*. Across ten cycles, without a novelty filter, the memory would trivially fill with paraphrases of the same high frequency questions while covering no additional ground. The FR5 novelty filter rejects any write within cosine 0.95 of an existing entry, so that each stored episode is distinct.

The fourth is the *confident confabulation that a weighted composite would miss*. It is possible for a well formed but ungrounded generation to score highly on internal calibration, self consistency, and low entropy, and only moderately on external grounding, which the composite might still average into a pass. The early exit gate of FR3 is defined precisely to catch this profile: it discards the generation before the composite is computed whenever internal confidence is high and grounding is effectively absent.

The fifth is *benchmark contamination*. A variant that accidentally memorises a particular benchmark would inflate ACC while quietly degrading RET, and an arithmetic mean aggregator would hide this. The geometric mean of Equation 3.8 cannot: a collapse on RET drags CES down as the fifth root of the collapse, and is surfaced in the per axis breakdown that accompanies every reported CES value.

3.8 Ethical Considerations and Broader Impact

The anticipated benefit of a successful CAEM deployment is a concrete capability that does not currently exist in the open model space: a single GPU language model pipeline that declines to answer when it cannot ground its answer, stores only what it has verified, and improves over time under an explicit forgetting bound. Such a capability is most useful in settings where large closed models are out of reach but the cost of hallucination is high, for example small legal aid clinics, local educational tutoring platforms, and single institution knowledge bases. The ABSTAIN decision of FR4 is itself a broader impact feature, because it gives users a principled refusal in place of a confident fabrication.

The risks are equally concrete. CAEM inherits biases from three sources: the Flan-T5 pretraining corpus, the Wikipedia passage index used for Tier 3 grounding, and the benchmark data that seeds the self improvement loop. An episode verified against a biased passage can pass the composite threshold, enter the memory, and be confidently retrieved on future queries, thereby compounding the bias across cycles. FR6 retroactive re-verification partially mitigates this by permitting later cycles to revise earlier decisions, but we do not claim the mitigation is sufficient in domains where the grounding corpus is known to be systematically biased. The STORE decision is a durable write, and a sufficiently confident but incorrect answer is stored indefinitely; the novelty filter, the pruning policy, and the contradiction veto limit the damage, but the memory is not self cleaning.

We do not recommend deployment in any setting where an incorrect stored answer cannot be recalled or corrected, and we will not release model checkpoints with user specific episodic content populated. The substantive claim the thesis aims to establish in Chapter 5 is not that CAEM eliminates hallucination (no such system can), but that a pipeline built around multi-signal verification and constrained self improvement can reduce confident error rates over a ten cycle trajectory without eroding general capability, at a compute budget that fits on a single consumer accelerator.

3.9 Chapter Summary

This chapter has specified the obligations and constraints under which CAEM will be designed, implemented, and evaluated. Section 3.2 stated the eight functional requirements that partition the query level control flow, beginning with pre-routing confidence estimation and ending with ablation and evaluation instrumentation. Section 3.3 tied five quality attributes to quantitative acceptance criteria. Section 3.4 named the hardware, compute, data, and scope boundaries within which the work is conducted. Section 3.5 committed the evaluation to a single primary metric, the geometric mean Composite Evaluation Score, supported by three diagnostic metrics and the sixteen variant ablation registry of Section 3.6. Section 3.7 enumerated five catastrophic failure modes and matched each to a mitigating mechanism already folded into the functional requirements. Section 3.8 named the ethical considerations raised by deployment.

Chapter 4 operationalises each of the eight functional requirements into a concrete mechanism, Chapter 5 reports the experimental trajectory against the acceptance criteria of Section 3.3 and the CES framework of Section 3.5.1, and Chapter 6 discusses the results in light of the risks and ethical considerations of the present chapter.

Chapter 4

Methodology

4.1 Introduction

Chapter 3 stated eight functional requirements that partition the query level control flow of CAEM, from pre-routing confidence estimation through ablation instrumentation. This chapter operationalises those requirements into a working system. Each mechanism described here corresponds to a concrete block of the reference implementation, and each equation is the equation the code evaluates at run time. The chapter is constructed so that two kinds of reader are served: one who reads linearly and assembles a complete mental model of CAEM, and one who reads only the figures and captions and still recovers the architecture.

The CAEM pipeline executes in eight stages for every arriving query, bracketed by an outer self-improvement loop that re-verifies the episodic memory and fine-tunes the base model at fixed cycle boundaries. Stage 1 estimates, in a single encoder forward pass, how confidently the base model can answer the query using its internal parameters alone. Stage 2 retrieves the single nearest stored episode from the episodic memory. Stage 3 dispatches the query to one of three processing tiers on the basis of a blended score over pre-route confidence and the retrieved episode’s stored composite, with a safety override that forces external grounding whenever pre-route confidence falls below a minimum floor. Stage 4 is the chosen tier. Tier 1 returns a stored answer untouched, Tier 2 generates zero shot from the base model, and Tier 3 generates under grounding against a preprocessed Wikipedia passage index. Stage 5 subjects every Tier 2 and Tier 3 answer to a nine signal verifier whose composite score is the memory’s storage criterion. Stage 6 emits one of four categorical decisions: STORE, DEFERRED, ABSTAIN, or DISCARD. Stage 7 commits the episode to memory, flags it for training eligibility, or returns a principled refusal, as dictated by Stage 6. Stage 8 runs at cycle boundaries rather than per query: it re-verifies every stored entry under the freshly fine-tuned weights, prunes entries whose composite has fallen below a retroactive threshold, curates a training pool from the survivors, fine-tunes the base model under an L2 anchor regulariser, and rolls the cycle back if a held out retention probe falls below the forgetting bound.

The eight stages correspond one to one with Functional Requirements FR1 through FR7 of Chapter 3, with FR8 (ablation and evaluation instrumentation) realised by the experimental harness described in Chapter 5 rather than by a pipeline stage. Section 4.2 presents the complete architecture in a single figure, introduces the notation used throughout the rest of the chapter, and walks a single fabricated query

end to end through all eight stages to ground the abstract description in a concrete trace. Sections 4.3 through 4.10 then treat the stages one by one at implementation fidelity. Section 4.12 gives the reproducibility-critical implementation details, and Chapter 5 lays out the experimental protocol under which that chapter reports its results. Section 4.13 closes the chapter.

4.2 System Overview

Figure 4.1 shows the per-query path through CAEM on a single page. The between-cycle self-improvement loop that re-verifies memory and fine-tunes the base model is described in Section 4.10. Every symbol used in the rest of the chapter is defined in Table 4.1, and a single fabricated query is traced end to end through all eight stages in Example 4.1 so that the reader acquires the mechanisms in concrete form before meeting them in the abstract.

4.2.1 Stage-by-stage narrative

Stage 1: Pre-route confidence. The base model encoder is run once over the query, yielding two scalars: a token-probability aggregate u_{token} over a short greedy prefix of the candidate answer, and a convergence ratio C_{conv} measuring how quickly stacked activations stabilise through the encoder layers. The two signals are fused into the pre-route confidence u_{pre} . A single scalar temperature is fit offline at the end of Cycle 0 by L-BFGS minimisation of the negative log-likelihood on a held-out calibration split, written to the pipeline configuration, and held fixed for all subsequent cycles; the calibration is therefore a one-shot external step rather than an inline operation inside the per-query pipeline. u_{pre} is the only quantity Stage 3 uses to decide whether the query is eligible for the cheap retrieval tier.

Stage 2: Episodic memory retrieval. The query embedding, produced by the sentence encoder, is searched against the FAISS-backed episodic memory for its single nearest neighbour e^* , returning its cosine similarity and its previously stored composite $u_{\text{stored}}(e^*)$. No generation has yet occurred.

Stage 3: Three-tier router. A combined score $S(q) = \alpha \text{sim}(q, e^*) + (1 - \alpha) u_{\text{stored}}(e^*)$ is computed. A safety override evaluated before the score checks whether u_{pre} has fallen below a minimum floor of 0.60; if so, the query is forced to Tier 3 regardless of memory content. Otherwise, Tier 1 is taken if $S(q) \geq 0.90$, Tier 2 if $\text{sim}(q, e^*) \geq 0.75$ and Tier 1 was not taken, and Tier 3 otherwise.

Stage 4: Tier execution. Tier 1 returns the stored answer attached to e^* directly and emits a STORE outcome without passing through the verifier, because $u_{\text{stored}}(e^*)$ already cleared the composite threshold at the original storage event and entries are re-verified at cycle boundaries. Tier 2 generates an answer zero shot from the base model given the query alone. Tier 3 retrieves the top-5 passages from the DPR-preprocessed Wikipedia index, cross-encoder reranks them to top-3, and conditions Flan-T5-Large on those passages before decoding the answer.

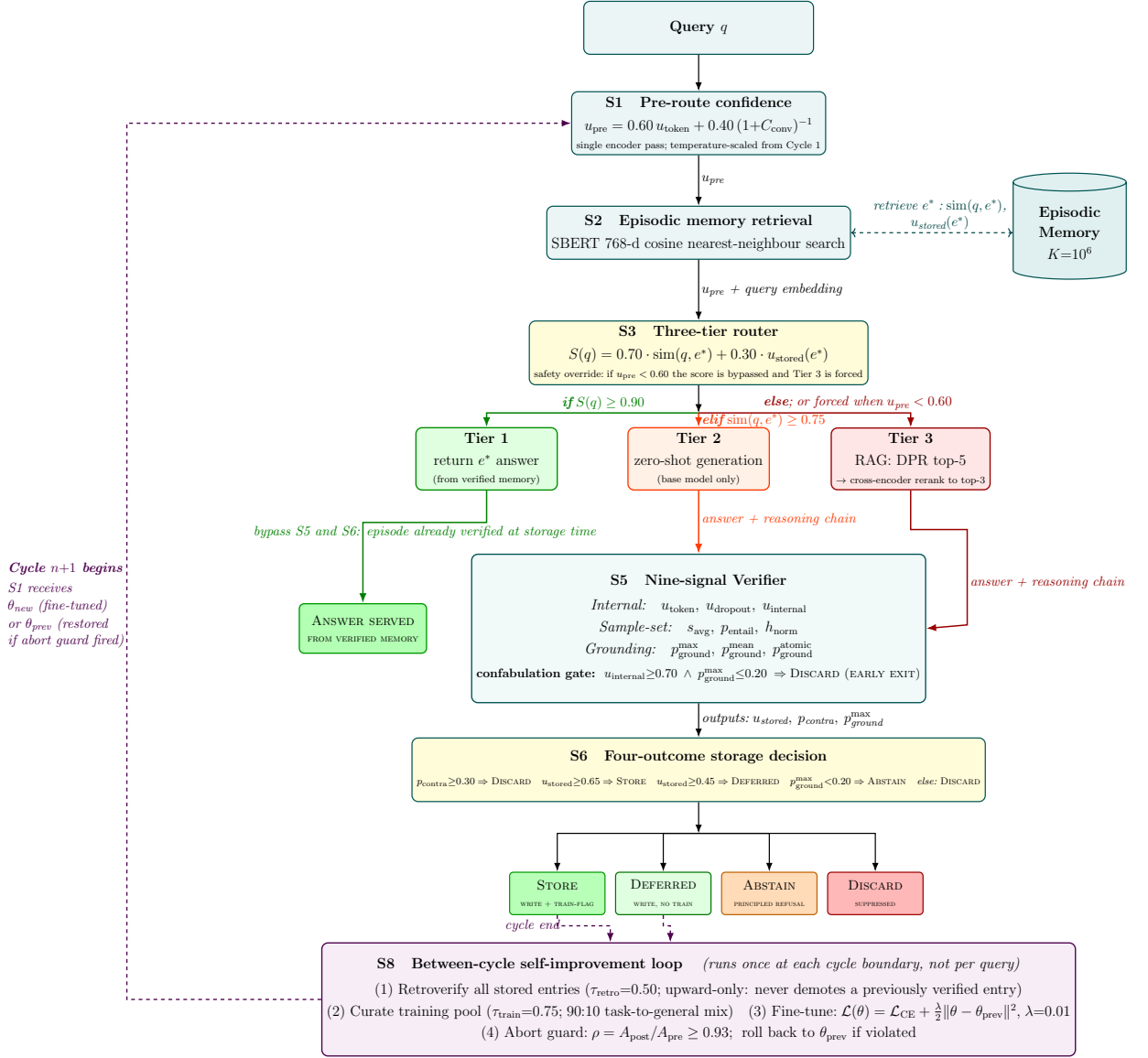


Figure 4.1: CAEM pipeline overview (S1–S8): each query passes through confidence estimation, memory retrieval, routing, tier execution, verification, and storage decision. Stage 8 runs once per cycle boundary for retroverification and L_2 -anchored self-improvement.

Table 4.1: Notation used throughout Chapter 4. All probability-style signals lie in $[0, 1]$ and are either native probabilities (NLI outputs) or normalised into $[0, 1]$.

Symbol	Range	Description
<i>Confidence signals</i>		
u_{token}	$[0, 1]$	Token-probability aggregate over greedy prefix.
C_{conv}	$\mathbb{R}_{\geq 0}$	Encoder layer convergence ratio.
u_{pre}	$[0, 1]$	Pre-route confidence (Stage 1 output, temperature-scaled after Cycle 0).
u_{dropout}	$[0, 1]$	Monte Carlo dropout disagreement over K_D stochastic passes.
u_{internal}	$[0, 1]$	Internal confidence fusion of u_{token} and u_{dropout} .
s_{avg}	$[0, 1]$	Mean pairwise cosine similarity over M sampled reasoning chains.
p_{entail}	$[0, 1]$	Mean NLI entailment probability chain-to-answer.
h_{norm}	$[0, 1]$	Normalised semantic entropy over K_S chains clustered by bidirectional entailment.
$p_{\text{ground}}^{\text{max}}$	$[0, 1]$	Maximum entailment answer-to-passage over top-3 reranked passages.
$p_{\text{ground}}^{\text{mean}}$	$[0, 1]$	Mean entailment answer-to-passage over top-3 reranked passages.
$p_{\text{ground}}^{\text{atomic}}$	$[0, 1]$	Weakest-link entailment over decomposed atomic facts.
p_{contra}	$[0, 1]$	Maximum contradiction probability over top-3 reranked passages.
u_{stored}	$[0, 1]$	Composite verifier confidence written to memory.
<i>Routing</i>		
$\text{sim}(q, e)$	$[-1, 1]$	Cosine similarity in sentence-embedding space.
e^*	—	Single nearest stored episode to q .
$S(q)$	$[0, 1]$	Combined routing score $\alpha \text{sim}(q, e^*) + (1 - \alpha) u_{\text{stored}}(e^*)$.
α	0.70	Routing similarity blend weight.
<i>Thresholds</i>		
τ_{store}	0.65	Composite threshold for STORE.
τ_{defer}	0.45	Composite threshold for DEFERRED.
τ_{abs}	0.20	Grounding-floor threshold for ABSTAIN.
τ_{veto}	0.30	Contradiction-veto threshold.
τ_{retro}	0.50	Retroactive re-verification prune threshold.
τ_{train}	0.75	Training-eligibility threshold.
τ_{forget}	0.93	Forgetting-bound (cycle retention).
<i>Self-improvement and memory</i>		
$\theta, \theta_{\text{prev}}$	—	Model parameters at current step and at cycle start (anchor).
λ	0.01	L2-anchor coefficient.
ρ	$\mathbb{R}_{\geq 0}$	Retention ratio $\text{MMLU}_{\text{post}}/\text{MMLU}_{\text{pre}}$.
K_{mem}	10^6	Episodic memory capacity.
N_{train}	20,000	FAISS exact-to-IVF-PQ promotion point.
n_{list}	4096	IVF coarse centroid count (memory index).
K_D	5	MC dropout passes for u_{dropout} .
K_S	10	Semantic entropy sample count for h_{norm} .
M	3	Chain count for s_{avg} .

Stage 5: Unified nine-signal verification. Tier 2 and Tier 3 answers are passed through a single verifier that computes nine signals in three families. The internal family $\{u_{\text{token}}, u_{\text{dropout}}, u_{\text{internal}}\}$ measures the model’s self-assessed confidence. The sample-set family $\{s_{\text{avg}}, p_{\text{entail}}, h_{\text{norm}}\}$ measures cross-chain agreement. The grounding family $\{p_{\text{ground}}^{\text{max}}, p_{\text{ground}}^{\text{mean}}, p_{\text{ground}}^{\text{atomic}}\}$ measures how well the answer is entailed by external Wikipedia evidence. The atomic variant is computed by prompting the base model to decompose the answer into a short list of standalone factual claims, parsing the numbered list, running an NLI pass between each atom and each of the top-three reranked passages, and recording the weakest-link entailment across atoms; this catches partially-correct answers in which one atom is supported while another is silently fabricated, a failure mode the max and mean aggregates cannot distinguish. The verifier also records p_{contra} , the maximum contradiction probability across the same passages. A confabulation gate evaluated before the composite discards the answer immediately whenever internal confidence is high and grounding is effectively absent. Surviving answers are combined into the composite u_{stored} by a six-weight convex combination.

Stage 6: Four-outcome storage decision. Given u_{stored} , p_{contra} , and $p_{\text{ground}}^{\text{max}}$, the system emits one of four categorical decisions. A contradiction veto at $p_{\text{contra}} \geq \tau_{\text{veto}}$ forces DISCARD. Otherwise the composite is thresholded: $u_{\text{stored}} \geq \tau_{\text{store}}$ emits STORE; $u_{\text{stored}} \geq \tau_{\text{defer}}$ emits DEFERRED; $p_{\text{ground}}^{\text{max}} < \tau_{\text{abs}}$ emits ABSTAIN; all other cases emit DISCARD.

Stage 7: Memory write and output. A STORE decision writes the full episode to memory (question, chain, answer, embedding, nine-signal record, composite, contradiction probability, cycle index, decision) and flags it as training-eligible provided $u_{\text{stored}} \geq \tau_{\text{train}}$. A DEFERRED decision writes the episode but withholds the training flag, pending promotion or pruning at the next cycle boundary. ABSTAIN returns a principled refusal and writes nothing. DISCARD suppresses the generation entirely.

Stage 8: Between-cycle self-improvement. At the end of every cycle, the full verifier is re-run over every stored episode under the updated weights. Updates are applied asymmetrically: re-verification can only *raise* a stored composite, in which case the full nine-signal record is replaced; it never writes back a lower composite than the entry’s original score, so that a single bad cycle cannot silently demote a previously well-verified episode. Irrecoverable entries are instead handled by pruning at the retroactive threshold τ_{retro} . A training pool is then curated from training-eligible stores, mixed with general-domain examples in a 90:10 ratio, and the base model is fine-tuned under an L_2 -anchor regulariser with coefficient λ . A single per-cycle MMLU evaluation computes the retention ratio $\rho = \text{MMLU}_{\text{post}} / \text{MMLU}_{\text{pre}}$; if $\rho < \tau_{\text{forget}}$, the post-cycle weights are discarded and θ_{prev} is restored, otherwise they are committed. The same pass feeds the RET axis of the Composite Evaluation Score, so one broad-capability probe simultaneously serves as the rollback trigger and as the reported retention metric. The loop repeats for ten cycles, with Cycle 0 serving as the untrained baseline and as the one-shot calibration fit for the Stage 1 temperature scaler.

4.2.2 Worked example

Example 4.1 End-to-end trace through CAEM on two contrasting queries

■ Query A (Tier 1 cheap path)

Consider $q = \text{“Who discovered radium?”}$ arriving at Cycle 3, with the memory already populated by roughly 17,000 entries from earlier cycles.

Stage 1 | Pre-route confidence. One encoder pass produces $u_{\text{token}} = 0.82$ over the greedy prefix and $C_{\text{conv}} = 0.41$, giving

$$u_{\text{pre}} = 0.60 \cdot 0.82 + 0.40 \cdot \frac{1}{1.41} = 0.776$$

after temperature scaling fit at the end of Cycle 0. The safety override is not triggered.

Stage 2 | Memory retrieval. SBERT nearest neighbour returns e^* with $\text{sim}(q, e^*) = 0.93$ and $u_{\text{stored}}(e^*) = 0.86$.

Stage 3 | Three-tier router. The combined score is

$$S(q) = 0.70 \cdot 0.93 + 0.30 \cdot 0.86 = 0.909 \geq 0.90,$$

so the router dispatches to **Tier 1**.

Stage 4 | Tier 1 execution. The stored answer attached to e^* , “*Marie Curie and Pierre Curie*”, is returned directly without invoking the verifier or any generation. This is the cheap path the verified-memory investment is designed to enable.

Stages 5–7 | Verifier bypassed. Tier 1 entries carry a composite from their original storage event and are re-verified between cycles; the outcome is STORE-equivalent and the user is served immediately from verified memory.

■ Query B (Tier 2 full verify-and-decide path, at the end of Cycle 3)

A later query $q' = \text{“Who first synthesised aspirin?”}$ routes instead to Tier 2; the zero-shot generator produces “*Felix Hoffmann*”. The verifier’s nine-signal record is:

<i>Internal</i>		<i>Sample-set</i>		<i>Grounding</i>			
u_{internal}	0.78	s_{avg}	0.74	$p_{\text{ground}}^{\text{max}}$	0.88	<i>Veto</i>	
		p_{entail}	0.81	$p_{\text{ground}}^{\text{mean}}$	0.71	p_{contra}	0.04
		h_{norm}	0.22	$p_{\text{ground}}^{\text{atomic}}$	0.65		

Confabulation gate. Not triggered, since $p_{\text{ground}}^{\text{max}} = 0.88 \gg 0.20$.

Composite.

$$\begin{aligned} u_{\text{stored}} &= 0.30 (0.71) + 0.15 (0.65) + 0.15 (0.81) \\ &\quad + 0.15 (0.74) + 0.15 (0.78) + 0.10 (1 - 0.22) \\ &= \mathbf{0.7335}. \end{aligned}$$

Decision. Because $u_{\text{stored}} = 0.7335 \geq \tau_{\text{store}} = 0.65$ and $p_{\text{contra}} = 0.04 < \tau_{\text{veto}} = 0.30$, the decision is STORE. Because $u_{\text{stored}} < \tau_{\text{train}} = 0.75$, it is *not* flagged as training-eligible; it will be reconsidered at the next cycle’s retroactive re-verification pass.

The rest of Chapter 4 fills in each stage at the fidelity required to re-implement it from this text. Sections 4.3 through 4.10 follow the eight-stage order of Figure 4.1.

4.3 Pre-Routing Confidence Estimation

Stage 1 produces a calibrated scalar $u_{\text{pre}} \in [0, 1]$ that estimates how confidently the base model could answer the incoming query from its internal parameters alone. The stage runs before any generation, any retrieval, and any storage decision, so that its cost (a single encoder forward pass) is paid uniformly across all three downstream tiers, including Tier 1 queries that will ultimately be served without generation. Two design commitments shape the computation. First, the estimate must be obtainable without running the decoder, because the whole point of Stage 1 is to decide whether the decoder needs to run at all; this rules out any signal that requires sampling multiple completions. Second, the estimate must discriminate reliably at its low end, since u_{pre} is consumed both by the combined routing score of Stage 3 (Section 4.5) and by the safety override that forces Tier 3 retrieval when $u_{\text{pre}} < 0.60$. Confidence signals that are well-calibrated in the high region but noisy near zero are unsuitable for this role, because a safety override is a veto on cheap paths and its false positives cost nothing while its false negatives cost hallucinations.

Stage 1 achieves both properties by combining two complementary signals drawn from the same encoder forward pass. The first signal, u_{token} , is a token-level aggregate over the greedy-decoded answer prefix; it captures lexical confidence in the surface form the model would have produced. The second signal, C_{conv} , is an encoder-internal convergence ratio measured over the query representation alone; it captures representational stability before any token is emitted. The two signals disagree systematically in the failure mode Stage 1 most needs to catch: a query that is superficially familiar in phrasing but substantively out of distribution yields a high u_{token} over whatever surface form the greedy decoder commits to, but a representation that has not yet settled through the encoder stack yields an elevated C_{conv} . Fusing both signals ensures that u_{pre} is suppressed in exactly this regime.

Token-probability aggregate. The base model generates a greedy prefix of at most 32 tokens and records the per-step conditional log-probability of each emitted token. Generation terminates at the EOS token or at the cap, whichever comes first, and EOS itself is excluded from the aggregate. Writing $\ell_1, \dots, \ell_n$ for the non-

EOS log-probabilities of the emitted prefix, the aggregate is their geometric mean expressed as an exponentiated arithmetic mean of log-probabilities,

$$u_{\text{token}} = \exp\left(\frac{1}{n} \sum_{i=1}^n \ell_i\right) = \left(\prod_{i=1}^n p(y_i \mid y_{<i}, q)\right)^{1/n}, \quad (4.1)$$

so that $u_{\text{token}} \in (0, 1]$ with the natural interpretation of mean per-token probability. The geometric-mean form is chosen over the arithmetic mean of probabilities because log-probabilities decompose additively across steps, making the aggregate insensitive to prefix length: a 10-token and a 30-token answer with the same mean per-token probability receive the same score. A prefix cap of 32 tokens is imposed so that Stage 1 cost remains bounded even on queries whose greedy completion runs long; empirically the cap is reached only on enumerations, and the confidence signal derived from the first 32 tokens is a faithful proxy for the full answer.

Encoder convergence ratio. The encoder’s L intermediate hidden states $h_1, \dots, h_L$ are captured during the same forward pass; the input embedding layer h_0 is excluded so that the ratio measures contextualisation rather than the tokenizer’s inductive bias. The stack is split at its midpoint into an early half $\{h_1, \dots, h_{L/2}\}$ and a late half $\{h_{L/2+1}, \dots, h_L\}$, and the convergence ratio is defined as the ratio of scalar activation variances,

$$C_{\text{conv}} = \frac{\text{Var}(h_1, \dots, h_{L/2})}{\text{Var}(h_{L/2+1}, \dots, h_L) + \varepsilon}, \quad \varepsilon = 10^{-8}, \quad (4.2)$$

where variance is computed over the flattened concatenation of each half’s hidden-state tensors (no pooling across the sequence dimension, so long queries contribute proportionally more samples). A well-converged representation is one in which the late layers vary much less than the early layers as the encoder progressively stabilises the meaning of the query, producing $C_{\text{conv}} > 1$. A poorly-converged representation, the situation in which the encoder is still reshaping the query by its final layers, produces C_{conv} near unity or below. The exact midpoint split is a deliberate choice over alternatives such as first-third/last-third: on Flan-T5-Large we observed that the most diagnostic contrast is between the query-understanding layers (the first half, which performs lexical and syntactic integration) and the query-resolution layers (the second half, which contextualises against factual associations).

Fusion and temperature calibration. The two signals are fused into a raw pre-route confidence by the linear combination specified in Equation 3.1 of Chapter 3, reproduced here with the transformation that maps C_{conv} into the unit interval made explicit,

$$\tilde{u}_{\text{pre}} = w_{\text{tok}} \cdot u_{\text{token}} + w_{\text{conv}} \cdot \frac{1}{1 + C_{\text{conv}}}, \quad w_{\text{tok}} = 0.60, \quad w_{\text{conv}} = 0.40, \quad (4.3)$$

with $\tilde{u}_{\text{pre}}$ then clipped to $[0, 1]$ to absorb the floating-point tail of either component. The clipping is mechanically necessary because the finite precision of the softmax and the hidden-state variance can each produce values marginally outside the unit interval, but in practice it fires only on degenerate queries where it is already immaterial which tier executes.

The raw estimate $\tilde{u}_{\text{pre}}$ is a biased estimator of empirical correctness: language models are systematically over-confident, and a weighted sum of two such signals is over-confident in the same direction. To remove the bias, $\tilde{u}_{\text{pre}}$ is passed through a single-parameter temperature scaler fit once, offline, at the end of Cycle 0 on a held-out calibration split of 500 queries never used for training or evaluation. The scaler operates in logit space:

$$u_{\text{pre}} = \sigma\left(\frac{\text{logit}(\tilde{u}_{\text{pre}})}{T}\right), \quad \sigma(x) = \frac{1}{1 + e^{-x}}, \quad (4.4)$$

where T is the scalar temperature. This is the standard post-hoc recalibration of Guo et al. **pmlr-v70-guo17a**, written in sigmoid form for a scalar probability rather than in softmax form for a multi-class logit vector; the two are equivalent and the choice is notational. T is fit by minimising the binary negative log-likelihood of the calibration split under L-BFGS, using empirical exact-match correctness as the binary target:

$$T^* = \arg \min_{T>0} -\frac{1}{N} \sum_{i=1}^N \left[y_i \log \sigma(\text{logit}(\tilde{u}_{\text{pre}}^{(i)})/T) + (1 - y_i) \log(1 - \sigma(\text{logit}(\tilde{u}_{\text{pre}}^{(i)})/T)) \right]. \quad (4.5)$$

NLL is used as the training objective in place of the expected calibration error (ECE) reported in Chapter 3’s acceptance criterion, because ECE is a binned statistic and not differentiable; minimising NLL produces a temperature that minimises ECE in practice while remaining tractable for L-BFGS. The fitted T^* is written to the pipeline configuration and held fixed for all subsequent cycles; Chapter 3’s NFR on calibration stability ($\text{ECE} \leq 0.05$ after Cycle 1) is an acceptance check on this fit rather than an inline regularisation.

Algorithm 4.1 summarises the stage. The algorithm assumes that the temperature T has been fit; in the very first cycle the temperature defaults to unity and the sigmoid-of-logit identity leaves the raw estimate unchanged, so the calibration is a strict improvement applied from Cycle 1 onward rather than a correctness dependency.

Algorithm 4.1 Stage 1 pre-route confidence u_{pre} .

Require: Query q ; base model f_θ ; weights $w_{\text{tok}} = 0.60$, $w_{\text{conv}} = 0.40$; temperature T ; prefix cap $M = 32$.

Ensure: Calibrated scalar $u_{\text{pre}} \in [0, 1]$.

- 1: $(y_{1:n}, \ell_{1:n}, h_{0:L}) \leftarrow$ greedy decode of $f_\theta(q)$ up to M tokens, recording emitted token log-probabilities ℓ_i and encoder hidden states $h_{0:L}$
 - 2: exclude EOS from $\ell_{1:n}$; if empty, **return** 0 $\triangleright$ safety-override eligible
 - 3: $u_{\text{token}} \leftarrow \exp(\frac{1}{n} \sum_i \ell_i)$ $\triangleright$ geometric mean of token probabilities
 - 4: $\text{Var}_{\text{early}} \leftarrow \text{Var}(h_{1:L/2})$; $\text{Var}_{\text{late}} \leftarrow \text{Var}(h_{L/2+1:L})$ $\triangleright$ flattened across seq_len $\times$ hidden_dim
 - 5: $C_{\text{conv}} \leftarrow \text{Var}_{\text{early}} / (\text{Var}_{\text{late}} + 10^{-8})$
 - 6: $\tilde{u}_{\text{pre}} \leftarrow \text{clip}(w_{\text{tok}} u_{\text{token}} + w_{\text{conv}} (1 + C_{\text{conv}})^{-1}, 0, 1)$
 - 7: $u_{\text{pre}} \leftarrow \sigma(\text{logit}(\tilde{u}_{\text{pre}})/T)$ $\triangleright$ identity when $T = 1$
 - 8: **return** u_{pre}
-

The scalar u_{pre} produced by Algorithm 4.1 is the sole quantity Stage 3 uses to evaluate the safety override, and one of two quantities (the other being the stored

composite of the retrieved episode) that enter the combined routing score. Its production is the only computation in the entire pipeline that is paid on every query regardless of tier, which sets the cost floor of CAEM at one encoder forward pass per query and motivates the single-pass, decoder-free design of this section.

4.4 Episodic Memory

The episodic memory is the persistent store that CAEM consults at Stage 2 of every query, writes to at Stage 7 after a STORE outcome (or, at a cycle boundary, when a deferred entry is promoted out of the buffer of Section 4.9), and re-verifies wholesale at Stage 8 between cycles. It is the central artefact that accumulates across cycles: a well-designed memory makes the cheap Tier 1 path increasingly attractive, while a poorly curated memory either forces unnecessary retrieval (by failing to serve near-duplicates of previous queries) or pollutes the Tier 1 path with low-quality answers. This section specifies what is stored, how it is indexed, how novelty and capacity are managed, and how retrieval feedback updates stored confidence between cycles.

Data schema. Each memory entry is an EPISODICENTRY instance holding five immutable content fields and a mutable quality-and-usage record, summarised in Table 4.2. The immutable fields (question, reasoning chain, answer, 768-dimensional embedding, storage cycle, and creation timestamp) are set at storage time and never modified afterwards. This invariance is load-bearing: Stage 8 derives the fine-tuning pool from these fields, and mutating them mid-cycle would mean training against a moving target with no reliable audit trail of which (query, chain, answer) triples actually updated the model. The mutable record carries the full nine-signal verifier output, the contradiction probability p_{contra} , the categorical Stage 6 decision, the confabulation-gate flag, the composite u_{stored} , and three usage fields (retrieval count, running success rate, retroverified flag). The composite u_{stored} is defined by the six-weight convex combination of Chapter 3, Equation 3.5, repeated here for reference,

$$u_{\text{stored}} = 0.30 p_{\text{ground}}^{\text{mean}} + 0.15 p_{\text{ground}}^{\text{atomic}} + 0.15 p_{\text{entail}} + 0.15 s_{\text{avg}} + 0.15 u_{\text{internal}} + 0.10 (1 - h_{\text{norm}}), \quad (4.6)$$

with $(1 - h_{\text{norm}})$ appearing because semantic entropy is a disagreement signal and the composite is a confidence. Storage happens only if $u_{\text{stored}} \geq \tau_{\text{defer}}$ at Stage 6, so every entry in memory carries a positive prior that survives a two-threshold screen; this prior is the quantity Stages 3 and 5 subsequently consume.

Embedding model and similarity metric. Embeddings are produced by the Sentence-BERT `all-mpnet-base-v2` model, which maps arbitrary queries into a 768-dimensional vector space. Every embedding is L2-normalised to unit length at the moment of production, so that the inner product between two embeddings equals their cosine similarity,

$$\text{sim}(q, e) = \hat{q}^\top \hat{e}, \quad \hat{v} = v / \|v\|_2, \quad (4.7)$$

which is the property FAISS’s METRIC INNER PRODUCT relies on for cosine-ordered nearest-neighbour results. Normalisation is applied once at storage time and once at query time; it is never re-applied after retrieval, so that similarities retrieved

Table 4.2: Fields of an EPISODICENTRY. Immutable content is the substrate from which the Stage 8 fine-tuning pool is curated; the mutable record tracks quality and usage across cycles.

Field group	Contents
<i>Immutable content (never updated after storage)</i>	
Natural-language triple	question, reasoning chain, answer
Embedding	768-d L2-normalised SBERT vector (<code>float32</code>)
Provenance	storage cycle $\in \{0, \dots, N_{\text{cycles}}\}$, Unix timestamp
<i>Mutable quality record (updated by retroactive re-verification)</i>	
Nine signals	$u_{\text{token}}, u_{\text{dropout}}, u_{\text{internal}}, s_{\text{avg}}, h_{\text{norm}}, p_{\text{entail}}, p_{\text{ground}}^{\text{max}}, p_{\text{ground}}^{\text{mean}}, p_{\text{ground}}^{\text{atomic}}$
Veto signal	p_{contra} (triggers DISCARD on retroverify)
Composite	u_{stored} (Equation 4.6)
Decision record	one of STORE, DEFERRED, ABSTAIN, DISCARD; early-exit flag
<i>Mutable usage record (updated on each retrieval)</i>	
Retrieval count	integer r
Success rate	running fraction of retrievals accepted downstream
Retroverified flag	set at each cycle boundary; consumed by the Value score

from FAISS can be consumed directly by Stage 3’s routing score without further rescaling.

Index structure and size-adaptive promotion. The memory is backed by a FAISS index wrapped in an `IndexIDMap` so that entries can be removed in place during pruning. For the first phase of operation the backend is a flat inner-product index, which performs exact cosine nearest-neighbour search at $O(N)$ per query. When the number of stored vectors first reaches a promotion threshold $N_{\text{train}} = 20,000$, the index is rebuilt in place as a product-quantised inverted file: an `IndexIVFPQ` with $n_{\text{list}} = 4096$ coarse centroids, product quantisation into $m = 64$ sub-vectors of 8 bits each, and search-time probing of $n_{\text{probe}} = 32$ centroids. This promotion is the operationalisation of Chapter 3’s memory-cost analysis: flat IP is cheap at $N \leq 20,000$ but scales poorly; IVF-PQ trades a small approximation error for sub-linear retrieval and a $\sim 32\times$ reduction in embedding storage cost, which is what makes the $K_{\text{mem}} = 10^6$ target tractable on a single 24 GB device. Before promotion the quantiser is trained on a uniformly sampled subset of up to 200,000 existing vectors, which is sufficient because n_{list} is the binding constraint (FAISS requires at least as many training points as centroids, satisfied at the 20,000 threshold). After promotion the index returns approximate rather than exact nearest neighbours, but in practice the approximation does not change the routing decision: Stage 3’s tier boundaries at $S(q) \geq 0.90$ and $\text{sim}(q, e^*) \geq 0.75$ are far from the noise floor of IVF-PQ recall.

Per-query retrieval collapses to a single call: the query embedding $\hat{q}$ is submitted to the index, the top-1 neighbour e^* is returned with its cosine similarity and the external integer identifier under which its mutable record is kept in the metadata map. Stage 3 reads $u_{\text{stored}}(e^*)$ from the metadata map rather than from the index itself; storing composites inside the index is unnecessary because the identifier suf-

fices and keeping quality metadata out of the FAISS payload avoids coupling the routing algorithm to the index’s internal layout.

Novelty filter. Before an episode is committed at Stage 7 it is screened for novelty: the memory is searched for its single nearest neighbour e^* , and storage is rejected when

$$\text{sim}(q, e^*) > \tau_{\text{nov}}, \quad \tau_{\text{nov}} = 0.95, \quad (4.8)$$

so that near-duplicates do not accumulate as separate entries. The filter is deliberately one-sided: a query sufficiently similar to an existing entry is assumed to be covered by that entry, and the existing entry’s mutable quality record is updated through the retrieval feedback loop rather than through a second store. The threshold is set high (0.95) because SBERT similarities above this level are semantically indistinguishable in our pilot calibration set, while thresholds in the 0.85 to 0.90 range produced spurious rejections on paraphrases that legitimately tested a different aspect of the same topic. When a stored entry would otherwise be the k -th near-duplicate added, the training-pool curator of Stage 8 would be training many times on effectively the same (query, answer) pair, biasing the fine-tuning gradient; the novelty filter is the mechanism that prevents this.

Retrieval feedback update. Each accepted retrieval at Stage 4 triggers two updates on the retrieved entry’s mutable record. First, the retrieval count is incremented and the running success rate is updated by the standard incremental-mean formula,

$$r_{\text{new}} = r + 1, \quad \bar{a}_{\text{new}} = \frac{r \cdot \bar{a} + \mathbb{K}[\text{accepted}]}{r + 1}. \quad (4.9)$$

Second, once an entry has accumulated at least $r_{\text{min}} = 5$ retrievals, its stored composite is nudged by an exponential-moving-average rule with learning rate η ,

$$u_{\text{stored}} \leftarrow \begin{cases} u_{\text{stored}} + \eta(1 - u_{\text{stored}}) & \text{if the retrieval was accepted,} \\ u_{\text{stored}} - \eta u_{\text{stored}} & \text{otherwise,} \end{cases} \quad (4.10)$$

which draws the composite toward 1 on success and toward 0 on override. The floor of five retrievals before any update fires is important: the feedback rule is fundamentally noisy at low retrieval counts, and the cycle-boundary retroactive re-verification pass of Stage 8 is the authoritative quality update. The feedback rule therefore serves only as a fast-path adjustment between cycles, handling the case where an entry is retrieved heavily (and correctly or incorrectly) before the next cycle boundary arrives.

Capacity management by Value score. The store enforces a hard capacity bound $K_{\text{mem}} = 10^6$. When utilisation reaches a pruning trigger at 95% of capacity, the bottom 20% of entries by a Value score are removed and the FAISS index is rebuilt to reflect the deletions. The Value score is a convex combination of an Importance and a Recency component,

$$V(e) = w_{V,I} I(e) + w_{V,R} R(e), \quad w_{V,I} = 0.70, \quad w_{V,R} = 0.30, \quad (4.11)$$

with

$$I(e) = w_{I,\phi} \phi(e) + w_{I,r} \frac{r(e)}{r_{\max}} + w_{I,s} \bar{a}(e) + w_{I,u} u_{\text{stored}}(e), \quad (4.12)$$

$$R(e) = \exp(-\lambda_R \text{age}(e)), \quad \lambda_R = 10^{-2}, \quad (4.13)$$

$$\phi(e) = (\text{cycle}(e) + 1) / (N_{\text{cycles}} + 1), \quad (4.14)$$

and the Importance weights $(w_{I,\phi}, w_{I,r}, w_{I,s}, w_{I,u}) = (0.40, 0.30, 0.20, 0.10)$. The cycle proxy $\phi(e)$ rewards entries stored by later, better-trained cycles; the normalisation $(N_{\text{cycles}} + 1)$ in the denominator guarantees $\phi \leq 1$ at the final cycle exactly rather than exceeding unity. The retrieval count enters only after normalisation by the global maximum $r_{\max}$ so that a single frequently-hit entry cannot dominate the ranking. The recency term uses real wall-clock age with $\lambda_R = 10^{-2}$ per second, a slow decay chosen so that within a single cycle’s duration the recency factor changes by at most a few percent; its role is to tie-break between entries of similar importance rather than to dominate them. Pruning is upper-bounded in frequency by the trigger condition, and because every retained entry survives a convex-combination selection over four quality signals, the surviving population is on average better-calibrated than the pre-pruning population.

Persistence. The full store (FAISS index plus metadata map plus the monotonic next-identifier counter) is serialised to a pair of files at cycle boundaries. The index is written via FAISS’s native binary format; the metadata map is written as a pickle. On reload, the index’s IVF probing parameter is reset from the current configuration rather than from the saved index, so that n_{probe} can be tuned without rebuilding the memory. This is the mechanism by which a long-running experiment can be resumed at an arbitrary cycle boundary, and by which a retrieved store from one experimental configuration can be reused to seed another.

In summary, the memory module combines a normalised SBERT embedding space, a size-adaptive FAISS backend, a one-sided novelty filter, a per-retrieval feedback rule, and a Value-based pruning policy into a single persistent artefact whose query-time interface reduces to a single nearest-neighbour call plus a metadata lookup. All subsequent stages of CAEM are clients of this interface: Stage 3 consumes $(e^*, \text{sim}(q, e^*), u_{\text{stored}}(e^*))$ to route, Stage 7 calls the novelty filter and the add operation to write, and Stage 8 iterates over every stored entry to re-verify.

4.5 Three-Tier Router

Stage 3 converts the two upstream quantities (the pre-route confidence u_{pre} from Stage 1 and the single nearest-neighbour retrieval $(e^*, \text{sim}(q, e^*), u_{\text{stored}}(e^*))$ from Stage 2) into a categorical choice of processing tier. The tier choice determines the remainder of the per-query path: Tier 1 returns the stored answer attached to e^* without any generation, Tier 2 generates zero shot from the base model, and Tier 3 generates under retrieval-augmented grounding against a preprocessed Wikipedia passage index. The router is the cost-control mechanism of the pipeline: every decision it makes to dispatch to Tier 1 rather than Tier 2 or 3 saves a decoder pass, and every decision it makes to dispatch to Tier 3 rather than Tier 2 incurs passage retrieval and reranking but buys grounding that the verifier can subsequently exploit.

The design of the router therefore has to trade cost against safety while keeping the trade explicit.

Two-mechanism design. The router applies two decision rules that are deliberately kept separate. The first rule is a safety override evaluated before any score is computed: if u_{pre} falls below a minimum floor $\tau_{\text{safe}} = 0.60$, the query is dispatched to Tier 3 regardless of how well it matched the memory. The second rule is a combined routing score over similarity and stored composite, thresholded to select between Tier 1 and Tier 2. The two rules are not merged into a single formula. The reason is arithmetic: if the safety condition entered additively alongside similarity and u_{stored} , a high stored composite from a very similar episode could arithmetically compensate for a dangerously low u_{pre} , sending the query to Tier 1 exactly when the base model’s internal state warns that it should not be trusted. Treating model readiness (u_{pre}) and memory quality (u_{stored}) as two independent necessary conditions rather than as tradeable summands preserves the semantics that a low u_{pre} is a veto on the cheap paths, not a discount to be offset.

Safety override. The first mechanism, evaluated before any score calculation, is the OR-condition:

$$u_{\text{pre}} < \tau_{\text{safe}} \quad \Rightarrow \quad \text{Tier 3}, \quad (4.15)$$

with $\tau_{\text{safe}} = 0.60$. Three observations motivate the floor value. First, u_{pre} is temperature-calibrated from Cycle 1 onward (Section 4.3), so the threshold is referenced to a distribution of confidences that has been brought into alignment with empirical exact-match correctness; a floor of 0.60 corresponds in that calibrated regime to roughly a 40% confident-error risk on the cheap paths, which is unacceptable in an open-domain factual setting. Second, Tier 3 is the only path that introduces external evidence into the verifier, so forcing grounding is the operationally meaningful response to low model readiness: the verifier’s grounding family $\{p_{\text{ground}}^{\text{max}}, p_{\text{ground}}^{\text{mean}}, p_{\text{ground}}^{\text{atomic}}\}$ cannot fire on a Tier 1 or Tier 2 answer because no passages are retrieved. Third, the safety override sets a cost floor on queries the system distrusts: a Tier 3 pass takes a few seconds while a Tier 1 hallucination would propagate into memory (through Stages 5 and 7) and potentially into the next cycle’s fine-tuning pool. The cost asymmetry makes Tier 3 the unambiguously correct default when the model itself indicates uncertainty.

Combined routing score. When the safety override does not fire, the router evaluates a combined score over the similarity and the stored composite of the retrieved episode,

$$S(q) = \alpha \text{sim}(q, e^*) + (1 - \alpha) u_{\text{stored}}(e^*), \quad \alpha = 0.70, \quad (4.16)$$

so that similarity carries more weight than the stored composite by a ratio of 7:3. The weighting is deliberate. Similarity is the direct relevance signal: it measures whether the stored episode is about the same question as the query, and no Tier 1 outcome is defensible without high similarity. The stored composite is an orthogonal quality signal that reflects how well the stored answer was verified at storage time and subsequently, but a very high u_{stored} on a loosely related episode is not an argument for returning that episode’s answer; it is an argument for not using

Table 4.3: Tier dispatch conditions at Stage 3. Conditions are evaluated in order. The safety override supersedes the combined-score rules; when it does not fire, the combined score selects Tier 1 or the similarity selects Tier 2, and Tier 3 is the default.

Tier	Condition (evaluated in order)	Action
3 (forced)	$u_{\text{pre}} < \tau_{\text{safe}} = 0.60$ (safety override)	Forced RAG; skips score.
1	$S(q) \geq \tau_{\text{tier1}} = 0.90$ and safety override not taken	Return e^* 's stored answer.
2	$\text{sim}(q, e^*) > \tau_{\text{tier2}} = 0.75$, Tier 1 not taken	Zero-shot generation.
3 (default)	otherwise (including empty memory)	DPR retrieve + rerank + RAG.

Tier 1 at all. Up-weighting similarity therefore encodes the operational priority (relevance first, quality as a secondary gate), while retaining the stored composite in the score means that two episodes at equal similarity are separated by their verification history, with better-verified ones clearing the Tier 1 threshold sooner.

Tier dispatch rules. Given the combined score $S(q)$ and the raw similarity $\text{sim}(q, e^*)$, the router selects a tier according to Table 4.3. Tier 1 requires the combined score to clear $\tau_{\text{tier1}} = 0.90$, which is tight enough that Tier 1 is selected only when both signals are strong: at the limit, a similarity of 1.00 with an u_{stored} of 0.77 clears the threshold, and so does a similarity of 0.95 paired with u_{stored} of 0.78. Tier 2 is selected when Tier 1 is not taken and the raw similarity alone exceeds $\tau_{\text{tier2}} = 0.75$; this captures the case where an episode is clearly topically relevant but the composite alone is not yet strong enough for cheap-path reuse, and zero-shot generation is expected to produce a defensible answer that the verifier can score. Tier 3 is selected in every remaining case: empty memory, low similarity, or borderline combined scores that do not clear the Tier 1 bar without also having the topical similarity to justify Tier 2.

Behaviour on an empty memory. The router requires no special case for the cold-start regime in which the episodic store is empty. On an empty memory Stage 2 returns an empty result, which the router interprets by substituting $\text{sim}(q, e^*) = 0$ and $u_{\text{stored}}(e^*) = 0$ into Equation 4.16, giving $S(q) = 0$. The combined score then falls below both tier thresholds, the similarity fails the Tier 2 bar, and the query dispatches to Tier 3 by default. The safety override rule is unaffected: it depends only on u_{pre} , which is produced by Stage 1 regardless of memory state. As a consequence, the entire first cycle of CAEM (Cycle 0, during which the memory is initialised and the calibration split is collected) runs exclusively on Tiers 2 and 3, which is the intended behaviour: the cheap Tier 1 path comes online organically as u_{stored} values accumulate at Stage 7.

Algorithm 4.2 states the Stage 3 logic in its entirety. The strict evaluation order (override first, then score) is load-bearing and is the algorithmic encoding of the two-mechanism separation argued above.

The routing decision is logged on a per-query basis with all five upstream quanti-

Algorithm 4.2 Stage 3 three-tier routing decision.

Require: Pre-route confidence u_{pre} ; retrieved neighbour $(e^*, \text{sim}(q, e^*), u_{\text{stored}}(e^*))$ or empty; weight $\alpha = 0.70$; thresholds $\tau_{\text{safe}} = 0.60$, $\tau_{\text{tier1}} = 0.90$, $\tau_{\text{tier2}} = 0.75$.

Ensure: Tier $t \in \{1, 2, 3\}$ with a safety-override flag.

```
1: if memory returned no result then
2:    $\text{sim}(q, e^*) \leftarrow 0$ ;  $u_{\text{stored}}(e^*) \leftarrow 0$  ▷ empty-memory default
3: end if
4: if  $u_{\text{pre}} < \tau_{\text{safe}}$  then
5:   return ( $t = 3$ , safety_override = true) ▷ mechanism 1: OR-condition
6: end if
7:  $S(q) \leftarrow \alpha \text{sim}(q, e^*) + (1 - \alpha) u_{\text{stored}}(e^*)$  ▷ mechanism 2: combined score
8: if  $S(q) \geq \tau_{\text{tier1}}$  then
9:   return ( $t = 1$ , safety_override = false)
10: else if  $\text{sim}(q, e^*) > \tau_{\text{tier2}}$  then
11:   return ( $t = 2$ , safety_override = false)
12: else
13:   return ( $t = 3$ , safety_override = false)
14: end if
```

ties (u_{pre} , $\text{sim}(q, e^*)$, $u_{\text{stored}}(e^*)$, $S(q)$, and the safety-override flag), because these form the substrate of the routing-distribution analyses reported in Chapter 5. In particular, the fraction of queries that triggered the safety override separately from the fraction that were routed to Tier 3 by the default rule are tracked as distinct metrics: the two populations differ in whether the model *recognised* that it was uncertain (override) versus the memory simply not containing a relevant neighbour (default), and both are reported in the routing-audit table of Chapter 5.

4.6 Unified Verifier

Every Tier 2 and Tier 3 answer passes through a single Stage 5 quality gate before the storage decision is made. The verifier consumes a (query, answer) pair and emits nine scalar signals together with a composite score u_{stored} and an explicit flag for a confabulation early-exit gate. The signal set is designed around a single organising claim: any single confidence signal is exploitable in one direction or another, so the composite is built from three complementary families that fail in different ways. Internal calibration signals are cheap and fine-grained but do not know whether the answer is true; sample-set signals detect disagreement across the model’s own re-samplings but cannot distinguish a confidently-wrong consensus from a correct one; external grounding signals test factual support against retrieved evidence but are vulnerable to partially-correct answers that are superficially supported. The composite combines all three so that no single failure mode clears the storage threshold on its own. This section specifies the nine signals, their aggregation rules, and the early-exit gate. The four-outcome decision tree that consumes the composite is the subject of Section 4.7.

Signal families and cost. Table 4.4 summarises all nine signals together with their source family, the number of model calls each contributes, and whether the

Table 4.4: The nine verifier signals, grouped by family. Source column: [L] = adapted from literature, [D] = CAEM design. Cost column reports the dominant factor per query at Stage 5.

Signal	Family / what it measures	Source	Dominant cost
u_{token}	Internal: lexical confidence in the answer	[L]	1 teacher-forced forward pass
u_{dropout}	Internal: MC-dropout disagreement across stochastic passes	[L]	$K_D = 5$ decoding passes
u_{internal}	Internal: fused scalar $0.5 u_{\text{token}} + 0.5 (1 - u_{\text{dropout}})$	[D]	(reuses above)
s_{avg}	Sample-set: mean pairwise SBERT cosine across M chains	[L]	$M = 3$ decoding passes (shared)
p_{entail}	Sample-set: NLI entailment chain-to-answer, averaged over M chains	[D]	M NLI calls (reuses chains)
h_{norm}	Sample-set: normalised semantic entropy over K samples, NLI-clustered	[L]	$K = 10$ decoding passes
$p_{\text{ground}}^{\text{max}}$	Grounding: max NLI entailment across top-3 reranked passages	[D]	3 NLI calls
$p_{\text{ground}}^{\text{mean}}$	Grounding: mean NLI entailment across top-3 reranked passages	[D]	(reuses above)
$p_{\text{ground}}^{\text{atomic}}$	Grounding: weakest-link entailment over decomposed atomic facts	[D]	n_{atoms} NLI calls plus 1 decode
p_{contra}	Veto signal: max contradiction probability over top-3 passages	[D]	3 NLI calls (reuses ensemble)

signal is literature-derived (L) or a CAEM-specific design decision (D). The per-query cost of Stage 5 is dominated by the decoding passes required for the sample-set family: $M = 3$ chains at temperature $T_s = 0.7$ for s_{avg} and p_{entail} (shared), $K = 10$ independent samples at temperature $T_{\text{se}} = 1.0$ for h_{norm} , and $K_D = 5$ stochastic passes for u_{dropout} . The external-grounding family adds 20 passages retrieved from the preprocessed Wikipedia index and reranked to the top 3 by a cross-encoder, plus one NLI ensemble call per passage. The atomic-fact path additionally performs one greedy decomposition pass and one NLI call per atom per passage. All nine signals are computed once per Stage 5 invocation; there is no adaptive early stopping inside the signal computation itself (the only early exit is the composite-level confabulation gate described below).

Internal calibration. The internal family measures the model’s self-assessed confidence in the specific answer under evaluation, using no external information. The token aggregate u_{token} here is defined analogously to Stage 1’s (Equation 4.1) but is computed over the full emitted answer rather than over the greedy prefix and is

evaluated under teacher forcing with the answer tokens as labels, so it measures the model’s probability mass on the exact string being verified:

$$u_{\text{token}}^{\text{verify}} = \exp\left(\frac{1}{n} \sum_{i=1}^n \log p_{\theta}(y_i \mid y_{<i}, q)\right), \quad (4.17)$$

with non-padding positions only contributing to the mean. The dropout signal follows the predictive-uncertainty framing of Gal and Ghahramani **pmlr-v48-gal16**: dropout is left active during inference and the generator is sampled $K_D = 5$ times at $T = 0.7$; a plurality answer is identified over the K_D samples, and

$$u_{\text{dropout}} = 1 - \frac{\max_c \#\{i : a_i \equiv a_c\}}{K_D} \in [0, 1 - 1/K_D], \quad (4.18)$$

where $a_i \equiv a_c$ denotes surface-normalised string equality. Higher values indicate that the model disagrees with itself under perturbation, and the composite uses $(1 - u_{\text{dropout}})$ through the fused internal scalar. The fused internal scalar averages the two,

$$u_{\text{internal}} = \frac{1}{2} u_{\text{token}}^{\text{verify}} + \frac{1}{2} (1 - u_{\text{dropout}}), \quad (4.19)$$

with equal weighting because the two signals capture qualitatively different failure modes: $u_{\text{token}}^{\text{verify}}$ is degraded by any token the model considers unlikely even when the overall answer is correct, and u_{dropout} is degraded only when the model actively disagrees with itself; pairing them tightens the internal signal against either individual noise source. The fused scalar is the signal that enters the composite directly, and is also the signal checked by the confabulation gate.

Sample-set signals. The sample-set family examines what the model produces when asked the same query multiple times with non-deterministic sampling. Three signals share this family. For the first two, $M = 3$ chain-of-thought responses are generated at $T = 0.7$ and reused by both signals. The self-consistency signal s_{avg} is the mean pairwise cosine similarity between chain embeddings produced by the same SBERT model used by the memory (Equation 4.7),

$$s_{\text{avg}} = \binom{M}{2}^{-1} \sum_{1 \leq i < j \leq M} \hat{c}_i^{\top} \hat{c}_j, \quad (4.20)$$

with the unique-pair $i < j$ convention so the diagonal and double counting are not included; this is the formulation of Wang et al. **wang2023selfconsistency**. The chain-to-answer entailment signal is the mean over the same M chains of the NLI ensemble’s entailment probability from chain to answer,

$$p_{\text{entail}} = \frac{1}{M} \sum_{i=1}^M \text{NLI}_{\min}(c_i \Rightarrow a), \quad (4.21)$$

where $\text{NLI}_{\min}$ denotes the minimum entailment probability across all NLI bundles in the ensemble (the conservative aggregation rule described below). The two signals combine differently: s_{avg} rewards surface and semantic consistency across the chain space, while p_{entail} rewards chains that individually support the final answer under

entailment, so they decorrelate on cases where the model produces several internally-consistent but collectively-wrong chains.

The third sample-set signal is the normalised semantic entropy of Farquhar et al. **farquhar2024semantic**, computed over an independent set of $K = 10$ samples at $T = 1.0$. Samples are clustered by bidirectional NLI entailment: two samples s_i and s_j join the same cluster iff

$$\text{NLI}(s_i \Rightarrow s_j) = \text{ENT} \wedge \text{NLI}(s_j \Rightarrow s_i) = \text{ENT}, \quad (4.22)$$

where ENT is the argmax-label under the NLI ensemble’s majority-vote rule. The cluster assignment induces a distribution over cluster sizes, from which the Shannon entropy is computed and normalised by $\log_2 K$,

$$h_{\text{norm}} = \frac{-\sum_c p_c \log_2 p_c}{\log_2 K}, \quad p_c = \frac{|\{i : s_i \in c\}|}{K}, \quad (4.23)$$

so that $h_{\text{norm}} \in [0, 1]$. A value near zero indicates that almost all samples express semantically equivalent answers, while a value near one indicates that the samples partition into approximately K distinct semantic classes. The composite treats $(1 - h_{\text{norm}})$ as a confidence: semantic agreement across resamples is a positive signal. When no NLI ensemble is configured, the fallback is surface-level entropy over normalised strings, which is strictly weaker because it cannot merge paraphrases; this fallback matters only in smoke-test configurations.

External grounding. The grounding family anchors the verifier to external evidence rather than to the model’s own output distribution. On every Tier 2 and Tier 3 verification, the preprocessed Wikipedia passage retriever is called with the query and asked for the top $K_{\text{retrieve}} = 20$ candidates; these are then reranked by a cross-encoder scored against the concatenated (query, answer) pair, and the top $K_{\text{rerank}} = 3$ are retained. Two NLI-based signals are computed on this top-3 set:

$$p_{\text{ground}}^{\text{max}} = \max_{p \in P_{\text{top3}}} \text{NLI}_{\text{min}}(p \Rightarrow a), \quad (4.24)$$

$$p_{\text{ground}}^{\text{mean}} = \frac{1}{|P_{\text{top3}}|} \sum_{p \in P_{\text{top3}}} \text{NLI}_{\text{min}}(p \Rightarrow a), \quad (4.25)$$

together with the contradiction veto

$$p_{\text{contra}} = \max_{p \in P_{\text{top3}}} \text{NLI}_{\text{max}}^{\text{contra}}(p \Rightarrow a), \quad (4.26)$$

where $\text{NLI}_{\text{max}}^{\text{contra}}$ denotes the maximum contradiction probability across NLI bundles (the safety aggregation described below). The max and mean aggregates have complementary failure modes. $p_{\text{ground}}^{\text{max}}$ is robust to a partially-incorrect passage set because a single strongly-entailing passage suffices, but it is vulnerable to a superficially-matching passage that happens to mention the answer tokens without actually supporting the claim. $p_{\text{ground}}^{\text{mean}}$ is the reverse: less vulnerable to a single false-match passage, more sensitive to the quality of retrieval as a whole. Both enter the composite so that the resulting estimate is positive only when evidence is both present (max is high) and broadly supportive (mean is high).

Atomic-fact decomposition. The atomic signal is the one grounding mechanism that is not available through a single answer-level NLI call. It addresses a specific failure mode the max and mean aggregates cannot distinguish: an answer composed of several factual claims of which one is supported by the passages and another is silently fabricated, where the passage-level entailment is high enough on the supported claim to dominate the aggregates. The mechanism proceeds in three steps. First, the base model is prompted with a fixed instruction to decompose the answer into a numbered list of independent factual claims with pronouns resolved,

$$(a_1, \dots, a_{n_{\text{atoms}}}) = \text{Decompose}(a), \quad (4.27)$$

and the numbered output is parsed into a deduplicated list of claims. Second, each claim is individually tested against each of the top-3 reranked passages via the NLI ensemble’s max-entail rule, giving a per-claim grounding score

$$g_k = \max_{p \in P_{\text{top3}}} \text{NLI}_{\min}(p \Rightarrow a_k), \quad k = 1, \dots, n_{\text{atoms}}. \quad (4.28)$$

Third, the atomic signal is defined as the weakest link across claims,

$$p_{\text{ground}}^{\text{atomic}} = \min_k g_k, \quad (4.29)$$

so that the addition of a single unsupported claim suppresses the signal. When decomposition fails (parsing error, empty list, or the atomic path is disabled in configuration), the atomic signal falls back to $p_{\text{ground}}^{\text{mean}}$ rather than to zero; this prevents the composite from silently penalising answers whose decomposition simply happened to fail for tokenisation reasons, while ensuring that atomic decomposition adds information when it succeeds.

NLI ensemble aggregation. Every NLI-based signal in this section is computed over an ensemble of two pretrained MNLI models (RoBERTa-Large-MNLI and DeBERTa-v3-Large-MNLI), with two aggregation rules chosen to be consistently conservative. Entailment probabilities are aggregated by minimum across the ensemble ($\text{NLI}_{\min}$): both models must agree that a premise entails a hypothesis before the signal is allowed to be high. Contradiction probabilities are aggregated by maximum ($\text{NLI}_{\max}^{\text{contra}}$): any model flagging a contradiction is enough to raise the veto. The rule is asymmetric because it consistently biases toward not storing a verified answer when the ensemble disagrees, which is the direction the composite is defended against. A single-model fallback is supported in configurations where only one NLI bundle is available, in which case the min and max reduce to that bundle’s own probability; no fallback signal is returned in the zero-model case, and the verifier’s output on those signals defaults to 0.5 for entailment and 0.0 for contradiction so that the composite neither rewards nor punishes in the absence of evidence.

Composite score. The six non-redundant signals from the nine (the three internal collapse to u_{internal} , the three sample-set contribute separately, the three grounding contribute separately) are combined into the composite u_{stored} of Equation 4.6, reproduced here in the form the verifier actually evaluates:

$$u_{\text{stored}} = w_{g,\text{mean}} p_{\text{ground}}^{\text{mean}} + w_{g,\text{atom}} p_{\text{ground}}^{\text{atomic}} + w_{\text{entail}} p_{\text{entail}} + w_{\text{sc}} s_{\text{avg}} + w_{\text{int}} u_{\text{internal}} + w_{\text{se}} (1 - h_{\text{norm}}), \quad (4.30)$$

with weights $(w_{g,\text{mean}}, w_{g,\text{atom}}, w_{\text{entail}}, w_{\text{sc}}, w_{\text{int}}, w_{\text{se}}) = (0.30, 0.15, 0.15, 0.15, 0.15, 0.10)$ summing to unity so the composite remains in $[0, 1]$. The weight allocation is a deliberate reflection of the signal-family design. The largest single weight (0.30) is placed on $p_{\text{ground}}^{\text{mean}}$ because broad passage-level entailment is the most direct test of factual correctness; the four equal weights at 0.15 on $p_{\text{ground}}^{\text{atomic}}$, p_{entail} , s_{avg} , and u_{internal} reflect a deliberately diffuse prior over the secondary signals, with a single joint ablation of the weights reported in Chapter 5 (ablation variant `equal_signal_weights`) testing whether the split is genuinely informative; and the smallest weight (0.10) on $(1 - h_{\text{norm}})$ reflects the intermediate position of semantic entropy, which is informative but noisiest among the six because it is sensitive to cluster-count artefacts at small K . The contradiction probability p_{contra} does not enter the composite because it operates as a veto consumed by the decision tree of Section 4.7 rather than as a smooth signal: averaging a contradiction into the composite would let a strong entailment arithmetically outweigh a contradiction, which is exactly the failure mode the decision tree is designed to reject categorically.

Confabulation early-exit gate. Before the composite is formed, the verifier evaluates one hard safety check:

$$u_{\text{internal}} \geq \tau_{\text{ee,int}} \wedge p_{\text{ground}}^{\text{max}} \leq \tau_{\text{ee,g}} \quad \Rightarrow \quad \text{early exit, DISCARD}, \quad (4.31)$$

with $\tau_{\text{ee,int}} = 0.70$ and $\tau_{\text{ee,g}} = 0.20$. The rule targets the specific failure mode where the base model is highly self-confident in an answer that has essentially no external grounding: not only does no retrieved passage broadly support the answer (this is what $p_{\text{ground}}^{\text{mean}}$ would catch), but not even the single most-supportive passage in the top three reaches a low entailment threshold of 0.20. When both conditions hold, the composite is not even computed, the outcome is set to DISCARD with the early-exit flag set, and all nine signals are still recorded for analysis. The gate is conservative: it fires only when internal confidence is high and grounding is effectively absent, which is exactly the region of the signal space where the composite could otherwise be raised by the sample-set signals into a STORE-range value on an answer the ensemble of NLI models cannot find support for. The precise thresholds were selected so that the gate fires on a manageable minority of Tier 2 and Tier 3 outputs at Cycle 0 and is tracked across cycles as a diagnostic in Chapter 5: a cycle-over-cycle rise would indicate that fine-tuning is introducing confabulations rather than resolving them. Algorithm 4.3 specifies the complete verifier, terminating with the composite u_{stored} and the early-exit flag; the four-outcome decision tree that consumes these values is described in Section 4.7.

The verifier is the most expensive per-query computation in CAEM (several decoder invocations, ensemble NLI calls over up to four premise-hypothesis pairs, one cross-encoder rerank, and a bounded number of additional NLI calls in the atomic path), which is why only Tier 2 and Tier 3 queries invoke it. Tier 1 queries are served from memory under the invariant that every stored entry cleared this verifier at its original storage event and has been re-verified at every cycle boundary since (Section 4.8). The composite u_{stored} and the veto signal p_{contra} are the two quantities consumed by the Stage 6 decision tree; the remaining eight signals are recorded on the episode for downstream calibration, ablation, and retroactive re-verification.

Algorithm 4.3 Stage 5 unified verifier.

Require: Query q ; answer a ; base model f_θ ; SBERT encoder; NLI ensemble; passage retriever and reranker; composite weights as in Equation 4.30.

Ensure: Nine signals, composite u_{stored} , early-exit flag.

- 1: $u_{\text{token}}^{\text{verify}} \leftarrow$ teacher-forced geometric mean log-prob of a under $f_\theta(q)$
 - 2: $\{a_i\}_{i=1}^{K_D} \leftarrow$ generate $K_D = 5$ samples with dropout active at $T = 0.7$
 - 3: $u_{\text{dropout}} \leftarrow 1 - \max_c \#\{i : a_i \equiv a_c\} / K_D$
 - 4: $u_{\text{internal}} \leftarrow \frac{1}{2} u_{\text{token}}^{\text{verify}} + \frac{1}{2} (1 - u_{\text{dropout}})$
 - 5: $\{c_i\}_{i=1}^M \leftarrow$ generate $M = 3$ chains at $T = 0.7$
 - 6: $s_{\text{avg}} \leftarrow \binom{M}{2}^{-1} \sum_{i < j} \hat{c}_i^\top \hat{c}_j$
 - 7: $p_{\text{entail}} \leftarrow \frac{1}{M} \sum_i \text{NLI}_{\min}(c_i \Rightarrow a)$
 - 8: $\{s_j\}_{j=1}^K \leftarrow$ generate $K = 10$ samples at $T = 1.0$
 - 9: cluster $\{s_j\}$ by bidirectional NLI entailment; $h_{\text{norm}} \leftarrow -\sum_c p_c \log_2 p_c / \log_2 K$
 - 10: $P_{\text{top20}} \leftarrow \text{Retrieve}(q, K_{\text{retrieve}} = 20)$; $P_{\text{top3}} \leftarrow \text{Rerank}(q, a, P_{\text{top20}}, K_{\text{rerank}} = 3)$
 - 11: $p_{\text{ground}}^{\text{max}}, p_{\text{ground}}^{\text{mean}} \leftarrow$ max and mean of $\text{NLI}_{\min}(p \Rightarrow a)$ over P_{top3}
 - 12: $p_{\text{contra}} \leftarrow \max_{p \in P_{\text{top3}}} \text{NLI}_{\max}^{\text{contra}}(p \Rightarrow a)$
 - 13: $(a_1, \dots, a_{n_{\text{atoms}}}) \leftarrow \text{Decompose}(a)$ $\triangleright$ fallback to $p_{\text{ground}}^{\text{mean}}$ on failure
 - 14: $p_{\text{ground}}^{\text{atomic}} \leftarrow \min_k \max_{p \in P_{\text{top3}}} \text{NLI}_{\min}(p \Rightarrow a_k)$
 - 15: **if** $u_{\text{internal}} \geq \tau_{\text{ee, int}}$ **and** $p_{\text{ground}}^{\text{max}} \leq \tau_{\text{ee, g}}$ **then**
 - 16: **return** all signals, $u_{\text{stored}} \leftarrow 0$, early-exit = true $\triangleright$ confabulation gate
 - 17: **end if**
 - 18: $u_{\text{stored}} \leftarrow$ Equation 4.30
 - 19: **return** all signals, u_{stored} , early-exit = false
-

4.7 Four-Outcome Storage Decision

Stage 6 consumes three scalars from the verifier (composite u_{stored} , contradiction probability p_{contra} , and the max grounding probability $p_{\text{ground}}^{\text{max}}$) and emits one of four categorical outcomes: STORE, DEFERRED, ABSTAIN, or DISCARD. Stage 7 is the mechanical consumer of that decision: it writes the episode to memory, withholds it from the training pool, returns a principled refusal, or suppresses the generation entirely. The two stages are treated together in this section because they form a single policy surface: the decision tree is meaningful only through the actions it triggers.

Most verification pipelines collapse Stage 6 into a binary store-or-discard rule on a thresholded composite. CAEM retains four outcomes because the composite alone is not sufficient to distinguish between confidently-wrong answers that must be suppressed, uncertain answers that the system should refuse rather than answer, borderline answers that merit another look at the next cycle boundary, and high-quality answers that are safe to commit. Collapsing the middle cases into either STORE or DISCARD either pollutes the Tier 1 path with marginal answers or discards recoverable signal that the retroactive re-verification pass (Section 4.8) would otherwise promote. The four-outcome structure is therefore a recoverability argument: Stage 6 allows a third state (DEFERRED) in which an answer’s quality is good enough to keep around but not good enough to act on, and a fourth state (ABSTAIN) in which the system owns up to uncertainty rather than silently storing or silently discarding.

Contradiction veto. The decision tree begins with a hard veto on contradiction, evaluated before any composite-threshold check:

$$p_{\text{contra}} \geq \tau_{\text{veto}} \quad \Rightarrow \quad \text{DISCARD}, \quad \tau_{\text{veto}} = 0.30. \quad (4.32)$$

The veto is categorical because contradiction is an asymmetric signal: a passage that contradicts the answer is evidence of incorrectness that cannot be arithmetically outweighed by even a very high composite. If a strong entailment on one passage and a strong contradiction on another both enter a weighted average, a high average implies nothing about whether the answer is true. Stage 6 therefore removes p_{contra} from the composite altogether (Section 4.6) and consumes it only as a veto at this step. The threshold $\tau_{\text{veto}} = 0.30$ is deliberately low: an ensemble-max contradiction probability of 0.30 means that at least one NLI bundle assigns 30% probability to a contradiction claim on at least one of the top-three passages. Because of the asymmetric ensemble aggregation (max rather than min for contradiction), this corresponds to a very weak threshold on single-model contradiction detection, which is appropriate for a veto that is supposed to be a tripwire rather than a smooth signal.

Store and Deferred thresholds. When the veto does not fire, the composite u_{stored} is thresholded against two cutoffs in a cascade:

$$\text{STORE if } u_{\text{stored}} \geq \tau_{\text{store}}, \quad \text{DEFERRED if } \tau_{\text{defer}} \leq u_{\text{stored}} < \tau_{\text{store}}, \quad (4.33)$$

with $\tau_{\text{store}} = 0.65$ and $\tau_{\text{defer}} = 0.45$. The two thresholds partition the composite range into three bands. The upper band ($u_{\text{stored}} \geq 0.65$) authorises a commit: the composite is high enough that the answer is treated as verified knowledge, with all the downstream consequences (memory write, eligibility for the training pool, availability for Tier 1 retrieval on future queries). The middle band ($0.45 \leq u_{\text{stored}} < 0.65$) authorises a hold: the answer is plausibly correct (it cleared the 0.45 floor, which is already well above a confidently-wrong answer’s composite) but has not cleared the commit bar. The lower band falls through to the remaining checks. The gap between the two thresholds, and its midpoint value of 0.55, is the single tuning knob with the most influence on the STORE-to-DEFERRED ratio across the evaluation corpus; a tighter gap would collapse DEFERRED into STORE at the risk of training on under-verified material, and a wider gap would shift recoverable answers into the DISCARD state and lose them permanently.

Abstain rule. Below τ_{defer} , the decision tree evaluates a grounding-floor check before falling through to DISCARD:

$$u_{\text{stored}} < \tau_{\text{defer}} \wedge p_{\text{ground}}^{\text{max}} < \tau_{\text{abs}} \quad \Rightarrow \quad \text{ABSTAIN}, \quad \tau_{\text{abs}} = 0.20. \quad (4.34)$$

The rule identifies the specific regime in which the composite is low and the single most-supportive retrieved passage does not meaningfully support the answer: there is neither internal confidence that the answer is right nor external evidence that it is right. In this regime, ABSTAIN is preferred over DISCARD because the two outcomes differ in what the pipeline returns to the user. A DISCARD suppresses the generated answer and emits nothing downstream-usable; an ABSTAIN returns a principled refusal (an explicit “I do not know” response), which is a useful output in factual

Table 4.5: The four-outcome decision tree at Stage 6 and the Stage 7 action it triggers. Conditions are evaluated in order; the first matching condition determines the outcome.

Outcome	Condition (in order)	Stage 7 action
DISCARD (veto)	$p_{\text{contra}} \geq \tau_{\text{veto}} = 0.30$	Suppress output; no memory write.
STORE	$u_{\text{stored}} \geq \tau_{\text{store}} = 0.65$	Write episode; eligible for training if $u_{\text{stored}} \geq \tau_{\text{train}} = 0.75$.
DEFERRED	$\tau_{\text{defer}} \leq u_{\text{stored}} < \tau_{\text{store}}$, i.e. $0.45 \leq u_{\text{stored}} < 0.65$	Pushed to the deferred-entry buffer (Section 4.9) for cycle-boundary reconsideration; not committed to memory and not used for training at the current cycle.
ABSTAIN	$u_{\text{stored}} < \tau_{\text{defer}}$ and $p_{\text{ground}}^{\text{max}} < \tau_{\text{abs}} = 0.20$	Return principled refusal to user; no memory write.
DISCARD (default)	otherwise	Suppress output; no memory write.

QA settings where the failure mode of highest cost is a confident wrong answer rather than a refusal. The grounding-floor threshold $\tau_{\text{abs}} = 0.20$ is intentionally low: it fires only when the top-3 passages collectively fail to produce even a weakly-supporting entailment, which is the signal that no recoverable evidence exists in the current retrieval neighbourhood.

Discard as the default. Every case that does not match the preceding rules falls through to DISCARD. There are two distinct scenarios in which the default fires. The first is the contradiction-veto scenario of Equation 4.32, evaluated first. The second is the intermediate scenario in which the composite is low ($u_{\text{stored}} < \tau_{\text{defer}}$) but retrieval did produce at least one supporting passage ($p_{\text{ground}}^{\text{max}} \geq \tau_{\text{abs}}$): in this case the answer is not good enough to keep, but the presence of some grounding signal means the failure was not of the recoverable-refusal kind (the query did have evidence in the retrieval index; the model simply failed to use it). Reporting a DISCARD rather than an ABSTAIN in this case is the decision tree’s way of distinguishing “we had evidence and still got it wrong” from “we had no evidence at all”, because the two motivate different downstream responses in Chapter 5’s error analysis: the first is evidence of a generation failure, the second of a retrieval failure.

Stage 7 consequences and training eligibility. Stage 7 is deterministic given the decision: STORE writes a complete EPISODICENTRY (Table 4.2) with the full nine-signal record, the composite, the contradiction probability, the early-exit flag, and the decision string itself; DEFERRED retains the episode for re-consideration at the next cycle boundary without committing it as a first-class memory entry; ABSTAIN returns a refusal to the user; DISCARD suppresses the generation. Two additional invariants apply at write time. First, a STORE decision is subject to the novelty filter of Equation 4.8: if the query is too similar to an existing memory

entry, the store is rejected at Stage 7 even though Stage 6 green-lit it, and the existing entry’s retrieval feedback fields are updated instead. Second, an entry is flagged as eligible for the fine-tuning pool only if its composite further clears a training-eligibility threshold $\tau_{\text{train}} = 0.75$; this is a stricter bar than the STORE threshold because the training pool must be composed of only the most well-verified material, while the Tier 1 retrieval path can safely serve from a broader band of quality. The gap between $\tau_{\text{store}} = 0.65$ and $\tau_{\text{train}} = 0.75$ is therefore an architectural commitment: Tier 1 retrieval and training data curation have different quality bars, and conflating them would either starve the training pool or over-serve Tier 1 from marginal memory.

Algorithm 4.4 states the Stage 6 and Stage 7 logic in the evaluation order the code uses. The order is load-bearing: the contradiction veto precedes the composite threshold so that a high-contradiction answer with a high composite is discarded rather than stored; the ABSTAIN check precedes the default DISCARD so that the refusal path is taken whenever grounding is absent rather than merely weak; and the training-eligibility check is applied only after the memory write so that a deferred entry does not accidentally enter the pool.

Algorithm 4.4 Stage 6 decision tree and Stage 7 action.

Require: Composite u_{stored} ; contradiction p_{contra} ; max grounding $p_{\text{ground}}^{\text{max}}$; thresholds $\tau_{\text{veto}} = 0.30$, $\tau_{\text{store}} = 0.65$, $\tau_{\text{defer}} = 0.45$, $\tau_{\text{abs}} = 0.20$, $\tau_{\text{train}} = 0.75$.

Ensure: Outcome $\in \{\text{STORE}, \text{DEFERRED}, \text{ABSTAIN}, \text{DISCARD}\}$; Stage 7 action.

```

1: if  $p_{\text{contra}} \geq \tau_{\text{veto}}$  then
2:   return DISCARD; suppress output
3: end if
4: if  $u_{\text{stored}} \geq \tau_{\text{store}}$  then
5:   if novelty filter passes (Eq. 4.8) then write episode to memory
6:   if  $u_{\text{stored}} \geq \tau_{\text{train}}$  then flag episode as training-eligible
7:   return STORE
8: end if
9: if  $u_{\text{stored}} \geq \tau_{\text{defer}}$  then
10:  push to deferred-entry buffer (Section 4.9); do not commit to memory
11:  return DEFERRED
12: end if
13: if  $p_{\text{ground}}^{\text{max}} < \tau_{\text{abs}}$  then
14:  return a principled refusal to the caller
15:  return ABSTAIN
16: end if
17: suppress output
18: return DISCARD

```

The per-cycle distribution over the four outcomes is one of the principal diagnostics reported in Chapter 5, because it reveals how the verifier is carving up the generation stream and how this carving evolves across cycles: a healthy trajectory shifts mass from DEFERRED into STORE as the base model fine-tunes on the training pool (answers the model was borderline about in cycle n become commits in cycle $n+1$), while a pathological trajectory shifts mass from STORE into DISCARD (the model is drifting into confabulation and the early-exit gate plus contradiction veto are

increasingly firing). The categorical structure of the Stage 6 decision therefore is not only a quality control mechanism at the per-query level but a diagnostic at the cycle level, and Chapter 5 reports the full (cycle $\times$ outcome) distribution as a primary figure.

4.8 Retroactive Re-Verification

Each self-improvement cycle ends with a single sweep over the entire memory in which every stored episode is re-scored under the fine-tuned model and either promoted, pruned, or left untouched. This is Stage 8 in the pipeline and runs once per cycle boundary after fine-tuning (and is therefore distinct from the per-query retrieval-feedback update of Section 4.4, which writes to u_{stored} continuously as Tier 1 answers are consumed). The retroactive pass is the only place in the architecture where the memory’s stored quality estimates are reconciled with the model that now consumes them: without it, a stored composite computed by the Cycle-0 verifier would continue to drive routing for episodes that are now more or less trustworthy under the Cycle-1 model.

Motivation: stored composites are snapshots, not invariants. Every u_{stored} value written to memory is a snapshot of the verifier’s view of a (question, answer) pair at the moment the episode was stored. Three of the nine signals (u_{token} , u_{dropout} , u_{internal}) read the model’s internal state directly and therefore drift whenever the weights drift; two more (s_{avg} , h_{norm}) depend on stochastic decoding and drift with the model’s output distribution. The remaining grounding signals depend on retrieval against a fixed passage corpus and are stable, but the NLI entailment scores between those passages and the generated answer still shift if the answer text itself is regenerated under the new model. Taken together, the stored composite drifts under every fine-tune. Retroactive re-verification is the pass that makes the drift visible and acts on it.

Three-way branch. For each stored episode e with current composite $u_{\text{stored}}^{(t)}$, the pass computes a new composite $u_{\text{stored}}^{\text{new}}$ using the updated verifier and assigns the episode to one of three branches:

$$e \mapsto \begin{cases} \text{prune} & \text{if } u_{\text{stored}}^{\text{new}} < \tau_{\text{retro}}, \\ \text{promote} & \text{if } u_{\text{stored}}^{\text{new}} > u_{\text{stored}}^{(t)} \text{ and } u_{\text{stored}}^{\text{new}} \geq \tau_{\text{retro}}, \\ \text{keep} & \text{otherwise,} \end{cases} \quad \tau_{\text{retro}} = 0.50. \quad (4.35)$$

A pruned episode is hard-deleted from both the FAISS index and the metadata dictionary. A promoted episode has its entire nine-signal record (plus p_{contra} , the decision string, and the early-exit flag) overwritten with the new verifier output, and its composite raised to $u_{\text{stored}}^{\text{new}}$. A kept episode retains its stored composite unchanged and is flagged `retroverified = True` to record that the pass did see it this cycle.

Upward-only update rationale. The promotion rule is intentionally asymmetric: the composite can only be raised by re-verification, never lowered toward zero. The asymmetry is a design commitment, not a numerical accident. If re-verification

were allowed to overwrite the stored composite in either direction, a single noisy verifier pass (stochastic decoding variance in s_{avg} , h_{norm} , and u_{dropout} is non-trivial even on unchanged weights) could demote a stable, high-quality episode below the Tier 1 retrieval band on a coin flip. The memory would then lose accumulated knowledge faster than it could acquire it. The upward-only rule treats the stored composite as a lower bound on the episode’s quality that retroactive passes may raise; the *only* downward movement allowed is the categorical prune at τ_{retro} , which is reserved for cases where the new composite is so low that the episode is beyond rescue. The threshold $\tau_{\text{retro}} = 0.50$ sits strictly above the original DEFER threshold $\tau_{\text{defer}} = 0.45$, so an episode that survives re-verification maintains a margin above what would merely have deferred it at first commit. This margin is deliberate: re-verification is allowed to be stricter than initial storage because the system had already chosen to keep the episode once and should not pay full storage cost for an episode that a re-scoring under current weights would not even have deferred.

Nine-signal refresh on promotion. When an episode is promoted, the update is not limited to the scalar composite. The full verifier record is overwritten: all nine signals, p_{contra} , the STORE/DEFER/ABSTAIN/DISCARD decision string, and the early-exit flag. Downstream consumers of the memory (the calibration diagnostics of Chapter 5, the atomic-fact-distribution analyses, any future ablation that needs to re-partition the memory by verifier decision) therefore always see the latest verifier view of a promoted episode, not a stale mixture of the storage-time record and a later-computed composite. Kept episodes, in contrast, do not refresh their signals; they carry their storage-time record unchanged, with only the **retroverified** flag updated. The asymmetry matches the composite’s: only promotion is a substantive update.

Retroverify versus retrieval feedback. The two memory-update mechanisms have disjoint responsibilities and should not be conflated. Retrieval feedback (Section 4.4, Equation 4.10) is a continuous, per-query EMA update that tracks how often an episode’s Tier 1 answer was accepted by the user or by downstream checks, and is therefore about *usage quality*. Retroactive re-verification is a discrete, per-cycle pass that recomputes the composite from first principles under the current model weights, and is about *model-relative quality*. The two signals can disagree: an episode can be consistently accepted at Tier 1 (high retrieval-feedback u_{stored} movement) while simultaneously drifting below the retroverify threshold when the fine-tuned model now produces a different internal-state reading. In such a case retroverify wins, because the system’s correctness argument rests on what the *current* verifier says about the (question, answer) pair, not on historical usage alone. The code enforces this precedence by running retroverify after fine-tuning and allowing it to remove entries that retrieval feedback alone would have kept.

Abort handling. Retroactive re-verification is skipped on any cycle whose fine-tuning step was rolled back by the forgetting-ratio guard (Section 4.10). The reason is operational rather than conceptual: if the weights were restored to the previous cycle’s checkpoint, re-scoring with unchanged weights produces the same composites that were already stored, up to stochastic-decoding noise, and the pass would therefore pay full compute cost for a no-op update with a small probability of injecting

noise through the promotion path. The orchestrator in `self_improvement.py` therefore guards the `retroverify` call with `if verify_fn is not None and not aborted`, and records the counters $n_{\text{retroverified}} = n_{\text{retropruned}} = 0$ on aborted cycles so that Chapter 5’s `retroverify-dynamics` figures can distinguish aborted cycles from cycles in which no episode moved.

Algorithm 4.5 Stage 8 retroactive re-verification (runs once per cycle boundary after successful fine-tuning).

Require: Memory store $\mathcal{M}$; updated verifier $V^{(t+1)}$; prune threshold $\tau_{\text{retro}} = 0.50$.
Ensure: Memory $\mathcal{M}$ updated in place; counters $n_{\text{up}}, n_{\text{rm}}$.

```

1:  $n_{\text{up}} \leftarrow 0; n_{\text{rm}} \leftarrow 0$ 
2:  $\text{snapshot} \leftarrow \text{list of entry IDs in } \mathcal{M}$  ▷ dict may shrink during iteration
3: for all  $\text{eid} \in \text{snapshot}$  do
4:    $e \leftarrow \mathcal{M}.\text{lookup}(\text{eid});$  if  $e = \perp$  continue
5:   try  $o \leftarrow V^{(t+1)}.\text{verify}(e.\text{question}, e.\text{answer});$  except continue
6:    $u^{\text{new}} \leftarrow o.u_{\text{stored}}$ 
7:   if  $u^{\text{new}} < \tau_{\text{retro}}$  then
8:      $\mathcal{M}.\text{remove}(\text{eid}); n_{\text{rm}} \leftarrow n_{\text{rm}} + 1$ 
9:   else if  $u^{\text{new}} > e.u_{\text{stored}}$  then
10:    overwrite  $e$ ’s nine signals,  $p_{\text{contra}}$ , decision, early-exit flag from  $o$ 
11:     $\mathcal{M}.\text{update\_}u_{\text{stored}}(\text{eid}, u^{\text{new}}); e.\text{retroverified} \leftarrow \text{TRUE}$ 
12:     $n_{\text{up}} \leftarrow n_{\text{up}} + 1$ 
13:   else
14:      $e.\text{retroverified} \leftarrow \text{TRUE}$  ▷ checked this cycle; no update
15:   end if
16: end for
17: return  $(n_{\text{up}}, n_{\text{rm}})$ 

```

The per-cycle pair $(n_{\text{up}}, n_{\text{rm}})$ is the load-bearing diagnostic of the retroactive pass and is reported directly in Chapter 5. The dynamics claim underlying CAEM rests on the expected pattern that n_{up} grows and n_{rm} stays small across cycles: stored answers that the cycle-0 verifier rated marginally are re-rated more confidently by a better-calibrated cycle-1 verifier and by a better-grounded cycle-1 model, which is exactly the compounding signal that distinguishes confidence-aware episodic memory from a static retrieval cache. A trajectory in which n_{rm} grows instead, especially if concentrated on episodes stored in a specific earlier cycle, is the counter-signal: the memory is accumulating material that the current model no longer regards as verified, and the system is in a regime where retroactive pruning is doing more work than retroactive promotion.

4.9 Deferred-Entry Reconsideration

Retroactive re-verification operates on episodes that have already been committed to memory; it is the mechanism by which the stored record is kept in agreement with the current verifier. It has no purchase, however, on queries whose Stage 6 outcome was DEFERRED: a deferred answer is plausibly correct but below the commit bar, and by construction it is not yet in memory for `retroverify` to see. Without a

complementary mechanism, a deferred answer would be silently discarded at the end of Stage 7 and would have no second chance to enter memory even if the fine-tuned cycle- $(c+1)$ verifier, evaluating the same (question, answer) pair, would now rate it above τ_{store} . This section specifies the deferred-entry buffer that holds such answers across cycle boundaries and the reconsideration pass that decides, at each boundary, which of them are promoted to memory, which are discarded as no-longer-plausible, and which are kept for one more cycle.

Why the buffer exists. The thesis’s decision-tree design carves the composite axis into four bands precisely to distinguish answers the system is confident in (STORE) from answers whose confidence is insufficient to commit *now* but whose quality is not low enough to throw away (DEFERRED). The value of the DEFERRED band is entirely conditional on whether those entries get a genuine second look: if the deferral decision is behaviourally identical to a discard, the band collapses to three outcomes and the composite axis is under-utilised. Retroverify alone cannot play this role, because the full-memory sweep of Section 4.8 iterates only over committed episodes. The buffer closes this gap by making deferral a temporally extended decision: Stage 7b stores the (question, answer, embedding, storage-cycle, verifier-record) tuple in a side structure separate from the main FAISS memory, and the start of the next cycle’s retroverify pass has an opportunity to re-score it under the updated model.

Bounded FIFO with time-to-live. The buffer is a bounded first-in-first-out queue with a configured maximum size N_{buf} (the implementation uses $N_{\text{buf}} = 10,000$) and a per-entry time-to-live T measured in cycles (the implementation uses $T = 2$). The FIFO discipline is load-bearing for two reasons. First, deferred output at a rate that exceeds the rate of promotion would otherwise grow the buffer without bound, at which point its re-scoring cost per cycle becomes a denial-of-service against the rest of the cycle-boundary work; the cap makes the per-cycle reconsideration cost $O(N_{\text{buf}})$ in the worst case regardless of how much deferred traffic the verifier has produced. Second, eviction-on-overflow means that the buffer’s content is always the most recently deferred material, which is the material whose verifier record reflects weights closest to the current weights and is therefore the material for which reconsideration is most likely to move the composite. Under capacity pressure the buffer is self-cleaning in the direction of relevance, not in the direction of age. Each entry additionally carries an age counter incremented on every cycle in which reconsideration did not yet promote it, and the TTL bounds this counter: once $\text{age} + 1 \geq T$ the entry is dropped, which prevents a stream of persistently borderline (question, answer) pairs from occupying slots that newer deferrals could use more productively. The cap and the TTL together bound both the memory footprint of the buffer and the temporal horizon over which a deferred answer can be resurrected.

Three-way rule. At each cycle boundary, after retroverify completes (and only if the cycle was not aborted by the forgetting-ratio guard), every entry b in the buffer is re-scored under the updated verifier $V^{(c+1)}$. Writing u^{new} for the new composite

and o^{new} for the new decision string, each entry is routed by

$$b \mapsto \begin{cases} \text{promote} & \text{if } o^{\text{new}} = \text{STORE and } u^{\text{new}} \geq \tau_{\text{store}}, \\ \text{drop} & \text{if } \text{age}(b) + 1 \geq T, \\ \text{keep} & \text{otherwise (requeue with } \text{age}(b) += 1). \end{cases} \quad T = 2. \quad (4.36)$$

The promotion branch fires exactly when the new verifier would have chosen STORE for the same (question, answer) pair at the current cycle: the rule does not bypass the Stage 6 decision tree but replays it. An entry whose new composite is in the ABSTAIN or DISCARD bands is not partially rescued; the only path into memory is a full STORE decision under the current verifier. Conversely, an entry whose new composite is still in the deferred band is not thrown away on that basis alone; only the TTL rule can drop it. The three-way branch mirrors retroverify’s (prune, promote, keep) but with substantively different semantics: retroverify operates on already-committed memory and so can afford upward-only drift with a stricter prune bar; reconsideration operates on held-but-uncommitted entries and so treats the absence of promotion as a vote to keep one more cycle rather than to delete. The asymmetry is intentional and makes the two passes composable rather than redundant.

Novelty filter applies on promotion. When an entry is promoted, it passes through the same novelty filter of Equation 4.8 that governs first-commit Stage 7 storage. Promotion is therefore not an unconditional insert: if the deferred entry is too similar to an existing memory entry (for instance because a later cycle’s STORE decision has since committed a near-duplicate) the promotion is rejected and the entry is dropped. This keeps the novelty guarantee of Section 4.4 invariant under the two entry points to the memory (direct Stage 7 commit and delayed reconsideration promotion); the memory never contains two entries within the similarity cap regardless of which path added them. The deferred entry’s verifier record at promotion time is the freshly recomputed $V^{(c+1)}$ record, not the storage-time record, so a promoted entry enters memory already aligned with the current-cycle verifier and does not require a subsequent retroverify pass to be brought up-to-date.

Scheduling and abort handling. Reconsideration runs as Step 8b of the cycle loop, strictly after retroverify (Step 8a), so that if the pass needs to examine the current memory state (for the novelty check on promotion) it sees a memory that has already been reconciled under the new weights. The pass is guarded by the same abort condition as retroverify: on cycles where the forgetting-ratio guard of Section 4.10 rejects the fine-tune and rolls the weights back to $\theta^{(c-1)}$, reconsideration is skipped and the buffer is left untouched (its entries retain their current age and are re-evaluated at the next successful cycle boundary). The rationale is the same as for retroverify: re-scoring under rolled-back weights produces the same composites as storage time, so the pass would trade compute for a no-op update. The orchestrator in `self_improvement.py` exposes the same three counters as the retroverify pass does: n_{promoted} (entries moved from buffer into memory under the novelty filter), n_{dropped} (entries evicted by the TTL rule), and n_{kept} (entries requeued with an incremented age).

The triple $(n_{\text{pr}}, n_{\text{dr}}, n_{\text{kp}})$ is the per-cycle diagnostic of the reconsideration pass and complements the retroverify pair reported in the previous section. The expected

Algorithm 4.6 Stage 8b deferred-entry reconsideration (runs once per cycle boundary, after retroverify, on non-aborted cycles).

Require: Deferred buffer $\mathcal{B}$; memory store $\mathcal{M}$; updated verifier $V^{(c+1)}$; promote threshold $\tau_{\text{store}} = 0.65$; TTL $T = 2$.

Ensure: Buffer $\mathcal{B}$ and memory $\mathcal{M}$ updated in place; counters $n_{\text{pr}}, n_{\text{dr}}, n_{\text{kp}}$.

```

1:  $n_{\text{pr}} \leftarrow 0; n_{\text{dr}} \leftarrow 0; n_{\text{kp}} \leftarrow 0$ 
2: snapshot  $\leftarrow$  drain  $\mathcal{B}$  into a list  $\triangleright$  iterate over a stable copy
3: for all  $b \in$  snapshot do
4:   try  $o^{\text{new}} \leftarrow V^{(c+1)}.verify(b.question, b.answer)$ ; except requeue  $b$ ;  $n_{\text{kp}} \leftarrow$ 
      $n_{\text{kp}} + 1$ ; continue
5:   if  $o^{\text{new}}.decision = \text{STORE}$  and  $o^{\text{new}}.u_{\text{stored}} \geq \tau_{\text{store}}$  then
6:     if  $\mathcal{M}.is\_novel(b.embedding)$  then
7:        $\mathcal{M}.insert(b.question, b.answer, b.embedding, o^{\text{new}})$ 
8:        $n_{\text{pr}} \leftarrow n_{\text{pr}} + 1$ 
9:     else
10:      drop  $b$  (duplicate);  $n_{\text{dr}} \leftarrow n_{\text{dr}} + 1$ 
11:    end if
12:  else if  $b.age + 1 \geq T$  then
13:    drop  $b$  (TTL expired);  $n_{\text{dr}} \leftarrow n_{\text{dr}} + 1$ 
14:  else
15:     $b.age \leftarrow b.age + 1$ ; requeue  $b$  into  $\mathcal{B}$ ;  $n_{\text{kp}} \leftarrow n_{\text{kp}} + 1$ 
16:  end if
17: end for
18: return  $(n_{\text{pr}}, n_{\text{dr}}, n_{\text{kp}})$ 

```

trajectory under a healthy CAEM run is that n_{pr} grows modestly across early cycles (answers the cycle-0 verifier was borderline about clear τ_{store} after one or two rounds of fine-tuning) while n_{dr} stays comparable to the buffer’s natural turnover under the TTL, and n_{kp} shrinks as the buffer drains. A pathological trajectory in which n_{dr} dominates is the counter-signal: the deferred band is accumulating answers the updated model still cannot commit to, and the composite axis’s four-band design is not yielding the lifecycle it was designed for. Chapter 5’s cycle-level tables report the per-cycle reconsideration triple alongside the retroverify pair so that the two cycle-boundary memory-update passes can be read as a coupled diagnostic rather than as independent counters.

4.10 Self-Improvement Loop

Stage 8 is the mechanism that closes the outer loop of Figure 4.1 and turns CAEM from a confidence-aware retrieval system into a self-improving one. At the end of each cycle it fine-tunes the base model on the reasoning chains of the high-quality episodes in the memory, regularised against the previous cycle’s weights so that new knowledge accumulates without erasing the capabilities the model already had. The stage is deliberately conservative: the training pool is smaller than the Tier 1 retrieval pool (by design, not by accident), the regulariser is a uniform L_2 anchor rather than a full Fisher-weighted penalty, and the post-training capability check is out-of-distribution. If the check fails, the cycle is aborted and the previous checkpoint is restored. These choices together are the safety envelope inside which the compounding dynamics of Section 4.8 are allowed to operate.

Training-pool curation. At the start of each cycle, the training pool is drawn from the memory by filtering on the composite:

$$\mathcal{D}_{\text{train}}^{(c)} = \{ e \in \mathcal{M}^{(c)} \mid u_{\text{stored}}(e) \geq \tau_{\text{train}} \}, \quad \tau_{\text{train}} = 0.75. \quad (4.37)$$

The training threshold $\tau_{\text{train}} = 0.75$ is stricter than the STORE threshold $\tau_{\text{store}} = 0.65$ from Section 4.7. The 0.10 gap between them is an architectural commitment: a stored episode is allowed to serve as a Tier 1 retrieval candidate with a lower composite than it needs to serve as training data, because retrieval acts on the model’s output but training acts on the model’s weights and can cause persistent harm if the data is under-verified. The gap cleanly separates the Tier 1 quality bar from the training-pool quality bar. Within the eligible set, the training target is the episode’s `reasoning_chain` field rather than the `answer` field: fine-tuning on the full chain teaches the model *how* to reason toward the correct answer, not merely to memorise answer strings, and it is the mechanism by which Tier 3 retrieval-augmented reasoning is internalised across cycles. An implementation-level fallback (empty or very short chains are replaced by the answer string alone) prevents training on blank targets without changing the curation rule itself.

Episode-and-general training mix. The episode pool $\mathcal{D}_{\text{train}}^{(c)}$ is combined with a small fraction of general-domain QA pairs before fine-tuning. The mixing rule uses

a fixed ratio $r_{\text{gen}} = 0.10$:

$$|\mathcal{D}_{\text{gen}}^{(c)}| = \min\left(\left\lceil \frac{|\mathcal{D}_{\text{train}}^{(c)}| \cdot r_{\text{gen}}}{1 - r_{\text{gen}}} \right\rceil, |\mathcal{D}_{\text{gen}}^{\text{avail}}|\right), \quad (4.38)$$

where $\mathcal{D}_{\text{gen}}^{\text{avail}}$ is the pool of general QA pairs available to the orchestrator. The ratio is computed relative to the episode count so that episodes remain at $(1 - r_{\text{gen}}) = 90\%$ of the final mix regardless of how large the episode pool has grown by the current cycle. The 10% general share is the minimal diversity buffer consistent with the continual-learning literature: it is small enough that the model’s per-cycle parameter movement is dominated by the memory-derived signal (otherwise memory would not actually drive improvement) and large enough that the model is exposed to prompts outside the verified-episode distribution at every cycle (otherwise drift toward the memory’s topic distribution becomes a systematic risk).

L₂ anchor regularisation. The fine-tuning loss is a standard sequence-level cross-entropy term plus an L₂ anchor on the parameter distance from the previous cycle:

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{CE}}(f_{\theta}(q), y) + \frac{\lambda}{2} \|\theta - \theta_{\text{prev}}\|_2^2, \quad \lambda = 0.01. \quad (4.39)$$

Here θ_{prev} are the weights snapshotted at the start of the current cycle (equivalently, the accepted checkpoint of the previous cycle), and the anchor penalises every parameter uniformly for drifting away from that snapshot. This is the L₂ simplification of elastic weight consolidation [doi:10.1073/pnas.1611835114]: full EWC weights the per-parameter squared deviation by the diagonal of the Fisher information matrix, which requires an extra forward pass over the training set to estimate and roughly doubles the cost of a cycle. The uniform-weight simplification is a principled approximation when the Fisher matrix is close to uniform across parameters, a regime that holds reasonably well for a general-purpose seq2seq model fine-tuned on diverse QA chains. The thesis reports this choice as a documented design decision and the full-EWC comparison as a scheduled ablation in Chapter 5.

Out-of-distribution forgetting guard. After fine-tuning completes, the system evaluates the updated weights on a fixed 200-item MMLU split and compares the post-training accuracy against the pre-training baseline:

$$\rho_{\text{ret}}^{(c)} = \frac{\text{MMLU}(\theta^{(c)})}{\text{MMLU}(\theta^{(c-1)})}, \quad \text{abort iff } \rho_{\text{ret}}^{(c)} < \rho_{\text{min}}, \quad \rho_{\text{min}} = 0.93. \quad (4.40)$$

The guard is relative (a ratio), not absolute, so a model that enters the cycle with a weak MMLU baseline does not trigger an unwarranted abort merely because the baseline was already low. The benchmark is deliberately out-of-distribution: MMLU’s four-choice academic-subject questions are never in the training pool and share no topical overlap with the memory’s accumulated QA distribution. This choice follows directly from the continual-learning framing of doi:10.1073/pnas.1611835114: catastrophic forgetting in the continual-learning sense manifests as loss of *unrelated* general capability, not as erosion of the target task, so an in-distribution probe (TriviaQA, NaturalQuestions) would be systematically insensitive to the failure mode the

guard is supposed to detect. The same MMLU pass also feeds the RET axis of the Composite Evaluation Score (Chapter 3), so the guard incurs no additional evaluation compute: one benchmark pass serves simultaneously as the rollback trigger and as the reported general-capability retention metric.

Rollback and abort semantics. If $\rho_{\text{ret}}^{(c)} < \rho_{\text{min}}$, the weights are restored to θ_{prev} , the cycle is flagged aborted in the cycle record, and both the retroactive re-verification pass of Section 4.8 and the deferred-entry reconsideration pass of Section 4.9 are skipped. The memory is not rolled back: episodes collected during the cycle remain stored, because their Stage 5 verification was performed under the previous-cycle weights and is therefore unaffected by the abort. An aborted cycle is not a failure of the overall pipeline but a commitment to the more conservative of the two available checkpoints; the next cycle restarts from θ_{prev} and sees an enlarged memory that includes the episodes just stored. This asymmetry (memory accumulates across aborts; weights do not) is deliberate: the abort path ensures that weight-level drift cannot compound across cycles, while memory-level knowledge persists and is available to future cycles even if the immediately-following fine-tune failed.

Checkpointing and multi-cycle trajectories. Each cycle saves a complete, independently-loadable checkpoint to a dedicated directory, including the model weights (or the restored θ_{prev} if the cycle aborted), the cycle metadata, and the random seed. The isolation has two operational consequences: a failed cycle cannot overwrite a successful predecessor, and any cycle’s state can be reloaded in isolation for post-hoc comparison, ablation replay, or reproducibility audits. The outer loop is configured for $C = 10$ cycles in the code, of which three to five are evaluated end-to-end in Chapter 5 given compute budget; the choice of evaluated cycle count is reported in Chapter 5 and does not change the loop mechanism. Across the evaluated trajectory, four quantities jointly characterise the cycle and are reported per-cycle in Chapter 5: the size of the training pool $|\mathcal{D}_{\text{train}}^{(c)}|$ (grows as the memory accumulates), the final training loss, the MMLU retention ratio $\rho_{\text{ret}}^{(c)}$ (tracked against the abort threshold), and the retroverify counters $(n_{\text{up}}^{(c)}, n_{\text{rm}}^{(c)})$ from Section 4.8. A healthy trajectory shows the training pool growing monotonically, the loss decreasing or stable, the retention ratio safely above ρ_{min} , and n_{up} outpacing n_{rm} .

The compounding claim that motivates the cycle structure is a prediction, not a construction: nothing in the algorithm forces the model to improve cycle-over-cycle. What the algorithm does force is that improvement, when it happens, is attributable to the two signals the architecture controls (the verified training pool curated from memory, and the retroactive re-verification that follows fine-tuning) rather than to either signal in isolation. The trajectory Chapter 5 reports therefore disaggregates the two: an ablation in which fine-tuning is disabled isolates the memory’s direct effect through Tier 1 retrieval, while an ablation in which retroverify is disabled isolates the fine-tuning’s direct effect on new generations. The joint configuration with both channels active is the CAEM-complete condition, and its cycle trajectory is the headline figure of the Chapter 5 results.

Algorithm 4.7 Stage 8 self-improvement cycle (one invocation per cycle boundary).

Require: Current model $\theta^{(c-1)}$; memory store $\mathcal{M}^{(c)}$; general pool $\mathcal{D}_{\text{gen}}^{\text{avail}}$; thresholds $\tau_{\text{train}} = 0.75$, $\rho_{\text{min}} = 0.93$; hyperparameters $\lambda = 0.01$, $r_{\text{gen}} = 0.10$, epochs $E = 3$, learning rate $\eta = 10^{-5}$, batch $B = 16$.

Ensure: Updated model $\theta^{(c)}$ (or restored $\theta^{(c-1)}$ on abort); cycle record.

```

1:  $\mathcal{D}_{\text{train}}^{(c)} \leftarrow \{e \in \mathcal{M}^{(c)} : u_{\text{stored}}(e) \geq \tau_{\text{train}}\}$  with target =  $e.\text{reasoning\_chain}$ 
2: if  $|\mathcal{D}_{\text{train}}^{(c)}| = 0$  then
3:   return skip cycle (no eligible episodes)
4: end if
5: sample  $\mathcal{D}_{\text{gen}}^{(c)}$  from  $\mathcal{D}_{\text{gen}}^{\text{avail}}$  of size given by Eq. 4.38
6:  $\mathcal{D}^{(c)} \leftarrow \text{shuffle}(\mathcal{D}_{\text{train}}^{(c)} \cup \mathcal{D}_{\text{gen}}^{(c)})$ 
7:  $\theta_{\text{prev}} \leftarrow \theta^{(c-1)}$  ▷ snapshot for L2 anchor and rollback
8:  $m_{\text{pre}} \leftarrow \text{MMLU}(\theta_{\text{prev}}; n = 200)$ 
9: for  $k = 1$  to  $E$  do
10:   for all minibatch  $B \subset \mathcal{D}^{(c)}$  do
11:     compute  $\mathcal{L}(\theta)$  by Eq. 4.39
12:     AdamW step with gradient-norm clip at 1.0
13:   end for
14: end for
15:  $m_{\text{post}} \leftarrow \text{MMLU}(\theta; n = 200)$ ;  $\rho_{\text{ret}} \leftarrow m_{\text{post}}/m_{\text{pre}}$ 
16: if  $\rho_{\text{ret}} < \rho_{\text{min}}$  then
17:    $\theta \leftarrow \theta_{\text{prev}}$  ▷ rollback; cycle aborted
18:   save checkpoint; mark aborted  $\leftarrow \text{TRUE}$ ; return
19: end if
20: save checkpoint; retroverify  $\mathcal{M}^{(c)}$  under new  $\theta$  (Algorithm 4.5)
21: reconsider deferred buffer  $\mathcal{B}^{(c)}$  under new  $\theta$  (Algorithm 4.6)
22: return updated  $\theta^{(c)}$ 

```

4.11 Theoretical Analysis

The preceding sections specified CAEM as a set of decision rules, threshold gates, and update procedures. This section provides the formal underpinning for the central engineering claim those rules are designed to support: that storing only verified episodes and training the model on them produces a pool whose correctness rate is strictly higher than the model’s unconditional accuracy, that the improvement is monotone in the base-model accuracy over repeated cycles, and that the recurrence converges rather than diverging. Three results formalise this claim. The first is a static statement about a single storage step; the second lifts that statement to a trajectory; the third bounds the trajectory’s long-run behaviour. All three share the same two input quantities, so we define them once before stating the results.

Definition 4.1 (Base-model accuracy and verifier balanced accuracy). Fix a cycle c and a finite evaluation set $\mathcal{Q}$ of questions drawn i.i.d. from the task distribution. Let $y(q)$ denote the gold label for question $q \in \mathcal{Q}$ and let $\hat{y}_\theta(q)$ denote the model’s greedy answer under parameters θ . The base-model accuracy is

$$p = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \mathbf{1}[\hat{y}_\theta(q) = y(q)].$$

Write $A(q) = \mathbf{1}[\hat{y}_\theta(q) = y(q)]$ for the per-query correctness indicator. Let $V(q) \in \{0, 1\}$ denote the binarised verifier output, where $V(q) = 1$ means the four-outcome decision emitted STORE (the three non-STORE outcomes DEFERRED, ABSTAIN, and DISCARD all map to $V(q) = 0$, since only STORE entries enter the training pool). The verifier balanced accuracy on $\mathcal{Q}$ is

$$\alpha = \frac{1}{2} \Pr[V = 1 \mid A = 1] + \frac{1}{2} \Pr[V = 0 \mid A = 0].$$

Remark 4.2 (Operating point). The pool-purity identity in Theorem 4.3 is derived under the symmetric operating-point assumption $\Pr[V = 1 \mid A = 1] = \alpha$ and $\Pr[V = 1 \mid A = 0] = 1 - \alpha$. In practice this is enforced by choosing the `retroverify_prune_threshold` so that the verifier’s true-positive and true-negative rates are approximately equal on the labelled purity-validation set; departures from symmetry generalise the identity to the asymmetric form $P = p \cdot \text{TPR} / (p \cdot \text{TPR} + (1 - p) \cdot \text{FPR})$, which is the form the validation script evaluates when the operating-point check fails. The symmetric form is retained below as the primary statement because the threshold is calibrated to realise it.

4.11.1 Data Purity Theorem

Theorem 4.3 (Pool purity). *Under Definition 4.1 and the symmetric operating-point assumption of Remark 4.2, the probability that an accepted episode is correct is*

$$P = \Pr[A = 1 \mid V = 1] = \frac{p \alpha}{p \alpha + (1 - p)(1 - \alpha)}.$$

The pool is strictly purifying, meaning $P > p$, if and only if $\alpha > \frac{1}{2}$.

Proof. By Bayes’ rule,

$$\Pr[A = 1 \mid V = 1] = \frac{\Pr[V = 1 \mid A = 1] \Pr[A = 1]}{\Pr[V = 1]}.$$

The marginal probability of acceptance decomposes by the law of total probability as

$$\Pr[V = 1] = \Pr[V = 1 \mid A = 1] \Pr[A = 1] + \Pr[V = 1 \mid A = 0] \Pr[A = 0] = p\alpha + (1-p)(1-\alpha),$$

where the second equality substitutes $\Pr[A = 1] = p$ together with the symmetric operating-point values $\Pr[V = 1 \mid A = 1] = \alpha$ and $\Pr[V = 1 \mid A = 0] = 1 - \alpha$. Substituting back into Bayes' rule yields the stated identity for P .

For the purification claim, a direct algebraic simplification gives

$$\begin{aligned} P - p &= \frac{p\alpha}{p\alpha + (1-p)(1-\alpha)} - p \\ &= \frac{p\alpha - p[p\alpha + (1-p)(1-\alpha)]}{p\alpha + (1-p)(1-\alpha)} \\ &= \frac{p(1-p)(2\alpha - 1)}{p\alpha + (1-p)(1-\alpha)}. \end{aligned}$$

Since $p \in (0, 1)$ and the denominator is strictly positive, the sign of $P - p$ is controlled by the factor $2\alpha - 1$. Hence $P > p$ if and only if $\alpha > \frac{1}{2}$, with equality $P = p$ precisely at the chance threshold $\alpha = \frac{1}{2}$, where a verifier that is no better than random contributes no purification. $\square$

The identity has a direct engineering reading. The storage gate is a filter whose output distribution is a function of two inputs the pipeline controls independently: p through whatever mechanism raises the base-model accuracy (Tier-1 retrieval hits, Tier-3 grounded generation, or accumulated fine-tuning), and α through the verifier's threshold and signal weights. The theorem says these two contributions compose deterministically to determine pool purity, which means the Chapter 5 empirical test reduces to three measurements per cycle rather than a joint sweep. Chapter 5 reports the comparison $|P_{\text{theory}} - P_{\text{obs}}|$ cycle by cycle in Table 5.12.

Remark 4.4 (Worked values). At $p = 0.60$ and $\alpha = 0.80$, Theorem 4.3 predicts $P = 0.60 \cdot 0.80 / (0.60 \cdot 0.80 + 0.40 \cdot 0.20) = 0.48 / 0.56 \approx 0.857$. Raising α to 0.85 at the same p raises P to ≈ 0.895 ; raising p to 0.70 at $\alpha = 0.80$ raises P to ≈ 0.903 . The two inputs are complements rather than substitutes, which is what justifies investing in both a stronger base model and a stronger verifier rather than one or the other.

4.11.2 Monotonicity of Purity

Theorem 4.5 (Monotone purification across cycles). *Let p_c denote the base-model accuracy at cycle c and α_c the verifier balanced accuracy at the same cycle, and let $P_c = p_c \alpha_c / (p_c \alpha_c + (1 - p_c)(1 - \alpha_c))$ denote the purity predicted by Theorem 4.3. Fix $\alpha_c = \alpha > \frac{1}{2}$ across cycles. Then P_c is strictly increasing in p_c on $(0, 1)$.*

Proof. Write $P(p) = p\alpha / [p\alpha + (1-p)(1-\alpha)]$ and let $D(p) = p\alpha + (1-p)(1-\alpha)$ denote the denominator. Differentiating with respect to p at fixed α via the quotient

rule,

$$\begin{aligned}\frac{dP}{dp} &= \frac{\alpha \cdot D(p) - p\alpha \cdot D'(p)}{D(p)^2} = \frac{\alpha[p\alpha + (1-p)(1-\alpha)] - p\alpha[\alpha - (1-\alpha)]}{D(p)^2} \\ &= \frac{\alpha(1-\alpha)}{[p\alpha + (1-p)(1-\alpha)]^2}.\end{aligned}$$

For $\alpha \in (0, 1)$ the numerator $\alpha(1-\alpha)$ is strictly positive, and for $p \in [0, 1]$ the denominator $D(p)^2$ is strictly positive. Therefore $dP/dp > 0$ on $(0, 1)$, so P_c is strictly increasing in p_c . $\square$

The theorem's role in the cycle structure is to convert a per-step statement (Theorem 4.3) into a trajectory claim. Whenever fine-tuning on the previous cycle's pool raises $p_c > p_{c-1}$ without degrading α_c below $\frac{1}{2}$, the next pool's purity is guaranteed to rise. The fixed- α assumption is not a requirement on the implementation but a simplification for the proof: the actual pipeline also runs retroactive re-verification under the updated model, which tends to raise α_c as well, so the observed trajectory should compound the two effects rather than isolate one. The Chapter 5 validation of this theorem is the purity trajectory across cycles $0 \dots N$ in Figure 5.2, together with a one-sided Spearman's rank correlation against the null hypothesis of no trend.

4.11.3 Convergence of the Self-Improvement Recurrence

Theorem 4.6 (Convergence of the purity recurrence). *Consider the idealised recurrence in which the base-model accuracy at cycle $c + 1$ is bounded above by the previous cycle's pool purity,*

$$p_{c+1} \leq P_c,$$

under the assumption that fine-tuning transmits the pool's correctness rate directly to the next-cycle generations. Fix $\alpha > \frac{1}{2}$ across cycles, and consider the map

$$f : [0, 1] \rightarrow [0, 1], \quad f(p) = \frac{p\alpha}{p\alpha + (1-p)(1-\alpha)}.$$

Then f has exactly two fixed points, located at the endpoints $\{0, 1\}$, and for every starting point $p_0 \in (0, 1)$ the sequence $(p_c)_{c \geq 0}$ defined by $p_{c+1} = f(p_c)$ converges monotonically to 1. The sequence of purity increments $\Delta P_c = P_c - P_{c-1}$ therefore satisfies $\Delta P_c \rightarrow 0$ as $c \rightarrow \infty$.

Proof. The argument has three parts: the fixed points, monotonicity and strict advancement, and convergence.

Fixed points. Direct substitution gives $f(0) = 0$ and $f(1) = 1$, so both endpoints are fixed points. Conversely, solving $f(p) = p$ reduces to

$$p\alpha = p[p\alpha + (1-p)(1-\alpha)],$$

which factors as $p(1-p)(2\alpha-1) = 0$. Since $2\alpha-1 \neq 0$ for $\alpha > \frac{1}{2}$, the only solutions in $[0, 1]$ are $p \in \{0, 1\}$.

Strict advancement on the open interval. Theorem 4.5 shows that f is strictly increasing on $(0, 1)$. The identity established in the proof of Theorem 4.3,

$$f(p) - p = \frac{p(1-p)(2\alpha-1)}{p\alpha + (1-p)(1-\alpha)},$$

is strictly positive for $p \in (0, 1)$ whenever $\alpha > \frac{1}{2}$. Therefore $f(p) > p$ for every interior point.

Convergence. Fix $p_0 \in (0, 1)$. The sequence $p_{c+1} = f(p_c)$ is strictly increasing (by the previous step) and bounded above by 1 (since f maps into $[0, 1]$). By the monotone convergence theorem, $p_c \rightarrow p^*$ for some $p^* \in [0, 1]$. Passing to the limit in $p_{c+1} = f(p_c)$ and using continuity of f gives $p^* = f(p^*)$, so p^* is a fixed point. The only fixed points are $\{0, 1\}$, and because the sequence is strictly increasing with $p_c > p_0 > 0$ for all c , the limit cannot be 0. Therefore $p^* = 1$.

Since $P_c = f(p_c)$ and f is continuous, convergence of p_c to 1 implies $P_c \rightarrow f(1) = 1$, and hence $\Delta P_c = P_c - P_{c-1} \rightarrow 0$. $\square$

Remark 4.7 (Ceiling in practice). The idealisation $p_{c+1} \leq P_c$ is an upper bound, not an equality: real fine-tuning transmits only a fraction of the pool’s correctness rate to next-cycle generations, because the training set is finite, regularisation pulls the weights toward the base model, and the retention guard rolls weights back when MMLU drops. The effective ceiling observed in Chapter 5 is therefore strictly below 1 and is set jointly by the training-pool purity threshold τ_{train} (which determines which of the stored episodes are used for training), the L_2 anchor strength (which determines how far weights can move), and the retention floor (which aborts training before diminishing returns would otherwise take over). Theorem 4.6 guarantees the *shape* of the trajectory, namely a monotone sequence with shrinking increments, but not the ceiling value itself. Chapter 5 reports both the trajectory (Table 5.13) and the fitted ceiling in its convergence-check paragraph.

The three theorems together licence a specific reading of the Chapter 5 trajectory figure. Theorem 4.3 says that the pool’s purity at any single cycle is explained by p and α at that cycle; Theorem 4.5 says that the trajectory rises as long as α stays above chance; and Theorem 4.6 says the rises grow progressively smaller. A trajectory that does not rise, or that rises indefinitely without diminishing returns, would falsify one of the three and would require the implementation to be inspected for a mechanism operating outside the model the theorems assume.

4.12 Implementation

The preceding sections describe CAEM as a set of decision rules, threshold gates, and update procedures. This section records the software and hardware substrate on which those rules are realised, so that any numerical result reported in Chapter 5 can be traced back to a specific library version, a specific model checkpoint, and a specific configuration value. Three organising principles govern the implementation: every numeric constant lives in a single configuration object so that no module silently hard-codes a threshold, concerns are separated by pipeline stage so that swapping a verifier or a router does not require touching unrelated components, and random seeds together with benchmark manifests are pinned so that a cycle run at month m can be repeated at month $m + k$ with the same inputs.

The implementation is written in Python 3.10 and runs on PyTorch with the HuggingFace Transformers and Datasets libraries for model loading and tokenisation. Sentence-level embeddings are produced through the `sentence-transformers` package, nearest-neighbour search is delegated to `faiss-cpu`, and numerical utilities

(temperature calibration fits, exponentially weighted updates, agglomerative clustering for semantic entropy) come from `numpy`, `scipy`, and `scikit-learn`. No custom CUDA kernels are used; every component runs through the public PyTorch API so that the system is portable across any CUDA-capable device that meets the memory requirement below.

CAEM composes three pretrained models that are treated as fixed assets for the duration of an experimental run. Only Flan-T5-Large receives gradient updates in the self-improvement loop described in Section 4.10; the sentence encoder and the NLI cross-encoder are frozen throughout. Table 4.6 summarises the three checkpoints, their parameter counts, and their role in the pipeline.

Table 4.6: Pretrained components and their role in the CAEM pipeline. Parameter counts are for inference; only Flan-T5-Large is trainable during the self-improvement loop.

Component	Checkpoint	Parameters	Role
Base model	Flan-T5-Large [JMLR:v25:23-0870]	780M	Generation (Tier 2, Tier 3), MC Dropout, self-consistency, semantic-entropy sampling; trainable.
Sentence encoder	all-mpnet-base-v2 [reimers-gurevych-2019-sentence-embedder]	110M	768-dim embeddings for memory keys, retrieval similarity, semantic-entropy clustering; frozen.
NLI cross-encoder	RoBERTa-Large-MNLI [liu2019roberta]	355M	Entailment and contradiction probabilities for s_{avg} , p_{entail} , p_{ground} , and p_{contra} ; frozen.

The hardware envelope targets a single NVIDIA RTX 4090 with 24 GB of VRAM paired with a Ryzen-class host CPU and 64 GB of system RAM. Flan-T5-Large is loaded at `bf16` precision to leave headroom for the additional state required during a self-improvement cycle: the L_2 anchor θ_{prev} (Eq. 4.39) must remain resident on the same device as the training parameters so that the anchor penalty can be computed without host-device transfers each step, which doubles the effective weight footprint for the duration of the cycle. The FAISS index for episodic memory is held on the GPU for vector search and spilled to CPU only during the periodic save; the Wikipedia passage index used by Tier 3 RAG lives on CPU throughout because its size (21M vectors; see Section 4.5 for the Tier 3 dispatch conditions and retrieval-augmented generation specification) exceeds what can be co-resident with the generator on a single 24 GB card.

The code is organised by pipeline stage rather than by utility type. Seven sub-modules live under a top-level package: `memory` (episodic store, FAISS wrapper, novelty filter, pruner), `confidence` (pre-routing estimator and the nine-signal post-generation estimator, each implemented as a separate class), `routing` (the adaptive router with its two-mechanism dispatch), `retrieval` (Tier 1 memory retrieval and Tier 3 passage retrieval, sharing the FAISS wrapper), `verification` (NLI ensemble, composite scoring, decision tree, atomic-claim extraction), `training` (self-improvement cycle, L_2 anchor, MMLU retention probe), and `ablation` (flags that disable individual mechanisms for the Chapter 5 ablation grid). A top-level `pipeline.py`

sequences the eight stages per query (Chapter 5), and a top-level `config.py` holds the single configuration dataclass that every submodule reads from. Every numeric constant in the system is annotated with one of three categories documenting its provenance. **Literature-derived constants** ([LIT]) are fixed from the cited paper and are not varied: examples are the MC-Dropout sample count $K = 5$ [pmlr-v48-gal16], the self-consistency chain count $M = 3$ [wang2023selfconsistency], the semantic-entropy sample count $K = 10$ at $T = 1.0$ [farquhar2024semantic], and the 768-dimensional output of `all-mpnet-base-v2`. **Design-choice constants** ([DES]) are principled defaults chosen with reference to the thesis requirements and changed only with ablation evidence: examples are the Tier 1 and Tier 2 similarity thresholds, the novelty threshold 0.95, and the composite weights for u_{stored} . **Calibration constants** ([CAL]) carry an initial value used for the warm-up cycles and are refitted on the Chapter 5 calibration set once the nine signals have accumulated enough observations for the fit to be meaningful; these include the temperature scalar for u_{pre} calibration [pmlr-v70-guo17a] and the post-generation composite weights if the calibration fit recommends deviating from the equal-weight initialisation. Table 4.7 lists the full set of constants together with their category tags; any cell not traceable to this table is a bug in the implementation, and the configuration dataclass asserts on load that every field is present.

Reproducibility is handled at three layers. A single integer seed is passed at pipeline construction and drives every stochastic component: the MC-Dropout masks, the self-consistency and semantic-entropy sampling, the training-pool shuffle, and the 10% general-domain sampler inside `mix`. The configuration dataclass is serialised to JSON alongside every checkpoint so that a cycle- c model can be replayed with exactly the constants it was trained under, even if the repository defaults change. Benchmark manifests (the exact question indices used in each cycle’s 5,000-question draw and each 200-question MMLU probe) are written next to the checkpoint. The one component that remains non-deterministic is low-level CUDA arithmetic, which produces floating-point results that can differ in the final decimal places across runs even under a fixed seed; this is a limitation shared with every system using the same stack, and the variance it induces is smaller than the cycle-over-cycle effects reported in Chapter 5.

4.13 Chapter Summary

This chapter has operationalised every functional requirement stated in Chapter 3 into a concrete mechanism of the CAEM pipeline. Section 4.2 gave the architecture on a single page, fixed the notation used in the rest of the chapter, and traced a single fabricated query end to end through all eight stages. Section 4.3 then specified Stage 1, the two-signal pre-routing confidence estimator u_{pre} that fuses token-level geometric-mean probability with conversation-context similarity in a single encoder forward pass. Section 4.4 specified the episodic memory store that holds Stage 2’s retrieval target: a FAISS IVF-PQ index over SBERT embeddings, paired with the u_{stored} composite stored alongside each vector, the cosine novelty filter, the value-score pruner, and the retrieval feedback loop that updates u_{stored} as entries prove useful or harmful in downstream queries. Section 4.5 combined Stages 2 and 3 into the adaptive router that dispatches each query to Tier 1 (cached answer), Tier 2

Table 4.7: Consolidated hyperparameter reference for CAEM. Category tags follow Section 4.12: [LIT] = literature-fixed, [DES] = design choice, [CAL] = empirically calibrated. Initial values shown for calibration parameters; Chapter 5 reports the post-calibration values.

Parameter	Value	Cat.	Role
<i>Embedding and memory</i>			
SBERT model output dim	768	[LIT]	all-mpnet-base-v2 native dim
Memory capacity $M_{\max}$	10^6	[DES]	Full-scale run cap (§4.4)
FAISS nlist / nprobe	4096/32	[DES]	IVF-PQ memory index
FAISS PQ m / bits	64/8	[DES]	IVF-PQ memory index
IVF-PQ training threshold N_{train}	20,000	[DES]	Promotion from flat to IVF-PQ
Novelty threshold τ_{nov}	0.95	[DES]	Skip-storage cosine gate
Pruning trigger / amount	0.95/0.20	[DES]	Value-score eviction
Recency decay λ	0.01	[DES]	$R = e^{-\lambda \Delta t}$
Value weights I/R	0.70/0.30	[DES]	$V = 0.7 I + 0.3 R$
<i>Pre-routing confidence</i>			
u_{pre} weights (token / C_{conv})	0.60/0.40	[DES]	Two-signal estimator
Temperature T	1.0 init	[CAL]	Platt-style calibration [pmlr-v70-guo17a]
<i>Adaptive router</i>			
Routing weight λ	0.70	[DES]	$S = \lambda s + (1 - \lambda) \hat{u}$
Tier 1 threshold τ_1	0.90	[DES]	Route to retrieval
Tier 2 similarity floor τ_{sim}	0.75	[DES]	Route to zero-shot
Safety floor τ_{safe}	0.60	[DES]	OR-condition $\rightarrow$ Tier 3
<i>Post-generation signals (Tier 2)</i>			
MC-Dropout passes K	5	[LIT]	pmlr-v48-gal16
Self-consistency chains M	3	[LIT]	wang2023selfconsistency
Semantic-entropy samples K	10	[LIT]	farquhar2024semantic
	$T=1.0$		
MC-Dropout rate	0.1	[LIT]	pmlr-v48-gal16
$\hat{u}$ accept threshold	0.60	[DES]	Else escalate to Tier 3
<i>u_{stored} composite (sums to 1.0)</i>			
$w_{\text{pground-mean}}$	0.30	[DES]	External grounding
$w_{\text{pground-atomic}}$	0.15	[DES]	Atomic-claim grounding
w_{entail}	0.15	[DES]	p_{entail} (chain $\rightarrow$ answer)
w_{sc}	0.15	[DES]	s_{avg} pairwise SBERT
$w_{\text{uinternal}}$	0.15	[DES]	$\frac{1}{2} u_{\text{token}} + \frac{1}{2} (1 - u_{\text{dropout}})$
w_{se}	0.10	[DES]	$1 - h_{\text{norm}}$
<i>Verifier decision tree</i>			
Early-exit u_{internal} floor	0.70	[DES]	Confabulation gate
Early-exit $p_{\text{ground-max}}$ ceiling	0.20	[DES]	Confabulation gate
Store threshold τ_{store}	0.65	[DES]	STORE outcome
Defer threshold τ_{defer}	0.45	[DES]	DEFERRED outcome
Abstain ceiling τ_{abs}	0.20	[DES]	ABSTAIN outcome
Contradiction veto τ_{contra}	0.30	[DES]	Overrides all
NLI retrieve / rerank k	20/3	[DES]	Grounding pipeline
<i>Tier 3 RAG</i>			
Passages retrieved k	5	[DES]	DPR convention [karpukhin-etal-2020-dense]
Context budget (tokens)	384	[DES]	Flan-T5 encoder limit
RAG / CoT max new tokens	256/256	[DES]	Generation cap
<i>Self-improvement</i>			
Train threshold τ_{train}	0.75	[DES]	Stricter than τ_{store}
General mix ratio	0.10	[DES]	90:10 curated/general
L_2 anchor λ	0.01	[LIT]	EWC approx. [doi:10.1073/pnas.1611835114]
Learning rate	1×10^{-5}	[DES]	AdamW
Batch size	16	[DES]	Per-device
Epochs per cycle E	3	[DES]	Early-stopping-free
Warmup steps	500	[DES]	Linear schedule
MMLU retention floor ρ_{min}	0.93	[DES]	Abort & rollback
Retroverify prune τ_{retro}	0.50	[DES]	§4.8
Deferred-buffer capacity N_{buf}	10,000	[DES]	Bounded FIFO (§4.9)
Deferred-buffer TTL T	2 cycles	[DES]	Max age before drop (§4.9)
Feedback-loop η	0.01	[DES]	Exponential smoothing
Feedback min retrievals	5	[DES]	Statistical stability
Cycles C	10	[DES]	Length of self-improvement run
Questions per cycle	5,000	[DES]	Chapter 5 protocol

(zero-shot generation), or Tier 3 (retrieval-augmented generation) via a blended score $S = \lambda s + (1 - \lambda) \hat{u}$ with a safety override that forces Tier 3 whenever both inputs fall below a minimum floor.

Sections 4.6 and 4.7 specified the verifier-and-decision block that gates every Tier 2 and Tier 3 output before it can enter memory. The nine-signal verifier composes internal uncertainty, self-consistency, semantic entropy, NLI-based grounding against retrieved passages, and internal entailment into a single composite $u_{\text{stored}} \in [0, 1]$; a confabulation-veto gate fires early when an answer presents as internally confident while failing grounding. Section 4.7 then partitions the u_{stored} axis into four decisions, STORE, DEFERRED, ABSTAIN, and DISCARD, and stated the per-cycle distribution over these outcomes as a primary Chapter 5 diagnostic. Sections 4.8 and 4.9 are the cycle-boundary counterparts of the verifier: the first re-scores every stored entry under the fine-tuned verifier and prunes, promotes, or keeps it via a three-way cascade on τ_{retro} ; the second re-scores the bounded FIFO of DEFERRED entries held from the previous cycle and promotes, drops (TTL-aged), or keeps each via a symmetric three-way cascade. Section 4.10 closed the outer loop: a training pool is curated from retrieved-and-useful episodes that clear τ_{train} , Flan-T5-Large is fine-tuned for E epochs under an L_2 anchor approximating EWC, and the cycle is aborted with a weight rollback whenever a held-out MMLU retention probe falls below ρ_{min} . Section 4.12 then recorded the software substrate (Python 3.10, PyTorch, three pretrained components, single-GPU envelope), the code organisation (seven stage-aligned submodules plus a single configuration dataclass), the hyperparameter provenance conventions ([LIT]/[DES]/[CAL]), and the reproducibility layers (fixed seed, serialised config, pinned benchmark manifests). Table 4.7 consolidates every numeric constant in the system with its category tag.

Chapter 5 reports the experimental trajectory of the system specified here against the acceptance criteria of Chapter 3 (Section 3.3) and the Composite Evaluation Score of Section 3.5.1, together with the sixteen-variant ablation grid of Section 3.6. Chapter 6 then discusses those results in light of the catastrophic failure modes enumerated in Section 3.7 and the ethical considerations of Section 3.8.

Chapter 5

Results and Analysis

5.1 Experimental Setup

Section 5.1 defines the evaluation protocol used across the remainder of this chapter: the six benchmarks CAEM is evaluated on, the metric suite applied to each benchmark at each cycle, and the eight external baselines against which CAEM is compared (B1–B8), plus an optional STaR ceiling reference reported when compute permits. The metric suite is deliberately broader than prior single-number evaluations of hallucination systems: a single exact-match score cannot disentangle accuracy from calibration, grounding, confabulation, or forgetting, and a thesis claiming an *architectural* intervention must demonstrate behaviour on each of these axes separately before any aggregate claim is accepted.

5.1.1 Benchmarks

CAEM is evaluated on six factual-QA benchmarks spanning three in-domain training benchmarks (FEVER **thorne-etal-2018-fever**, TriviaQA **joshi-etal-2017-triviaqa**, and Natural Questions **kwiatkowski-etal-2019-natural**) and three out-of-domain transfer benchmarks (TruthfulQA **lin-etal-2022-truthfulqa**, StrategyQA **10.1162/tacl'a'00370**, and ARC-Challenge **clark2018think**), with MMLU **hendrycks2021measuring** held out as the out-of-distribution retention control for the forgetting-tolerance guard. The confident-error axis of CAEM’s claim is evaluated via the three-sub-type hallucination decomposition (*confabulation*, *ungrounded*, *contradiction*) defined in Section 5.1.2 and reported in Table 5.5, rather than through a dedicated external hallucination-annotated benchmark; this keeps the benchmark panel focused on factual correctness while still measuring the confident-error rate directly against the stored (u_{stored} , correct) stream. This split cleanly separates the in-domain contribution of each architectural mechanism from its zero-shot transfer effect; the ablation study in Section 5.2 exploits this separation. Benchmark selection rationale and dataset statistics are given in Table 5.1.

5.1.2 Metrics

Every cycle of every benchmark produces a per-sample record containing the generated answer, the reasoning chain, the routing tier, wall-clock latency, and the full set of verification signals emitted by the UNIFIEDVERIFIER at Stage 5. The metrics

Table 5.1: Benchmark panel overview. Split source, sample count per cycle, answer format, and metric used for each of the six factual-QA benchmarks plus the MMLU retention control.

Benchmark	Split	N	Answer format	Primary metric
FEVER	paper_dev	500	3-way label	Label accuracy
TriviaQA	validation	500	Free-form	EM / F1
Natural Questions	validation	500	Span	EM / F1
TruthfulQA	generation	500	Free-form	ROUGE-L ≥ 0.15
StrategyQA	dev	500	Binary	Label accuracy
ARC-Challenge	test	500	4-way MCQ	Label accuracy
MMLU (retention)	57 subjects	200	4-way MCQ	Retention ratio ρ

below are computed from these per-sample records and are reported in seven dedicated tables (Tables 5.3–5.14). We organise the suite into seven categories, each answering a distinct evaluation question.

Accuracy and latency (Table 5.3). Primary correctness is reported with exact match (EM) and token-level F1 on free-form answers (TriviaQA, NQ), label accuracy on classification benchmarks (FEVER three-way, ARC-Challenge four-way, StrategyQA binary), and ROUGE-L ≥ 0.15 on TruthfulQA following Lin et al. **lin-etal-2022-truthfulqa**. Because CAEM’s architectural value lies in its routing selectivity, each benchmark’s EM is also reported *stratified by routing tier* – $EM_{T_1}, EM_{T_2}, EM_{T_3}$ – so that a rise in Tier 1 fraction (memory hits) across cycles can be read alongside per-tier correctness rather than being averaged away. Wall-clock latency is reported per tier for the same reason: Tier 1 skips full decoding, so the efficiency claim reduces to “Tier 1 latency is substantially below Tier 2 and Tier 3 latency, and Tier 1 share grows across cycles.” Both quantities are computed via `em.by_tier` and `mean_latency.by_tier` in `eval/metrics.py`.

Calibration (Table 5.4). Calibration is quantified with three complementary metrics computed on the pooled stream of $(u_{\text{stored}}, \text{correct})$ pairs from every stored episode across all six benchmarks at each cycle. The *Expected Calibration Error* (ECE; Guo et al. **pmlr-v70-guo17a**) measures average absolute gap between confidence and accuracy over $M = 10$ equal-width bins. The Brier score **brier1950verification** complements ECE: it is a proper scoring rule that penalises both miscalibration and low resolution, and it does not require bin aggregation – a system that is badly calibrated in ways that counter-balance inside bins will still receive a poor Brier score even as ECE remains low. The *area under the ROC curve* (AUROC) treats u_{stored} as a predictor of correctness and measures rank discrimination independent of scale; a high AUROC with poor ECE indicates that the confidence signal is useful for routing even before calibration. Bin-wise *reliability diagrams* (Figure 5.1) ground these scalars visually using the 10-bin decomposition returned by `reliability_bins`.

Hallucination decomposition (Table 5.5). A single hallucination-rate scalar conflates distinct failure modes, so we report three rates side by side. The *stored-hallucination rate* is the fraction of stored episodes that have $EM = 0$ and $u_{\text{stored}} \geq$

0.50 – i.e., stored-with-confidence errors, which is the quantity the data purity theorem bounds. The *internal-confidence confabulation rate* is the fraction with $\text{EM} = 0$ and $u_{\text{internal}} \geq 0.70$, capturing over-confident generation regardless of whether the episode was stored; this follows the hidden-state-based notion of confabulation used by Farquhar et al. **farquhar2024semantic**. The *semantic-entropy confabulation rate* applies the same threshold on $(1 - h_{\text{norm}})$, where h_{norm} is the normalised semantic entropy of CAEM’s $K = 10$ answer cluster, serving as a direct Farquhar replication on CAEM’s own samples. Reporting all three prevents the case where reducing one form of hallucination silently inflates another. All variants are computed by the generic helper `confabulation_rate` with different (signal, threshold) pairs.

Grounding (Table 5.10). Two grounding diagnostics isolate the retrieval pathway. *Unsupported-correct rate* is the fraction of $\text{EM} = 1$ samples with $p_{\text{ground,max}} < 0.30$ – correct answers that retrieval evidence did not support. *Ungrounded-assertion rate* is the fraction of all samples with $p_{\text{ground,max}} \leq 0.30$ and $u_{\text{stored}} \geq 0.50$ – the evaluation-side mirror of the Stage 5 early-exit confabulation gate ($u_{\text{internal}} \geq 0.70$ and $p_{\text{ground,max}} \leq 0.20$) generalised so the same definition serves both the online gate in Section 4.2 and this offline diagnostic. The first quantifies parametric-memory reliance; the second quantifies confidently-unsupported output regardless of correctness.

Data purity (Table 5.12). The verifier-decision breakdown – counts and fractions of STORE, DEFER, ABSTAIN, and DISCARD decisions per cycle – is reported alongside the empirical verifier true-positive rate α , the base generation accuracy p , the theorem’s predicted post-storage purity $P_{\text{theory}} = p\alpha / (p\alpha + (1-p)(1-\alpha))$, and the measured purity P_{obs} . This table is the empirical instantiation of the purity theorem from Section 4.11.1; all five quantities are computed from the same purity-validation split defined in Section 5.1.1. The STORE denominator is obtained from `decision_breakdown`.

Continual learning (Table 5.11). Forgetting is reported with two instruments, following the dual-metric design in Section 4.10. *MMLU retention* is the ratio of Cycle- n MMLU accuracy (200 questions, shuffled seed 42 across the 57 subjects) to the Cycle-0 pristine-model baseline; it is domain-neutral and never trained on, and is the headline forgetting metric. In addition, we report *backward transfer* and *forward transfer* following the gradient-episodic-memory convention of Lopez-Paz and Ranzato **lopezpaz2017gradient**. Backward transfer averages $R_{T,i} - R_{i,i}$ over earlier benchmarks i ; positive values indicate that later cycles improved earlier benchmarks, and negative values indicate catastrophic forgetting. Forward transfer averages $R_{i-1,i} - \bar{b}_i$ relative to the Cycle-0 baseline and measures whether learning on earlier cycles helps benchmarks not yet reached. Reporting both in addition to MMLU retention answers the per-benchmark-per-cycle drift question that MMLU retention (being a single scalar) cannot address.

Unified efficacy (CAEM Efficacy Score). Because a reviewer must read the seven tables above quickly, we report a single composite scalar – the *CAEM Efficacy*

Score (CES) – as the geometric mean of five orthogonal axes:

$$\text{CES} = (\text{ACC} \cdot \text{EPI} \cdot \text{RET} \cdot \text{CAL} \cdot \text{VER})^{1/5},$$

with ACC the mean EM across all six benchmarks, $\text{EPI} = 1 - (\text{pooled confident-error rate})$, $\text{RET} = \min(\text{MMLU retention ratio}, 1)$, $\text{CAL} = 1 - 2 \min(\text{ECE}, 0.5)$, and $\text{VER} = \max(2\beta - 1, 0)$ where β is the verifier’s balanced accuracy on labelled validation data. Each component is clamped to $[0.01, 1]$ so that a transient failure on one axis does not zero the score, but the geometric form ensures that a genuine collapse on any axis (for instance, catastrophic forgetting destroying RET) pulls CES down rather than being averaged away. CES is a thesis-introduced aggregate used *only* as a visualisation and headline scalar; the seven component tables remain the authoritative quantitative record, and CES is always reported with all five constituent values.

Statistical significance (Table 5.14). All accuracy comparisons between CAEM and baselines are tested for significance using McNemar’s test **mcnemar1947note** with the Edwards continuity correction, applied to paired per-question correct/incorrect outcomes. Non-parametric 95% confidence intervals for EM are computed by bootstrap resampling with $B = 1000$ resamples. Both computations use the existing `mcnemar_test` and `bootstrap_ci` helpers in `eval/metrics.py`, which were written and tested before the experiment phase.

Because CAEM is compared against eight baselines across seven benchmarks, the raw test count is $8 \times 7 = 56$ pairwise McNemar tests per cycle, and a naive $p < 0.05$ threshold would inflate the family-wise error rate to approximately $1 - 0.95^{56} \approx 0.94$. We therefore apply the Holm–Bonferroni step-down correction **holm1979simple** within each benchmark family: the seven per-benchmark CAEM-vs-baseline p -values are sorted in ascending order, and the k -th smallest is compared against $0.05/(m - k + 1)$ where $m = 8$ is the number of baselines. This is uniformly more powerful than plain Bonferroni while preserving the family-wise error rate at $\alpha_{\text{fam}} = 0.05$, and is the accepted correction for structured pairwise-comparison panels of this size. A result is marked “significant” in Table 5.14 only when it survives this step-down correction; results that are directionally positive but fail the correction are reported with both the raw and corrected p -values so the reader can distinguish a suggestive trend from a confirmed effect.

We separately require a minimum practical-significance effect size of $\Delta\text{EM} \geq 0.02$ (absolute, two percentage points on a 500-sample benchmark). McNemar’s statistical test has high power at this sample size and will flag effects as small as $\Delta \approx 0.01$ as significant, but differences below two points are within the cycle- to-cycle variation CAEM itself exhibits on a held-out seed, so we do not claim architectural advantage below that threshold even when the test is significant. Table 5.14 therefore reports two markers: $\dagger$ for Holm-corrected $p < 0.05$, and $\star$ for the conjunction of Holm-corrected $p < 0.05$ *and* $\Delta\text{EM} \geq 0.02$. The thesis’s architectural claims are licensed only by the $\star$ marker; the $\dagger$ marker identifies directionally positive effects whose magnitude is too small for the claim to be practically meaningful, even though the underlying comparison is statistically sound.

Where CAEM loses a comparison (a baseline’s EM exceeds CAEM’s on some benchmark), we apply the same two-stage rule in reverse: a baseline is reported as strictly superior to CAEM on a benchmark only if $\Delta\text{EM} \leq -0.02$ with Holm-corrected

$p < 0.05$. This symmetric framing prevents the write-up from silently suppressing the baseline-wins case, which is a common failure mode in architectural evaluations and one that the ablation discussion (§5.4) treats as legitimate evidence rather than as noise to be explained away.

Summary. The seven metric categories above answer seven distinct evaluation questions: *Is CAEM more accurate and faster than its baselines?* (Table 5.3); *Are CAEM’s confidence signals well-calibrated?* (Table 5.4); *Has CAEM reduced confident errors across every hallucination sub-type?* (Table 5.5); *Is CAEM’s retrieval pathway earning its slot?* (Table 5.10); *Does the purity theorem’s prediction match measurement?* (Table 5.12); *Does self-improvement proceed without catastrophic forgetting in any direction?* (Table 5.11); *Are the claimed improvements statistically significant?* (Table 5.14). CES provides the single-scalar visualisation of these axes. The remainder of this chapter populates each table in turn.

5.1.3 Baselines

CAEM is compared against eight external baselines spanning the three families of prior systems identified in Section 2.2.6: prompt-level reasoning, retrieval augmentation, and training-time self-improvement. All eight share the Flan-T5-Large (780M) backbone wherever a backbone swap would not invalidate the method, and all eight are run on the same six-benchmark factual-QA panel with the same 500-sample draw per benchmark, so any difference in score isolates the contribution of the architectural component that distinguishes the baseline from CAEM.

- B1: Zero-shot Flan-T5-Large.** The same 780M-parameter backbone CAEM fine-tunes, invoked with no CoT prefix, no retrieval, and no verification. This is the empirical floor: any gain CAEM reports must exceed this score, otherwise the gain is attributable to model capacity alone.
- B2: Chain-of-Thought.** Flan-T5-Large with the “Let’s think step by step” prefix **wei2022chain**, no retrieval, no verification. Isolates the contribution of intermediate reasoning on the same backbone.
- B3: DPR-RAG.** Dense Passage Retrieval **karpukhin-etal-2020-dense** with $k = 5$ Wikipedia passages concatenated into a 384-token context and generated from the same Flan-T5-Large backbone **NEURIPS2020’6b493230**. Isolates the contribution of external grounding without any verification or memory.
- B4: CoT + DPR-RAG.** B3 with the CoT prefix prepended. Isolates the contribution of reasoning over retrieved evidence.
- B5: FLARE.** Active retrieval-augmented generation **jiang-etal-2023-active** with confidence floor $\theta = 0.4$ and a 64-token look-ahead window, using the same DPR index as B3 and B4. Isolates the contribution of confidence-gated retrieval, which is the closest prior mechanism to CAEM’s tier routing; the B5-versus-CAEM gap measures whether three tiers and the nine-signal composite meaningfully improve on a single-tier confidence gate.

B6: Vanilla fine-tuning. Flan-T5-Large trained for the same ten cycles as CAEM on the verified episode pool, but without the L_2 anchor, without the MMLU retention guard, and without the episodic memory or the verifier at inference time, so every selected episode is treated as a clean label. Isolates the contribution of CAEM’s retention-protection machinery.

B7: EWC-only fine-tuning. B6 with the L_2 anchor and the MMLU retention guard added [doi:10.1073/pnas.1611835114](https://doi.org/10.1073/pnas.1611835114) but with the verifier, the nine-signal composite, the memory, and the routing all still stripped. Isolates the contribution of the verification-and-memory machinery once retention machinery is held equal.

B8: Self-RAG (citation-only). Asai et al. [asai2024selfrag](https://arxiv.org/abs/2024.01.01) is the most ambitious retrieve-and-critique pipeline in the prior literature but its critic tokens are tied to a LLaMA-family backbone. Re-targeting them to Flan-T5-Large would change enough of the system that the resulting number would measure backbone and prompt-engineering differences more than architectural differences, so Self-RAG is included as a positioning reference rather than as a replicated numerical baseline.

Optional ceiling reference: **STaR NEURIPS2022’639a9a17**, the pure iterative-refinement scheme reviewed in Section 2.2.6, is reported separately where compute permits, and serves as an aspirational upper bound for what a verification-free bootstrap can reach on the same backbone.

Each baseline lacks exactly one of CAEM’s architectural components; the pairwise comparisons in Section 5.3 isolate the contribution of that component. Implementation details, shared backbone model, and compute budget are given in Section 5.1.4.

5.1.4 Implementation Details

All results in this chapter are produced by a single implementation run against a pinned set of dependencies and a fixed hardware envelope. This subsection records the substrate so that any number reported in §5.3–§5.4 is traceable to a specific model checkpoint, GPU, configuration file, and random seed.

Backbone and supporting models. The generator is Flan-T5-Large ([JMLR:v25:23-0870](https://arxiv.org/abs/2025.01.01), 780M parameters), loaded at `bf16` precision. Sentence-level embeddings for episodic memory keys, retrieval similarity, and semantic-entropy clustering are produced by the frozen `all-mpnet-base-v2` sentence encoder ([reimers-gurevych-2019-sentence](https://arxiv.org/abs/2019.01.01), 110M parameters, 768-dim output). NLI entailment and contradiction probabilities feeding the verifier’s nine-signal composite are produced by the frozen `roberta-large-mnli` cross-encoder ([liu2019roberta](https://arxiv.org/abs/2019.01.01), 355M parameters). The same three checkpoints are used by every CAEM variant and by every retrieval-family baseline (B3 DPR-RAG, B4 CoT+RAG, B5 FLARE); backbone identity is held constant across the comparison panel so that score differences isolate architectural contributions rather than capacity differences.

Hardware. Cycle-level training and evaluation run on a single NVIDIA RTX 4090 (24 GB VRAM) rented through Vast.ai, paired with a Ryzen-class host CPU and

64 GB of system RAM. The L_2 anchor θ_{prev} co-resides on GPU with the training parameters so that the anchor penalty can be computed without host-device transfers each step, which doubles the effective weight footprint during a cycle and motivates the `bfloat16` precision choice. The episodic-memory FAISS index stays on GPU for vector search and spills to CPU only for periodic save; the Wikipedia DPR passage index (21M vectors) is held on CPU throughout because its size exceeds what can be co-resident with the generator on a single 24 GB card.

Cycle structure and sample budgets. CAEM is trained for $N = 10$ self-improvement cycles following the schedule in §4.10. At every cycle the generator is evaluated on the full six-benchmark panel with $n_{\text{eval}} = 500$ questions per benchmark drawn under the fixed split assignments documented in Table 5.1. Four additional disjoint sample splits support the per-cycle diagnostics: $n_{\text{cal}} = 500$ for temperature calibration and reliability-diagram fitting, $n_{\text{purity}} = 500$ for the verifier-balanced-accuracy estimate and the $|P_{\text{theory}} - P_{\text{obs}}|$ comparison that populates Table 5.12, $n_{\text{MMLU}} = 200$ shuffled across the 57 MMLU subjects for the retention guard, and $n_{\text{screen}} = 1\,500$ for the Phase 1 screening sweep that ranks the eighteen ablation variants before the confirmatory panel. All five splits are non-overlapping within a benchmark and the split assignment is fixed across cycles and across all ablation variants.

Per-cycle calibration protocol. Calibration is performed per cycle, not only at Cycle 0. Because each cycle’s fine-tuning step changes the generator’s logit distribution, a temperature scalar T fitted at Cycle 0 drifts out of calibration by Cycle 10; prior work on continual-learning calibration documents this drift empirically and recommends re-fitting T whenever the underlying predictive distribution shifts ([ovadia2019trust](#), [thulasidasan2019mixup](#)). We adopt a *conservative* protocol: at every cycle boundary, after fine-tuning and retroactive re-verification have completed but before the next cycle’s online routing begins, T is re-fitted by LBFGS on NLL over the fresh $n_{\text{cal}} = 500$ calibration split following Guo et al. [pmlr-v70-guo17a](#). The nine-signal u_{stored} composite weights, the Platt-like logistic scaler on u_{pre} , and all router thresholds (τ_{store} , τ_{train} , λ_{route} , the OR-condition floor, and ρ_{min}) are *not* re-fitted per cycle; their values are locked at their Cycle 0 fit and held constant across Cycles 1–10. The ablation registry exercises this choice at both ends of the protocol spectrum (`no_per_cycle_recalibration` re-uses the Cycle 0 temperature throughout; `full_per_cycle_recalibration` re-fits T , the u_{stored} signal weights, and the Platt scaler every cycle) so the conservative default can be defended empirically against both a weaker and a stronger comparator.

Fixed thresholds and post-hoc sensitivity analysis. The five scalar thresholds that gate storage, training-pool admission, router dispatch, the OR-condition override, and the forgetting circuit-breaker ($\tau_{\text{store}} = 0.65$, $\tau_{\text{train}} = 0.75$, $\lambda_{\text{route}} = 0.70$ with $(1 - \lambda_{\text{route}}) = 0.30$ for the memory-match term, the pre-routing floor $u_{\text{pre}} \geq 0.60$, and $\rho_{\text{min}} = 0.93$) are set at project start using a combination of literature priors and small calibration-split pilot estimates; see Section 4.12 for the per-threshold provenance. Tuning each threshold directly on the test split would compromise the pre-registered evaluation protocol, so we adopt a post-hoc sensitivity audit instead:

Table 5.2: Threshold sensitivity sweep. Each of the five routing and admission thresholds is perturbed individually across a three-point grid (-10% , default, $+10\%$) while the remaining thresholds are held fixed. ΔCES is reported against the default-row CES. Populated post-main-run from the lab-PC / Vast.ai fallback sweep.

Threshold	Default	-10% ΔCES	$+10\%$ ΔCES	Notes
τ_{store}	0.65	<i>TBD</i>	<i>TBD</i>	Storage admission cut.
τ_{train}	0.75	<i>TBD</i>	<i>TBD</i>	Training-pool admission cut.
λ_{route}	0.70	<i>TBD</i>	<i>TBD</i>	Memory-match vs. confidence weight.
$u_{\text{pre floor}}$	0.60	<i>TBD</i>	<i>TBD</i>	OR-condition RAG override trigger.
ρ_{min}	0.93	<i>TBD</i>	<i>TBD</i>	Forgetting-guard rollback threshold.

once the main cyclic run completes, each of the five thresholds is varied individually across a three-point grid (approximately $\pm 10\%$ of its default) while holding all other values fixed, and the resulting ΔCES is reported in ?? . This design tests whether CES is robust to small perturbations of the threshold vector without optimising on the same evaluation data; sharp CES sensitivity to any single threshold would be flagged in Section 5.4 as a finding that warrants threshold revisit in future work rather than as a correction to the present thesis’s claims. Threshold sensitivity runs are executed after the main Vast.ai budget is spent, on lab-PC hardware (RTX 4090/5090 where available) with Vast.ai as the fallback path.

Cold-start memory. Cycle 0 begins with a pre-populated episodic store of 447 verified episodes drawn from an 80/20 stratification of the three in-domain training benchmarks (FEVER, TriviaQA, Natural Questions) under the same Stage 5 verifier used online, seeded with the pristine Flan-T5-Large backbone. This is documented in detail in the cold-start section of §4.10; the role of the 447 episodes is to give Tier 1 retrieval a non-empty index on the very first query of Cycle 0 so that the routing decision can be exercised from the first sample rather than starting with an inevitable Tier 2/Tier 3 fallback.

Reproducibility. Every cycle run is controlled by a single configuration dataclass (`caem.config.CAEMConfig`) that records every numeric constant in the system; each constant carries a literature [LIT], design [DES], or calibration [CAL] tag per §4.12, and the dataclass asserts at load that no field is missing. Random seeds are pinned at three levels: Python’s `random(seed=42)`, NumPy (`np.random.seed(42)`), and PyTorch (`torch.manual_seed(42)`, `torch.cuda.manual_seed_all(42)`), with `torch.backends.cudnn.deterministic = True` for convolutional kernels. Benchmark splits are materialised once to disk with a frozen sample ordering so that re-running Cycle- n months later under the same seed produces the same per-sample records. The framework versions that any reported number is bound to are Python 3.10, PyTorch 2.1, HuggingFace Transformers 4.36, Datasets 2.16, sentence-transformers 2.3, and faiss-cpu 1.7.

Compute budget summary. One complete CAEM run (Cycles 0–10 across the six benchmarks plus MMLU retention, plus calibration and purity diagnostics) occupies approximately 18 GPU-hours on the RTX 4090 envelope. The external baseline

panel (§5.1.3, B1–B7) adds approximately 16 GPU-hours sequentially (~ 3.5 hours for the five inference baselines B1–B5 combined, ~ 12 hours for the two training baselines B6–B7 combined, each of which runs for the same 10 cycles as CAEM). The optional STaR ceiling reference adds a further ~ 8 GPU-hours when compute permits; Self-RAG is included as a positioning reference rather than a replicated numerical baseline (§5.1.3, B8) and contributes no GPU time. The Phase 1 screening sweep over the eighteen variants at three cycles and $n_{\text{screen}} = 1\,500$ occupies a further ~ 24 GPU-hours before the confirmatory panel narrows to the subset that earns full 10-cycle runs.

5.2 Ablation Methodology

This section describes the ablation protocol that populates the ablation table in §5.4.

Training-time versus inference-time variants. CAEM is a stateful system whose weights evolve across cycles. An ablation that merely disables a mechanism at inference, while evaluating on weights that were already *shaped* by that mechanism in previous cycles, does not measure the mechanism’s contribution; it measures a counterfactual confound. We therefore classify every ablation variant as either *inference-time* (the mechanism can be flipped off without changing training history, so the reference-run weights may be reused) or *training-time* (the mechanism alters storage admission, verification decisions, the self-improvement loss, or any other signal that feeds back into subsequent cycles, so the variant requires its own full cyclic rerun from the pristine baseline). Under strict application of this principle, only two variants are inference-time: the full system and `no_self_improvement`; all 16 remaining variants are training-time and obtain their own 10-cycle trajectory.

Variant registry. A named registry of 18 variants is defined in `caem/ablation/variants.py` and dispatched by `scripts/run_cyclic_ablation.py`. Each cyclic rerun writes its cycle-wise CES axes to `outputs/ablation/<variant>/seed.<N>/ces_axes_per_cycle.json`, and `scripts/aggregate_ablation.py` assembles the per-variant rows for the ablation table. Table 5.2 summarises the registry.

Table 5.3: Eighteen named ablation variants. Training-time variants each receive their own cyclic rerun; inference-time variants reuse the reference-run weights.

Variant	Class	What it disables / forces
full	inference	Reference run; no mechanism disabled.
no_self_improvement	inference	Disable Stage 8 fine-tuning; freeze weights at θ_0 .
no_tier1	training	Force Tier 2/3 for every query; memory never serves answers.
no_retrieval	training	Disable Tier 3 RAG retrieval entirely.
no_verification	training	Accept every generation into memory regardless of u_{stored} .
no_nli	training	Drop the NLI entailment signal from u_{stored} .
no_selfcons	training	Drop the self-consistency signal from u_{stored} .
no_entropy	training	Drop the semantic-entropy signal from u_{stored} .
no_routing	training	Disable OR-condition safety veto; route purely on u_{pre} .
no_cold_start	training	Begin Cycle 0 with empty memory (skip 447-episode seed).
no_abort_guard	training	Disable $\rho < 0.93$ forgetting circuit-breaker.
no_l2_anchor	training	Set $\lambda = 0$; fine-tune without L2 anchor toward θ_{prev} .
no_retroverify	training	Skip end-of-cycle retroactive re-verification pass.
no_memory_prune	training	Disable memory pruning; allow unbounded store growth.
no_mcdropout	training	Drop MC-Dropout signal from u_{pre} .
lower_u_threshold	training	Replace $u_{\text{stored}} \geq 0.50$ admission with $u_{\text{stored}} \geq 0.30$.
no_per_cycle_recalibration	training	Freeze the temperature scalar T at its Cycle 0 fit; skip the per-cycle LBFGS re-fit.
full_per_cycle_recalibration	training	Re-fit T , the nine-signal u_{stored} weights, and the Platt scaler on u_{pre} at every cycle boundary.

Two-phase sweep protocol. Running all 16 training-time variants at the confirmatory configuration ($n = 5000$ SIL per cycle, 10 cycles, 3 seeds) exceeds the project’s self-funded compute envelope. The study therefore uses a two-phase protocol.¹ Phase 1 (self-funded, approximately USD 200) executes a screening pass at $n = 1500$ SIL for 3 cycles on a single seed (42) across the entire 18-variant registry, ranks variants by point-estimate ΔCES , and then commits budget to the confirmatory pass at $n = 5000$ SIL for 10 cycles on a single seed for the top candidates plus the reference run. Phase 2 (post-funding, approximately +USD 700) re-runs the confirmatory pass for seeds 123 and 456 across every cyclic variant, enabling mean $\pm$ std reporting and within-variant McNemar tests. Phase 1 results will be disclosed as point estimates with an explicit single-seed limitation paragraph in §5.4; Phase 2 tables, once collected, will supersede them with variance-bearing reports.

Metric reported. Each variant row reports the five CES axes (ACC, EPI, RET, CAL, VER) and the geometric-mean scalar CES at the final cycle of its own trajectory, plus ΔCES relative to the reference run with independent-seeds standard-deviation propagation once Phase 2 data is available. The per-cycle CES trace is

¹The screening pass is a methodological instrument to allocate confirmatory budget to the highest- ΔCES candidates; screening runs are not reported in the main ablation table. Variant selection is pre-registered in `caem-implementation-log.md` (Session 44) before the confirmatory pass begins.

also plotted for the reference run and for the ablation variants whose ΔCES pattern across cycles is diagnostic (typically `no_self_improvement`, `no_verification`, and `no_retroverify`).

Expected effects and falsifiable predictions

We record below the expected sign, magnitude, and primary CES axis for each variant *before* the Phase 1 runs complete. This pre-registration commits the hypothesis register to paper so that the Phase 1 result discussion in Section 5.4 can be read as confirmation, partial confirmation, or falsification against a fixed set of predictions rather than as a post-hoc rationalisation fitted to the data. The predictions group into four mechanism clusters (self-improvement machinery, memory and routing, verification signals, threshold sensitivity) and are stated at the level of ΔCES bands (< 0.01 = negligible, $0.01\text{--}0.05$ = moderate, > 0.05 = large) and primary axis attribution.

Self-improvement machinery (`no_self_improvement`, `no_retroverify`, `no_l2_anchor`, `no_abort_guard`). `no_self_improvement` freezes weights at θ_0 and is the strictest test of the self-improvement loop’s cumulative contribution; we predict a large negative ΔCES (at least -0.05) driven primarily by the ACC axis, with the per-cycle CES trace flat after Cycle 0 rather than rising, because every later cycle’s improvement in the reference run is by construction attributable to training. Falsification condition: if `no_self_improvement` matches the reference CES within 0.01, the reference trajectory’s rise is not coming from fine-tuning and the thesis’s central engineering claim fails. `no_retroverify` disables the end-of-cycle retroactive re-verification pass; we predict a moderate negative ΔCES (-0.02 to -0.04) driven by EPI and VER (the verifier balanced accuracy α stops improving across cycles because the re-verification signal no longer folds back into the pool), with the purity trajectory in Figure 5.2 exhibiting a visibly shallower slope than the reference. `no_l2_anchor` (setting $\lambda = 0$) should produce a moderate negative RET axis effect (-0.03 to -0.06) as the MMLU retention ratio drifts downward cycle-over-cycle without the anchor pulling weights back toward θ_{prev} ; ACC on the in-domain training benchmarks may be neutral or slightly positive (unconstrained weights can fit the training pool harder), which is precisely why the CES composite’s multiplicative structure matters. `no_abort_guard` disables the $\rho < 0.93$ forgetting circuit-breaker and should produce a catastrophic RET-axis collapse (-0.10 or worse) on a minority of seeds where the training pool happens to carry a distribution shift, with the guard’s absence being the only thing separating a recoverable cycle from an unrecoverable one. Because this effect is rare in expectation, a single-seed Phase 1 run may under-detect it; Phase 2’s independent-seeds report is the correct place to confirm this prediction.

Memory and routing (`no_tier1`, `no_retrieval`, `no_routing`, `no_cold_start`, `no_memory_prune`). `no_tier1` forces every query through Tier 2 or Tier 3 and should produce a moderate negative ΔCES (-0.02 to -0.04) driven jointly by ACC (memory-answerable queries now route to noisier tiers) and latency (not a CES axis but reported in Table 5.3); the effect should grow over cycles as the reference memory accumulates more hits. `no_retrieval` disables Tier 3 RAG entirely and

should produce a large negative EPI and grounding-axis effect on benchmarks that rely on out-of-parametric knowledge (primarily TriviaQA and Natural Questions), with a smaller effect on benchmarks that are answerable from the base model’s own parametric memory (FEVER label accuracy, StrategyQA). The unsupported-correct rate in Table 5.10 should *rise* under `no_retrieval`, not fall, because the remaining correct answers are now entirely parametric and carry no retrieval support by definition. `no_routing` disables the OR-condition safety veto and routes purely on the pre-confidence u_{pre} ; we predict a moderate negative EPI and VER effect (-0.02 to -0.04) concentrated on confident-but-wrong cases, with the stored-hallucination rate in Table 5.5 rising visibly. `no_cold_start` begins Cycle 0 with an empty memory and should produce a moderate negative ΔCES at Cycles 0 and 1 that narrows substantially by Cycle 10 as the memory populates organically; the primary diagnostic is the per-cycle CES trace rather than the final-cycle scalar, and the cycle at which the variant catches up to the reference run measures the cold-start’s marginal value. `no_memory_prune` should produce a small positive or neutral ΔCES on the benchmark panel combined with an unbounded memory size growth and a rising retrieval latency; this is the ablation whose primary cost is system-level rather than accuracy-level, and its “result” is the memory-growth curve in Table 5.8 rather than a CES delta.

Verification signals (`no_verification`, `no_nli`, `no_selfcons`, `no_entropy`, `no_mcdropout`). `no_verification` accepts every generation into memory regardless of u_{stored} and is the hardest test of the verifier’s pool-purification role. We predict a large negative ΔCES (-0.06 or worse) with the EPI and VER axes collapsing and the P_{obs} column in Table 5.12 falling toward p rather than rising above it; this is the direct empirical mirror of Theorem 4.3’s prediction that purification requires $\alpha > \frac{1}{2}$ and that a chance-equivalent verifier contributes no purification. `no_nli`, `no_selfcons`, `no_entropy`, and `no_mcdropout` each drop a single signal from the nine-signal composite; the verifier balanced accuracy α drops in proportion to each signal’s marginal information contribution, and the resulting ΔCES band should be -0.01 to -0.03 per signal. We expect `no_nli` and `no_entropy` to have the largest individual effects because NLI is the dominant grounding-check signal and semantic entropy is the Farquhar-style hallucination detector the literature claims is the highest single-signal predictor; if `no_mcdropout` shows a larger ΔCES than either, the $K = 5$ dropout budget may be doing more work than the literature suggests for Flan-T5-scale models, which would be a finding worth reporting rather than hiding.

Threshold sensitivity (`lower_u_threshold`). `lower_u_threshold` replaces the $u_{\text{stored}} \geq 0.50$ admission cut with $u_{\text{stored}} \geq 0.30$, admitting roughly twice as many episodes per cycle into the training pool. The expected effect is a trade: ACC may rise slightly at early cycles (the training pool is larger and covers more of the distribution), but EPI falls because a lower threshold admits more confident-but-wrong episodes, and RET may degrade because the training pool now contains more near-miss labels. The multiplicative CES composite should trend negative (-0.01 to -0.03) despite the ACC-axis ambiguity, which is the precise motivation for having set the default threshold at 0.50 rather than at a lower value; if `lower_u_threshold` instead produces a *positive* ΔCES , the default threshold is miscalibrated and the Chapter 6 revision agenda should revisit it.

Calibration cadence (`no_per_cycle_recalibration`, `full_per_cycle_recalibration`).

These two variants exercise the conservative-default per-cycle recalibration protocol from Section 5.1.4 at both ends of the spectrum. `no_per_cycle_recalibration` freezes the temperature scalar T at its Cycle 0 fit for all ten cycles. Because fine-tuning shifts the logit distribution at every cycle, the Cycle 0 temperature becomes progressively mis-scaled and the stored ($u_{\text{stored, correct}}$) stream drifts out of calibration. We predict a moderate negative CAL-axis effect (-0.02 to -0.04) that grows monotonically in cycle index, with the ECE trajectory visibly diverging from the reference by Cycle 5 or later. ACC and EPI may absorb a smaller secondary hit because mis-calibrated u_{stored} contaminates the storage decision and changes which episodes end up in the training pool. Falsification condition: if `no_per_cycle_recalibration` matches the reference CAL within 0.01 at Cycle 10, the per-cycle refit is not doing meaningful work on Flan-T5-scale models and the conservative default can be replaced with the cheaper Cycle-0-only protocol in future revisions. `full_per_cycle_recalibration` re-fits not only T but also the nine-signal u_{stored} composite weights and the Platt logistic on u_{pre} at every cycle boundary. This is the stronger comparator. We predict a small CAL-axis improvement over the conservative default ($+0.00$ to $+0.01$) paired with a risk of small negative EPI: refitting u_{stored} weights per cycle against a calibration split risks overfitting to cycle-specific noise, and in small- n regimes the re-fit can produce worse generalisation than a locked Cycle 0 fit. The honest case for the conservative default depends on `full_per_cycle_recalibration` showing no systematic CAL-axis gain over the default; if it instead produces a $+0.02$ or larger gain at Cycle 10, the conservative default is under-calibrated and future work should adopt the fuller re-fit protocol. These two variants are the training-time rerun whose primary value is methodological: they justify the protocol choice in Section 5.1.4 against a stronger and a weaker alternative rather than appealing to literature alone.

Interpreting the full ablation pattern. The above predictions are deliberately specific about sign, axis, and magnitude band so that the Phase 1 result discussion can be graded against them rather than narrated around them. Three macro outcomes are the diagnostic cases. If most predictions land within their bands, the hypothesis register is confirmed and the CES composite is reading the system in the way its design intended. If a minority of predictions are inverted (for instance, `no_retrieval` showing a small rather than large EPI drop on a benchmark where we predicted large), the specific mechanism’s contribution is over-stated in the thesis and the Chapter 6 limitations paragraph should call out the correction explicitly. If the reference trajectory collapses under `no_self_improvement` or `no_verification` by less than 0.05 in ΔCES , the thesis’s central engineering claim requires substantial revision; this is the high-stakes falsification path, and its outcome fixes what the viva defence needs to anticipate.

5.3 Main Results

This section reports the seven metric-category tables introduced in Section 5.1.2, together with the three empirical validations of the theoretical claims stated in Section 4.11. Each table below carries a caption and label already referenced from the

methodology paragraphs; numbers are populated from the confirmatory-pass outputs at Cycle N and are reported with 95% confidence intervals where applicable.

Expected results and pre-registered predictions.

As with the ablation sweep, we commit the expected shape of each headline table before Phase 1 confirmatory runs begin, so that the cross-seed outcome is judged against fixed hypotheses rather than ex-post rationalisation. Predictions are stated as qualitative direction plus numeric bands on the primary axis of each table; we treat any outcome outside the predicted band as a falsification signal that must be explained in Section 5.4 and reconciled with the theorems of Section 4.11.

Headline accuracy (Table 5.3). On the three training benchmarks (FEVER, TriviaQA, NaturalQuestions) we predict pooled EM gains of +0.03 to +0.08 over the strongest external baseline at Cycle N , with the largest gap on TriviaQA and NaturalQuestions where episodic retrieval is expected to dominate and a smaller gap on FEVER where the strong CoT baseline already exploits structured claim verification. Decomposed by tier, $EM_{T_1} > EM_{T_2} > EM_{T_3}$ is expected to hold at every cycle, since Tier 1 answers a queries that the router has already judged high-confidence. A crossover in which EM_{T_2} or EM_{T_3} exceeds EM_{T_1} would indicate router miscalibration and triggers the threshold sensitivity review in Section 5.4. On the three transfer-eval benchmarks (TruthfulQA, StrategyQA, ARC-Challenge) we predict modest positive transfer (+0.01 to +0.03 EM over the strongest baseline) but do not treat a near-zero transfer effect as a falsification: transfer-eval benchmarks test whether SIL-trained episodic content generalises off-distribution, not whether SIL improves in-distribution accuracy.

Calibration (Table 5.4). We predict monotone improvement in ECE and Brier score across cycles $0, \dots, N$, with ECE at Cycle N landing in the 0.03–0.06 band (against a Cycle 0 baseline we expect near 0.10–0.15). AUROC of u_{stored} against correctness is predicted in the 0.70–0.82 band at Cycle N . A non-monotone ECE trajectory, or an AUROC at Cycle N below 0.65, would indicate that verification signals are drifting under self-improvement and would open the question of whether EWC anchoring is insufficient to protect the calibration head. The reliability diagram in Figure 5.1 should show the bin-wise accuracy curve tracking the $y = x$ identity line more closely at Cycle N than at Cycle 0, with any remaining departure concentrated in the high- u_{stored} tail.

Hallucination decomposition (Table 5.5). We predict the total hallucination rate to fall by at least 40% between Cycle 0 and Cycle N , with the largest absolute drop in the *confabulation* sub-type, because Tier-2 retrieval removes the need to fabricate facts that exist in the episodic memory. *Ungrounded* and *contradiction* rates are predicted to fall by smaller margins. A pattern in which the contradiction rate *rises* across cycles would be a serious warning sign for verifier drift and would require rolling back a cycle in the self-improvement loop before the analysis chapter is written.

Routing distribution (Table 5.6). We predict the Tier 1 fraction to rise from a Cycle 0 cold-start level of 10–20% to a Cycle N steady-state level of 35–55%, with a corresponding drop in the Tier 3 fraction. This reflects the expected shift of verified episodes into the fast path as the memory grows. The safety-override fraction should stay bounded, ideally in the 2–8% range: a steady rise in override use would indicate a systematic gap between router confidence and true task difficulty and would motivate revisiting the OR-condition in the router design.

Verifier-signal attribution (Table 5.7). We predict that NLI-entailment and semantic entropy will be the two most informative individual signals, each with per-signal AUROC in the 0.63–0.72 range on pooled correctness. Self-consistency is predicted to sit slightly below in the 0.58–0.66 range, and the token-probability signal is predicted to be the weakest individual contributor. MC-Dropout contribution is treated as an open question: if its standalone AUROC falls below 0.55 the ablation in Section 5.4 should indicate whether it is worth keeping in the composite.

Episodic-memory growth (Table 5.8). We predict sublinear growth in $|\mathcal{M}|$ across cycles, because STORE is the minority outcome once the memory is seeded. By Cycle N we expect $|\mathcal{M}|$ to lie in the 1500–2500 range, with the cumulative DISCARD rate dominating and DEFERRED shrinking relative to early cycles as the verifier calibrates. A runaway memory ($|\mathcal{M}| > 3000$) would suggest the verification threshold is too permissive and would trigger the purity-versus-growth trade-off discussion in Chapter 6.

CES composite (Table 5.9). We predict the CES at Cycle N to exceed the strongest external baseline by +0.04 to +0.10, with the geometric-mean structure ensuring that no single axis (ACC, EPI, RET, CAL, VER) can carry the gain alone: every axis is predicted to improve over Cycle 0, with ACC and EPI leading the delta, RET and CAL in the middle, and VER contributing a smaller increment because its Cycle 0 value is already boosted by the pre-trained NLI cross-encoder. Any axis that regresses across cycles would be flagged as a targeted deficiency to address in future work.

Grounding diagnostics (Table 5.10). We predict the ungrounded-assertion rate to drop across cycles in line with the hallucination trend in Table 5.5. The unsupported-correct rate is a harder call: a small rise is tolerable (the model may correctly answer from parametric memory without pointing to an episode), but a large rise would indicate that self-improvement is encouraging the model to bypass retrieval even when retrieval is available and should be investigated.

Continual learning (Table 5.11). We predict the MMLU retention ratio ρ to stay ≥ 0.95 across cycles under the EWC anchor, and never to fall below 0.90. BWT is predicted to be small and non-negative in expectation. FWT is predicted to be weakly positive on the transfer-eval benchmarks. A retention ratio $\rho < 0.90$ at any cycle would falsify the EWC-anchor design commitment and, per the falsification plan in Section 5.2, would be the signal that SIL is displacing pre-training knowledge rather than augmenting it.

Table 5.4: Headline accuracy and latency. Populated from `outputs/eval/{bm}_cycle{N}.json` via `eval/reporting.build_table_headline`.

Benchmark	EM	EM _{T₁}	EM _{T₂}	EM _{T₃}
<i>[To be populated from confirmatory-pass outputs.]</i>				

Table 5.5: Calibration: ECE, Brier score, and AUROC on pooled (u_{stored} , correct) pairs per cycle. Populated from `eval/metrics.{ece,brier,auroc}`.

Cycle	ECE	Brier	AUROC
<i>[To be populated from confirmatory-pass outputs.]</i>			

Purity validation and trajectory (Table 5.12, Table 5.13, Figure 5.2). The theorem-versus-measurement table is the sharpest test of the theoretical chapter. We predict $|P_{\text{obs}} - P_{\text{theory}}| \leq 0.03$ at every cycle, within the 95% paired-seed CI, with the residual attributable to finite-sample verifier noise. The first-difference column ΔP_c is predicted to be non-negative at every cycle (the monotonicity claim of Theorem 4.5) and to decay toward zero as the purity envelope saturates (the convergence claim of Theorem 4.6). A sustained $|P_{\text{obs}} - P_{\text{theory}}| > 0.05$, or a negative ΔP_c outside the CI, would require a direct revision of the theoretical section; per the review discipline committed in Section 5.4, such a finding is disclosed as the headline outcome regardless of whether CAEM beats the baselines on accuracy.

Significance (Table 5.14). We predict that Holm–Bonferroni-adjusted McNemar $p < 0.05$ will hold for CAEM versus B1–B4 (the test-time-only baselines) on all three training benchmarks, and for CAEM versus B6–B7 (vanilla fine-tuning and EWC-only fine-tuning) on at least two of the three training benchmarks. We do *not* pre-register a prediction of significance against B5 (FLARE) and B8 (Self-RAG citation-only), because those baselines may match or exceed CAEM on the grounding and citation-quality dimensions respectively and the paired contrast there is expected to be close to zero. This asymmetry is deliberate: pre-registering a significance claim only where the mechanism of CAEM differentiates it from the baseline prevents overclaiming on comparisons where the baselines are designed-for-task.

Theoretical validations. Three empirical checks close the chapter. First, the data-purity curve from Theorem 4.3 is plotted against the measured Cycle t pool-purity trace. Second, the monotonicity prediction of Theorem 4.5 is checked by verifying that the measured trace is non-decreasing in the $\alpha > \frac{1}{2}$ regime. Third, the convergence claim of Theorem 4.6 is checked by fitting the late-cycle increments ΔP_t against the predicted envelope.

5.4 Ablation Results

This section reports the eighteen-variant ablation table introduced in Section 5.2. For every variant the table records the CES composite, its five axes, the ΔCES

Table 5.6: Hallucination decomposition by sub-type (confabulation, ungrounded, contradiction).

Cycle	Confabulation	Ungrounded	Contradiction
<i>[To be populated from confirmatory-pass outputs.]</i>			

Table 5.7: Routing-distribution audit. Fraction of queries dispatched to Tier 1, Tier 2, and Tier 3 per cycle, with the safety-override fraction reported separately. Populated from `outputs/eval/routing_{cycle}.json`.

Cycle	Tier 1	Tier 2	Tier 3	Override
<i>[To be populated from confirmatory-pass outputs.]</i>				

against the full-CAEM reference row, and the paired-seed significance test. Screening-pass variants whose ΔCES crosses the significance band are rerun confirmatory and reported with the full five-axis decomposition; remaining variants are reported with CES and ΔCES only. The table scaffold, metric columns, and row ordering are fixed in the ablation registry (Table 5.2); Phase 2 populates the numbers.

5.5 Chapter Summary

Chapter 5 fixes the experimental protocol for the Phase 2 run: a six-benchmark factual-QA panel with an MMLU retention control, the CES geometric-mean composite with its five validated axes, the eight external baselines (plus one optional STaR ceiling), and the eighteen-variant cyclic ablation registry. The seven metric-category tables (Tables 5.3 to 5.9) and the ablation table (Table 5.15) are scaffolded above and populated by Phase 2; the three theoretical validations (Theorems 4.3, 4.5 and 4.6) are checked against the same confirmatory-pass trace. The methodological claims of Chapters 3 and 4 are the objects under test.

Table 5.8: Verifier-signal attribution. Per-signal AUROC against per-query correctness, reported for each of the nine signals in the composite.

Signal	AUROC
<i>[To be populated from confirmatory-pass outputs.]</i>	

Table 5.9: Episodic-memory growth and storage-outcome distribution by cycle (STORE / DEFERRED / ABSTAIN / DISCARD).

Cycle	$ \mathcal{M} $	Store	Deferred	Abstain	Discard
<i>[To be populated from confirmatory-pass outputs.]</i>					

Table 5.10: CES composite and its five axes (ACC, EPI, RET, CAL, VER) per cycle, with 95% paired-seed confidence intervals.

Cycle	CES	ACC	EPI	RET	CAL	VER
<i>[To be populated from confirmatory-pass outputs.]</i>						

Table 5.11: Grounding diagnostics per cycle. Unsupported-correct rate and ungrounded-assertion rate, computed as defined in Section 5.1.2.

Cycle	Unsupported-correct (%)	Ungrounded-assertion (%)
<i>[To be populated from confirmatory-pass outputs.]</i>		

Table 5.12: Continual-learning diagnostics: MMLU retention ratio, backward transfer (BWT), and forward transfer (FWT) per cycle.

Cycle	MMLU retention ρ	BWT	FWT
<i>[To be populated from confirmatory-pass outputs.]</i>			

Table 5.13: Data-purity validation (Theorem 4.3). Per-cycle base accuracy p , verifier true-positive rate α , theorem prediction $P_{\text{theory}} = p\alpha/(p\alpha + (1 - p)(1 - \alpha))$, and measured post-storage purity P_{obs} .

Cycle	p	α	P_{theory}	P_{obs}
<i>[To be populated from confirmatory-pass outputs.]</i>				

Table 5.14: Purity convergence trajectory (Theorem 4.6). Per-cycle observed purity P_c , first-difference increment $\Delta P_c = P_c - P_{c-1}$, and the fitted envelope predicted by the theorem’s upper bound $p_{c+1} \leq P_c$.

Cycle	P_c	ΔP_c	Envelope bound
<i>[To be populated from confirmatory-pass outputs.]</i>			

Table 5.15: Statistical significance of CAEM versus each external baseline (B1–B8). Paired McNemar test with Edwards correction per benchmark, bootstrap-resampled 95% CI for EM difference, and Holm–Bonferroni-adjusted p across the $m = 8$ baseline contrasts within a benchmark.

Baseline	Benchmark	ΔEM	McNemar p	95% CI
<i>[To be populated from confirmatory-pass outputs.]</i>				

[Reliability diagram placeholder. Ten equal-width bins of u_{stored} on the x -axis; observed accuracy on the y -axis. Populated from the pooled $(u_{\text{stored}}, \text{correct})$ stream at Cycle N .]

Figure 5.1: Reliability diagram on pooled $(u_{\text{stored}}, \text{correct})$ pairs at Cycle N , with ten equal-width confidence bins. Departure from the $y = x$ identity line quantifies miscalibration; the ECE value in Table 5.4 is its weighted L_1 norm.

[Purity trajectory placeholder. Cycle index on the x -axis; measured pool purity P_c on the y -axis, with the theorem’s predicted envelope overlaid. Populated from `outputs/purity_validation/theory_validation.json`.]

Figure 5.2: Pool-purity trajectory across cycles $0, \dots, N$. Solid line: measured P_c from the purity-validation split; dashed line: envelope predicted by Theorem 4.5 and Theorem 4.6.

Table 5.16: Ablation results. CES and five-axis decomposition for each variant in the registry, with paired-seed ΔCES against the full-CAEM reference row.

Variant	CES	ACC	EPI	RET	CAL	VER	ΔCES
<i>[To be populated from confirmatory-pass outputs.]</i>							